Land Cover Analysis of Tharaka Sub-county, Tharaka Nithi County, Kenya

Patrick Waweru Mwaura

2026-10-08

Contents

1 1. Study design **1**

2 2. Obtain Kenya's administrative boundaries **2**

3 3. Extract and save Tharaka Sub-county **5**

4 4. Initialize Google Earth Engine **7**

5 5. Convert the study boundary to Earth Engine geometry **8**

6 6. Define seasonal observation windows **8**

7 7. Cloud masking and Landsat band harmonization **9**

8 8. Create seasonal Landsat composites **11**

9 9. Check scene availability **23**

10 10. Optional target-window composite analysis **27**

11 34. Optional labelled Landsat classification (disabled by default) **140**

12 36. Export seasonal Landsat composites for image-led learning **141**

13 37. Local raster processing with QGIS from R **141**

14 38. Image-led unsupervised land-cover learning in R **141**

14.1 Interpretation and validation note 142

1 1. Study design

This study examines land-cover patterns and changes in Tharaka Sub-county, Tharaka Nithi County, Kenya.

The target years are 2000, 2010, 2020 and 2025.

The land-cover classes of interest are:

- Forest
- Bushland
- Grassland
- Cultivated farmland

- Settlement

Four observation windows will be investigated:

Season	Window
Dry	February 10–20
Dry	September 15–25
Wet	May 5–30
Wet	December 5–30

The initial stage retrieves and prepares satellite imagery. Classification and accuracy assessment will follow after suitable training samples have been established.

2 2. Obtain Kenya's administrative boundaries

```
# ========================================================
# 2. DOWNLOAD GADM LEVEL-2 BOUNDARIES
# ========================================================

kenya_adm2 <- geodata::gadm(
  country = "KEN",
  level = 2,
  path = "01_data/boundary"
)

print(kenya_adm2)
```

```
## class       : SpatVector
## geometry    : polygons
## dimensions  : 300, 13  (geometries, attributes)
## extent      : 33.90959, 41.92622, -4.720417, 5.061166  (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (EPSG:4326)
## names       :     GID_2 GID_0 COUNTRY   GID_1  NAME_1 NL_NAME_1          NAME_2 VARNAME_2 NL_NAME_2
## type        :     <chr> <chr>   <chr>   <chr>   <chr>     <chr>           <chr>     <chr>     <chr>
## values      : KEN.1.1_1   KEN   Kenya KEN.1_1 Baringo        NA             805        NA        NA
##               KEN.1.2_1   KEN   Kenya KEN.1_1 Baringo        NA Baringo Central        NA        NA
##               KEN.1.3_1   KEN   Kenya KEN.1_1 Baringo        NA   Baringo North        NA        NA
##               ...
```

```
print(names(kenya_adm2))
```

```
##  [1] "GID_2"     "GID_0"     "COUNTRY"   "GID_1"     "NAME_1"    "NL_NAME_1"
##  [7] "NAME_2"    "VARNAME_2" "NL_NAME_2" "TYPE_2"    "ENGTYPE_2" "CC_2"
## [13] "HASC_2"
```

```
# Inspect all available second-level administrative names
sort(unique(kenya_adm2$NAME_2))
```

```
##   [1] "805"                "Ainabkoi"           "Ainamoi"
##   [4] "Aldai"              "Alego Usonga"       "Awendo"
##   [7] "Bahati"             "Balambala"          "Banissa"
##  [10] "Baringo Central"    "Baringo North"      "Baringo South"
##  [13] "Belgut"             "Bobasi"             "Bomachoge Borabu"
##  [16] "Bomachoge Chache"   "Bomet Central"      "Bomet East"
```

```
##  [19] "Bonchari"            "Bondo"               "Borabu"
##  [22] "Budalangi"           "Bumula"              "Bura"
##  [25] "Bureti"              "Butere"              "Butula"
##  [28] "Buuri"               "Central Imenti"      "Changamwe"
##  [31] "Chepalungu"          "Cherangany"          "Chesumei"
##  [34] "Chuka/Igambang'Ombe" "Daadab"              "Dagoretti North"
##  [37] "Dagoretti South"     "Eldama Ravine"       "Eldas"
##  [40] "Embakasi Central"    "Embakasi East"       "Embakasi North"
##  [43] "Embakasi South"      "Embakasi West"       "Emgwen"
##  [46] "Emuhaya"             "Emurua Dikirr"       "Endebess"
##  [49] "Fafi"                "Funyula"             "Galole"
##  [52] "Ganze"               "Garissa Township"    "Garsen"
##  [55] "Gatanga"             "Gatundu North"       "Gatundu South"
##  [58] "Gem"                 "Gichugu"             "Gilgil"
##  [61] "Githunguri"          "Hamisi"              "Homa Bay Town"
##  [64] "Igembe Central"      "Igembe North"        "Igembe South"
##  [67] "Ijara"               "Ikolomani"           "Isiolo North"
##  [70] "Isiolo South"        "Jomvu"               "Juja"
##  [73] "Kabete"              "Kabondo Kasipul"     "Kabuchai"
##  [76] "Kacheliba"           "Kaiti"               "Kajiado Central"
##  [79] "Kajiado East"        "Kajiado North"       "Kajiado South"
##  [82] "Kajiado West"        "Kaloleni"            "Kamukunji"
##  [85] "Kandara"             "Kanduyi"             "Kangema"
##  [88] "Kangundo"            "Kapenguria"          "Kapseret"
##  [91] "Karachuonyo"         "Kasarani"            "Kasipul"
##  [94] "Kathiani"            "Keiyo North"         "Keiyo South"
##  [97] "Kesses"              "Khwisero"            "Kiambaa"
## [100] "Kiambu"              "Kibra"               "Kibwezi East"
## [103] "Kibwezi West"        "Kieni"               "Kigumo"
## [106] "Kiharu"              "Kikuyu"              "Kilgoris"
## [109] "Kilifi North"        "Kilifi South"        "Kilome"
## [112] "Kimilili"            "Kiminini"            "Kinango"
## [115] "Kinangop"            "Kipipiri"            "Kipkelion East"
## [118] "Kipkelion West"      "Kirinyaga Central"   "Kisauni"
## [121] "Kisumu Central"      "Kisumu East"         "Kisumu West"
## [124] "Kitui Central"       "Kitui East"          "Kitui Rural"
## [127] "Kitui South"         "Kitui West"          "Kitutu Chache North"
## [130] "Kitutu Chache South" "Kitutu Masaba"       "Konoin"
## [133] "Kuresoi North"       "Kuresoi South"       "Kuria East"
## [136] "Kuria West"          "Kwanza"              "Lafey"
## [139] "Lagdera"             "Laikipia East"       "Laikipia North"
## [142] "Laikipia West"       "Laisamis"            "Lamu East"
## [145] "Lamu West"           "Langata"             "Lari"
## [148] "Likoni"              "Likuyani"            "Limuru"
## [151] "Loima"               "Luanda"              "Lugari"
## [154] "Lungalunga"          "Lurambi"             "Maara"
## [157] "Machakos Town"       "Magarini"            "Makadara"
## [160] "Makueni"             "Malava"              "Malindi"
## [163] "Mandera East"        "Mandera North"       "Mandera South"
## [166] "Mandera West"        "Manyatta"            "Maragwa"
## [169] "Marakwet East"       "Marakwet West"       "Masinga"
## [172] "Matayos"             "Mathare"             "Mathioya"
## [175] "Mathira"             "Matuga"              "Matungu"
## [178] "Matungulu"           "Mavoko"              "Mbeere North"
```

```
## [181] "Mbeere South"        "Mbita"               "Mbooni"
## [184] "Mogotio"             "Moiben"              "Molo"
## [187] "Mosop"               "Moyale"              "Msambweni"
## [190] "Mt. Elgon"           "Muhoroni"            "Mukurweini"
## [193] "Mumias East"         "Mumias West"         "Mvita"
## [196] "Mwala"               "Mwatate"             "Mwea"
## [199] "Mwingi Central"      "Mwingi North"        "Mwingi West"
## [202] "Naivasha"            "Nakuru Town East"    "Nakuru Town West"
## [205] "Nambale"             "Nandi Hills"         "Narok East"
## [208] "Narok North"         "Narok South"         "Narok West"
## [211] "Navakholo"           "Ndaragwa"            "Ndhiwa"
## [214] "Ndia"                "Njoro"               "North Horr"
## [217] "North Imenti"        "North Mugirango"     "Nyakach"
## [220] "Nyali"               "Nyando"              "Nyaribari Chache"
## [223] "Nyaribari Masaba"    "Nyatike"             "Nyeri Town"
## [226] "Ol Jorok"            "Ol Kalou"            "Othaya"
## [229] "Pokot South"         "Rabai"               "Rangwe"
## [232] "Rarieda"             "Rongai"              "Rongo"
## [235] "Roysambu"            "Ruaraka"             "Ruiru"
## [238] "Runyenjes"           "Sabatia"             "Saboti"
## [241] "Saku"                "Samburu East"        "Samburu North"
## [244] "Samburu West"        "Seme"                "Shinyalu"
## [247] "Sigor"               "Sigowet/Soin"        "Sirisia"
## [250] "Sotik"               "South Imenti"        "South Mugirango"
## [253] "Soy"                 "Starehe"             "Suba"
## [256] "Subukia"             "Suna East"           "Suna West"
## [259] "Tarbaj"              "Taveta"              "Teso North"
## [262] "Teso South"          "Tetu"                "Tharaka"
## [265] "Thika Town"          "Tiaty"               "Tigania East"
## [268] "Tigania West"        "Tinderet"            "Tongaren"
## [271] "Turbo"               "Turkana Central"     "Turkana East"
## [274] "Turkana North"       "Turkana South"       "Turkana West"
## [277] "Ugenya"              "Ugunja"              "unknown 1"
## [280] "unknown 2"           "unknown 3"           "unknown 4"
## [283] "unknown 5"           "unknown 6"           "unknown 7"
## [286] "Uriri"               "Vihiga"              "Voi"
## [289] "Wajir East"          "Wajir North"         "Wajir South"
## [292] "Wajir West"          "Webute West"         "Webuye East"
## [295] "West Mugirango"      "Westlands"           "Wundanyi"
## [298] "Yatta"
```

```
# Identify possible Tharaka matches
grep(
  "Tharaka",
  unique(kenya_adm2$NAME_2),
  value = TRUE,
  ignore.case = TRUE
)
```

```
## [1] "Tharaka"
```

```
# List level-2 units within Tharaka-Nithi County
sort(unique(
  kenya_adm2$NAME_2[
    kenya_adm2$NAME_1 == "Tharaka-Nithi"
```

```
  ]
))
```

```
## [1] "Chuka/Igambang'Ombe" "Igembe South"        "Maara"
## [4] "Tharaka"
```

3 3. Extract and save Tharaka Sub-county

```
# ========================================================
# 3. EXTRACT THE STUDY AREA
# ========================================================

tharaka <- kenya_adm2[
  kenya_adm2$NAME_2 == "Tharaka" &
    kenya_adm2$NAME_1 == "Tharaka-Nithi",
]

if (nrow(tharaka) != 1) {
  stop(
    "Expected exactly one Tharaka feature. ",
    "Inspect the administrative names printed above ",
    "and adjust the selection before continuing."
  )
}

study_area <- tharaka

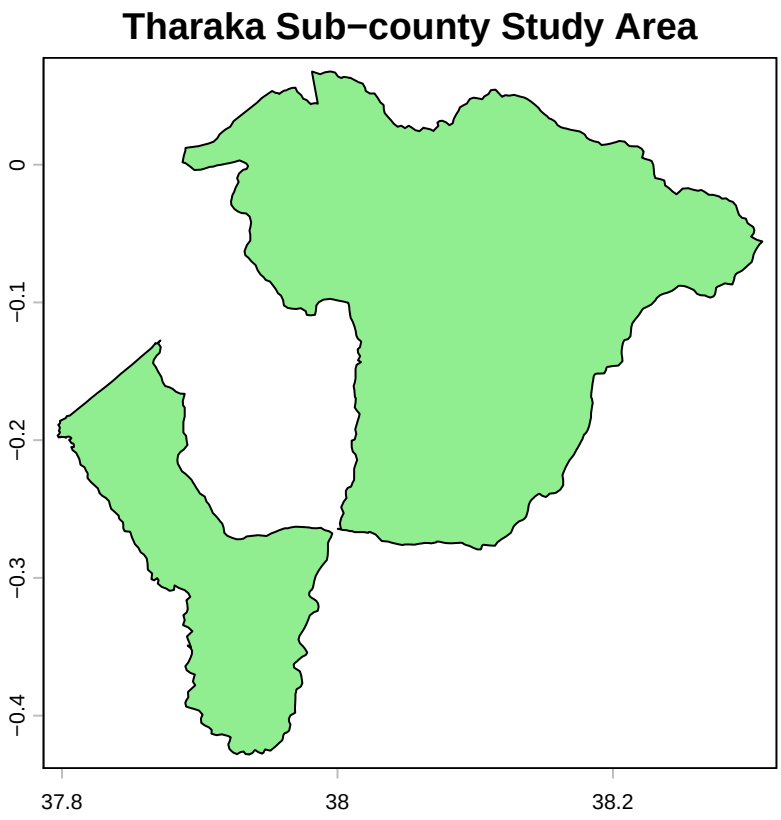
# Inspect the boundary
print(study_area)
```

```
## class       : SpatVector
## geometry    : polygons
## dimensions  : 1, 13  (geometries, attributes)
## extent      : 37.79683, 38.30847, -0.4282358, 0.067688  (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (EPSG:4326)
## names       :      GID_2 GID_0 COUNTRY    GID_1        NAME_1 NL_NAME_1  NAME_2 VARNAME_2 NL_NAME_2
## type        :      <chr> <chr>   <chr>    <chr>         <chr>     <chr>   <chr>     <chr>     <chr>
## values      : KEN.41.4_1   KEN   Kenya KEN.41_1 Tharaka-Nithi        NA Tharaka        NA        NA
```

```
print(terra::crs(study_area))
```

```
## [1] "GEOGCRS[\"WGS 84\",\n    DATUM[\"World Geodetic System 1984\",\n        ELLIPSOID[\"WGS 84\",63
```

```
plot(
  study_area,
  col = "lightgreen",
  main = "Tharaka Sub-county Study Area"
)
```




```
# Save vector boundary
terra::writeVector(
  study_area,
  "01_data/boundary/Tharaka_SubCounty.shp",
  overwrite = TRUE
)

# Save native R object
saveRDS(
  study_area,
  "01_data/boundary/tharaka_study_area.rds"
)

# Calculate approximate area
area_km2 <- sum(
  terra::expanse(study_area, unit = "km")
)

print(area_km2)
```

```
## [1] 1211.373
```

```
# Convert to sf and use geographic coordinates for Earth Engine
study_area_sf <- sf::st_as_sf(study_area)

study_area_sf <- sf::st_transform(
  study_area_sf,
```

```
  4326
)

print(sf::st_bbox(study_area_sf))
```

```
##       xmin       ymin       xmax       ymax
## 37.7968292 -0.4282358 38.3084717  0.0676880
```

4 4. Initialize Google Earth Engine

The Earth Engine API is installed in the dedicated `rgee` Python environment. A previous initialization attempt reached the API but reported that the Cloud Asset root was missing.

The code below catches that particular initialization error and checks whether a basic Earth Engine request still works. It does not suppress a genuine failure to access Earth Engine.

```
# ============================================================
# 4. EARTH ENGINE CONNECTION
# ============================================================

ee_project <- "thinking-leaf-305215"

# Import Earth Engine API
ee <- reticulate::import("ee", convert = FALSE)

# Initialize Earth Engine directly; bypass rgee asset-root prompts
ee$Initialize(project = ee_project)
```

```
## None
```

```
# ------------------------------------------------------------
# Test 1: Basic API request
# ------------------------------------------------------------

ee_test <- reticulate::py_to_r(
  ee$Number(1)$getInfo()
)

stopifnot(as.numeric(ee_test) == 1)

message("Basic Earth Engine request succeeded.")

# ------------------------------------------------------------
# Test 2: Landsat collection access
# ------------------------------------------------------------

landsat_test <- reticulate::py_to_r(
  ee$
    ImageCollection("LANDSAT/LC08/C02/T1_L2")$
    limit(1)$
    size()$
    getInfo()
)

stopifnot(as.numeric(landsat_test) == 1)
```

```r
message("Landsat collection access succeeded.")
message("Earth Engine project: ", ee_project)
```

5 5. Convert the study boundary to Earth Engine geometry

```r
# ============================================================
# 5. CONVERT LOCAL BOUNDARY TO EARTH ENGINE GEOMETRY
# ============================================================

roi_fc <- rgee::sf_as_ee(
  study_area_sf,
  via = "getInfo"
)

roi <- roi_fc$geometry()

# Confirm that a collection can be queried over the boundary
boundary_test <- ee$ImageCollection(
  "LANDSAT/LC08/C02/T1_L2"
)$filterBounds(roi)$limit(1)$size()$getInfo()

print(boundary_test)
```

```
## 1
```

6 6. Define seasonal observation windows

```r
# ============================================================
# 6. TARGET YEARS AND SEASONAL WINDOWS
# ============================================================

years <- c(2000, 2010, 2020, 2025)

windows <- tibble::tibble(
  window = c(
    "dry_feb",
    "dry_sep",
    "wet_may",
    "wet_dec"
  ),
  season = c(
    "Dry",
    "Dry",
    "Wet",
    "Wet"
  ),
  start_month = c(2, 9, 5, 12),
  start_day = c(10, 15, 5, 5),
  end_month = c(2, 9, 5, 12),
  end_day = c(20, 25, 30, 30)
)
```

```r
print(windows)
```

```
## # A tibble: 4 x 6
##   window  season start_month start_day end_month end_day
##   <chr>   <chr>        <dbl>     <dbl>     <dbl>   <dbl>
## 1 dry_feb Dry              2        10         2      20
## 2 dry_sep Dry              9        15         9      25
## 3 wet_may Wet              5         5         5      30
## 4 wet_dec Wet             12         5        12      30
```

7 7. Cloud masking and Landsat band harmonization

The code uses Landsat Collection 2 Level-2 surface reflectance. The common output bands are blue, green, red, near-infrared, shortwave infrared 1 and shortwave infrared 2.

```r
# ============================================================
# 7. LANDSAT MASKING AND SCALING
# ============================================================

mask_scale_l57 <- function(img) {

  qa <- img$select("QA_PIXEL")

  # Mask fill, dilated cloud, cirrus bit, cloud,
  # cloud shadow and snow flags.
  mask <- qa$bitwiseAnd(63)$eq(0)

  saturation <- img$select("QA_RADSAT")$eq(0)

  img$
    updateMask(mask)$
    updateMask(saturation)$
    select(
      c(
        "SR_B1", "SR_B2", "SR_B3",
        "SR_B4", "SR_B5", "SR_B7"
      ),
      c(
        "blue", "green", "red",
        "nir", "swir1", "swir2"
      )
    )$
    multiply(0.0000275)$
    add(-0.2)
}

mask_scale_l89 <- function(img) {

  qa <- img$select("QA_PIXEL")

  mask <- qa$bitwiseAnd(63)$eq(0)

  saturation <- img$select("QA_RADSAT")$eq(0)
```

```
  img$
    updateMask(mask)$
    updateMask(saturation)$
    select(
      c(
        "SR_B2", "SR_B3", "SR_B4",
        "SR_B5", "SR_B6", "SR_B7"
      ),
      c(
        "blue", "green", "red",
        "nir", "swir1", "swir2"
      )
    )$
    multiply(0.0000275)$
    add(-0.2)
}
```

```
# ============================================================
# SECTION 7: TARGET AND EXPANDED SEASONAL WINDOWS
# ============================================================

library(dplyr)
library(purrr)
library(tibble)
library(readr)
library(reticulate)

years <- c(2000, 2010, 2020, 2025)

# Original target windows
windows <- tibble::tribble(
  ~window,    ~season, ~start_month, ~start_day, ~end_month, ~end_day,
  "dry_feb",  "Dry",   2,            10,         2,          20,
  "dry_sep",  "Dry",   9,            15,         9,          25,
  "wet_may",  "Wet",   5,            5,          5,          30,
  "wet_dec",  "Wet",  12,            5,         12,          30
)

# Option B parameters
buffer_days <- 30

# Coverage criteria for this preliminary assessment
min_scenes <- 2
coverage_threshold <- 0.70

# Expand each target window by 30 days at each end.
# Earth Engine filterDate() uses an exclusive end date.
window_schedule <- tidyr::crossing(
  year = years,
  windows
) %>%
  mutate(
    target_start = as.Date(sprintf(
```

```
      "%04d-%02d-%02d", year, start_month, start_day
    )),
    target_end = as.Date(sprintf(
      "%04d-%02d-%02d", year, end_month, end_day
    )),
    expanded_start = target_start - buffer_days,
    expanded_end = target_end + buffer_days,
    expanded_end_exclusive = expanded_end + 1,
    key = paste(year, window, sep = "_")
  )

print(
  window_schedule %>%
    select(
      year, window, season, target_start, target_end,
      expanded_start, expanded_end
    )
)
```

```
## # A tibble: 16 x 7
##     year window  season target_start target_end expanded_start expanded_end
##    <dbl> <chr>   <chr>  <date>       <date>     <date>         <date>
##  1  2000 dry_feb Dry    2000-02-10   2000-02-20 2000-01-11     2000-03-21
##  2  2000 dry_sep Dry    2000-09-15   2000-09-25 2000-08-16     2000-10-25
##  3  2000 wet_dec Wet    2000-12-05   2000-12-30 2000-11-05     2001-01-29
##  4  2000 wet_may Wet    2000-05-05   2000-05-30 2000-04-05     2000-06-29
##  5  2010 dry_feb Dry    2010-02-10   2010-02-20 2010-01-11     2010-03-22
##  6  2010 dry_sep Dry    2010-09-15   2010-09-25 2010-08-16     2010-10-25
##  7  2010 wet_dec Wet    2010-12-05   2010-12-30 2010-11-05     2011-01-29
##  8  2010 wet_may Wet    2010-05-05   2010-05-30 2010-04-05     2010-06-29
##  9  2020 dry_feb Dry    2020-02-10   2020-02-20 2020-01-11     2020-03-21
## 10  2020 dry_sep Dry    2020-09-15   2020-09-25 2020-08-16     2020-10-25
## 11  2020 wet_dec Wet    2020-12-05   2020-12-30 2020-11-05     2021-01-29
## 12  2020 wet_may Wet    2020-05-05   2020-05-30 2020-04-05     2020-06-29
## 13  2025 dry_feb Dry    2025-02-10   2025-02-20 2025-01-11     2025-03-22
## 14  2025 dry_sep Dry    2025-09-15   2025-09-25 2025-08-16     2025-10-25
## 15  2025 wet_dec Wet    2025-12-05   2025-12-30 2025-11-05     2026-01-29
## 16  2025 wet_may Wet    2025-05-05   2025-05-30 2025-04-05     2025-06-29
```

8 8. Create seasonal Landsat composites

Landsat 5 and 7 are used for earlier years; Landsat 8 and 9 contribute when available for later years. Collections are merged after the bands have been standardized.

The end date is exclusive in Earth Engine, so one day is added to include the requested final day.

```
# ============================================================
# SECTION 8: LANDSAT PREPROCESSING FUNCTIONS
# ============================================================

mask_scale_l57 <- function(image) {

  qa <- image$select("QA_PIXEL")
```

```
  mask <- qa$bitwiseAnd(1)$eq(0)$
    And(qa$bitwiseAnd(2)$eq(0))$
    And(qa$bitwiseAnd(8)$eq(0))$
    And(qa$bitwiseAnd(16)$eq(0))$
    And(qa$bitwiseAnd(32)$eq(0))

  saturation_mask <- image$select("QA_RADSAT")$eq(0)

  reflectance <- image$select(
    c("SR_B1", "SR_B2", "SR_B3",
      "SR_B4", "SR_B5", "SR_B7"),
    c("blue", "green", "red",
      "nir", "swir1", "swir2")
  )$multiply(0.0000275)$add(-0.2)

  processed <- reflectance$
    updateMask(mask)$
    updateMask(saturation_mask)

  processed$copyProperties(
    image,
    c("system:time_start", "system:index")
  )
}


mask_scale_l89 <- function(image) {

  qa <- image$select("QA_PIXEL")

  mask <- qa$bitwiseAnd(1)$eq(0)$
    And(qa$bitwiseAnd(2)$eq(0))$
    And(qa$bitwiseAnd(4)$eq(0))$
    And(qa$bitwiseAnd(8)$eq(0))$
    And(qa$bitwiseAnd(16)$eq(0))$
    And(qa$bitwiseAnd(32)$eq(0))

  saturation_mask <- image$select("QA_RADSAT")$eq(0)

  reflectance <- image$select(
    c("SR_B2", "SR_B3", "SR_B4",
      "SR_B5", "SR_B6", "SR_B7"),
    c("blue", "green", "red",
      "nir", "swir1", "swir2")
  )$multiply(0.0000275)$add(-0.2)

  processed <- reflectance$
    updateMask(mask)$
    updateMask(saturation_mask)

  processed$copyProperties(
    image,
    c("system:time_start", "system:index")
```

```
  )
}
```

```
# ============================================================
# SECTION 9: EXPANDED-WINDOW AVAILABILITY AND COVERAGE
# ============================================================

library(dplyr)
library(tibble)
library(readr)
library(reticulate)

ee <- reticulate::import("ee", convert = FALSE)

# Ensure Earth Engine is initialized.
ee$Initialize(project = "thinking-leaf-305215")
```

```
## None
```

```
# Helper: call a Python method explicitly.
ee_method <- function(object, method, ...) {
  fn <- reticulate::py_get_attr(object, method)
  reticulate::py_call(fn, ...)
}

# Helper: safely retrieve an Earth Engine result.
ee_get_info <- function(object) {
  reticulate::py_to_r(
    ee_method(object, "getInfo")
  )
}

# Python callbacks for Earth Engine server-side mapping.
# These call the preprocessing functions defined in Section 8.
preprocess_l57_py <- reticulate::py_eval(
  paste0(
    "lambda image: image.select(",
    "['SR_B1','SR_B2','SR_B3','SR_B4','SR_B5','SR_B7'], ",
    "['blue','green','red','nir','swir1','swir2'])",
    ".multiply(0.0000275).add(-0.2)",
    ".updateMask(image.select('QA_PIXEL').bitwiseAnd(1).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(2).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(8).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(16).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(32).eq(0))))))",
    ".updateMask(image.select('QA_RADSAT').eq(0))",
    ".copyProperties(image, ['system:time_start','system:index'])"
  ),
  convert = FALSE
)

preprocess_l89_py <- reticulate::py_eval(
  paste0(
    "lambda image: image.select(",
    "['SR_B2','SR_B3','SR_B4','SR_B5','SR_B6','SR_B7'], ",
```

```
    "['blue','green','red','nir','swir1','swir2'])",
    ".multiply(0.0000275).add(-0.2)",
    ".updateMask(image.select('QA_PIXEL').bitwiseAnd(1).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(2).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(4).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(8).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(16).eq(0)",
    ".And(image.select('QA_PIXEL').bitwiseAnd(32).eq(0)))))))",
    ".updateMask(image.select('QA_RADSAT').eq(0))",
    ".copyProperties(image, ['system:time_start','system:index'])"
  ),
  convert = FALSE
)

sensor_specs <- list(
  list(
    id = "LANDSAT/LT05/C02/T1_L2",
    callback = preprocess_157_py
  ),
  list(
    id = "LANDSAT/LE07/C02/T1_L2",
    callback = preprocess_157_py
  ),
  list(
    id = "LANDSAT/LC08/C02/T1_L2",
    callback = preprocess_189_py
  ),
  list(
    id = "LANDSAT/LC09/C02/T1_L2",
    callback = preprocess_189_py
  )
)

min_scenes <- 2
coverage_threshold <- 0.70

get_window_result <- function(year, window, target_start,
                              target_end, expanded_start,
                              expanded_end_exclusive, key) {

  message("Checking: ", key)

  start_string <- format(expanded_start, "%Y-%m-%d")
  end_string <- format(expanded_end_exclusive, "%Y-%m-%d")

  scene_count <- 0L
  dates <- character(0)
  processed_collections <- list()

  for (sensor in sensor_specs) {

    raw <- ee$ImageCollection(sensor$id)
    raw <- ee_method(raw, "filterDate", start_string, end_string)
```

```
  raw <- ee_method(raw, "filterBounds", roi)

  n <- tryCatch(
    as.integer(ee_get_info(ee_method(raw, "size"))),
    error = function(e) {
      message("Scene-count error for ", sensor$id, ": ",
              conditionMessage(e))
      0L
    }
  )

  scene_count <- scene_count + n

  if (n > 0L) {

    # Extract dates from the original collection.
    date_callback <- reticulate::py_eval(
      "lambda image: image.set('acq_date', image.date().format('YYYY-MM-dd'))",
      convert = FALSE
    )

    dated_raw <- ee_method(raw, "map", date_callback)

    sensor_dates <- tryCatch(
      ee_get_info(ee_method(dated_raw, "aggregate_array", "acq_date")),
      error = function(e) {
        message("Date extraction failed for ", sensor$id, ": ",
                conditionMessage(e))
        character(0)
      }
    )

    dates <- c(dates, as.character(unlist(sensor_dates)))

    processed <- ee_method(raw, "map", sensor$callback)
    processed_collections[[length(processed_collections) + 1L]] <-
      processed
  }
}

dates <- sort(unique(dates))

if (scene_count == 0L || length(processed_collections) == 0L) {
  return(list(
    summary = tibble(
      year = year,
      window = window,
      key = key,
      target_start = as.character(target_start),
      target_end = as.character(target_end),
      expanded_start = start_string,
      expanded_end = as.character(expanded_end_exclusive - 1),
      scene_count = scene_count,
```

```
      unique_acquisition_dates = length(dates),
      acquisition_dates = paste(dates, collapse = "; "),
      valid_pixel_coverage = NA_real_,
      coverage_status = "no_scenes"
    ),
    image = NULL
  ))
}

# Merge processed collections.
merged <- processed_collections[[1]]

if (length(processed_collections) > 1L) {
  for (j in 2:length(processed_collections)) {
    merged <- ee_method(
      merged, "merge", processed_collections[[j]]
    )
  }
}

composite <- ee_method(merged, "median")
composite <- ee_method(composite, "clip", roi)

coverage <- tryCatch({

  valid_mask <- ee_method(composite, "select",
                          c("blue", "green", "red",
                            "nir", "swir1", "swir2"))
  valid_mask <- ee_method(valid_mask, "mask")
  valid_mask <- ee_method(valid_mask, "reduce", ee$Reducer$min())
  valid_mask <- ee_method(valid_mask, "rename", "valid")

  coverage_dictionary <- ee_method(
    valid_mask, "reduceRegion",
    reducer = ee$Reducer$mean(),
    geometry = roi,
    scale = 30,
    maxPixels = 1e9,
    bestEffort = TRUE
  )

  coverage_values <- ee_get_info(coverage_dictionary)
  value <- coverage_values[["valid"]]

  if (is.null(value)) NA_real_ else as.numeric(value)

}, error = function(e) {
  message("Coverage calculation failed for ", key, ": ",
          conditionMessage(e))
  NA_real_
})

status <- if (is.na(coverage)) {
```

```
    "coverage_unavailable"
  } else if (scene_count < min_scenes) {
    "insufficient_scenes"
  } else if (coverage < coverage_threshold) {
    "low_valid_pixel_coverage"
  } else {
    "adequate_coverage"
  }

  list(
    summary = tibble(
      year = year,
      window = window,
      key = key,
      target_start = as.character(target_start),
      target_end = as.character(target_end),
      expanded_start = start_string,
      expanded_end = as.character(expanded_end_exclusive - 1),
      scene_count = scene_count,
      unique_acquisition_dates = length(dates),
      acquisition_dates = paste(dates, collapse = "; "),
      valid_pixel_coverage = coverage,
      coverage_status = status
    ),
    image = composite
  )
}
```

```
# ============================================================
# SECTION 9B: RESUMABLE WINDOW PROCESSING AND SAFE EXTRACTION
# ============================================================

# Remove any R-level time limit that might interrupt processing.
setTimeLimit(
  cpu = Inf,
  elapsed = Inf,
  transient = FALSE
)

# ------------------------------------------------------------
# 1. Initialize or reuse existing results
# ------------------------------------------------------------

expected_keys <- as.character(window_schedule$key)

if (
  !exists("window_results", inherits = FALSE) ||
  !is.list(window_results) ||
  !identical(names(window_results), expected_keys)
) {
  window_results <- setNames(
    vector("list", length(expected_keys)),
    expected_keys
  )
```

```
} else {
  message("Reusing existing window_results.")
}

# Create the composite registry BEFORE processing. Each successful image is
# registered immediately below, so a later-window problem cannot prevent the
# earlier composites from being available to later sections.
# NOTE: RStudio may print `reached elapsed time limit` from
# .Call(`_reticulate_py_str_impl`, x) while its Environment pane tries to
# summarise an Earth Engine Python proxy. That IDE inspection warning does
# not by itself prove the assignment failed; use the checks below to confirm.
seasonal_composites <- setNames(
  vector("list", length(expected_keys)),
  expected_keys
)

# Output files: save checkpoints after each window
availability_file <- "Tharaka_Landsat_Expanded_Window_Summary.csv"

dates_file <- "Tharaka_Landsat_Acquisition_Dates.csv"

# ------------------------------------------------------------
# 2. Helper to save all completed summaries
# ------------------------------------------------------------

save_availability_outputs <- function(results) {

  valid_results <- Filter(
    function(x) {
      is.list(x) &&
        !is.null(x$summary)
    },
    results
  )

  if (length(valid_results) == 0L) {
    return(invisible(NULL))
  }

  availability_summary <- dplyr::bind_rows(
    lapply(valid_results, function(x) x$summary)
  ) %>%
    mutate(
      valid_pixel_coverage_pct =
        round(100 * valid_pixel_coverage, 1)
    ) %>%
    arrange(
      year,
      match(
        window,
        c("dry_feb", "dry_sep", "wet_may", "wet_dec")
      )
    )
```

```
  readr::write_csv(
    availability_summary,
    availability_file
  )

  readr::write_csv(
    availability_summary %>%
      select(
        year, window, target_start, target_end,
        expanded_start, expanded_end,
        scene_count, unique_acquisition_dates,
        acquisition_dates, coverage_status
      ),
    dates_file
  )

  invisible(availability_summary)
}

# ------------------------------------------------------------
# 3. Process only missing or previously failed windows
# ------------------------------------------------------------

for (i in seq_len(nrow(window_schedule))) {

  w <- window_schedule[i, ]
  key <- as.character(w$key)

  previous <- window_results[[key]]

  already_completed <-
    is.list(previous) &&
    !is.null(previous$summary) &&
    !identical(
      as.character(previous$summary$coverage_status[1]),
      "processing_error"
    )

  if (already_completed) {
    # Restore the image into the registry when re-running this chunk in an
    # existing R session.
    seasonal_composites[[key]] <- previous[["image"]]
    message("Skipping completed window: ", key)
    next
  }

  message("\nProcessing window: ", key)

  result <- tryCatch(
    get_window_result(
      year = w$year,
      window = w$window,
      target_start = w$target_start,
```

```
      target_end = w$target_end,
      expanded_start = w$expanded_start,
      expanded_end_exclusive = w$expanded_end_exclusive,
      key = key
    ),
    error = function(e) {

      message(
        "Window failed: ", key,
        " | ", conditionMessage(e)
      )

      list(
        summary = tibble(
          year = w$year,
          window = w$window,
          key = key,
          target_start = as.character(w$target_start),
          target_end = as.character(w$target_end),
          expanded_start = as.character(w$expanded_start),
          expanded_end = as.character(
            w$expanded_end_exclusive - 1
          ),
          scene_count = NA_integer_,
          unique_acquisition_dates = NA_integer_,
          acquisition_dates = NA_character_,
          valid_pixel_coverage = NA_real_,
          coverage_status = "processing_error"
        ),
        image = NULL
      )
    }
  )

  # Retain the result and register its Earth Engine image immediately.
  # The image remains a server-side Earth Engine object; no raster download
  # or Python-to-R conversion is performed here.
  window_results[[key]] <- result
  seasonal_composites[[key]] <- result[["image"]]

  # Save progress even if a later window fails.
  save_availability_outputs(window_results)

  gc(verbose = FALSE)
}

# ------------------------------------------------------------
# 4. Validate the registry without printing Python-backed objects
# ------------------------------------------------------------

# Populate any missing registry entries from completed checkpoint results.
# This supports resuming the chunk after an interrupted RStudio session.
for (key in expected_keys) {
```

```
  if (is.null(seasonal_composites[[key]])) {
    result <- window_results[[key]]
    if (is.list(result) && "image" %in% names(result)) {
      seasonal_composites[[key]] <- result[["image"]]
    }
  }
}

# ------------------------------------------------------------
# 5. Final summary and checks
# ------------------------------------------------------------

availability_summary <- save_availability_outputs(
  window_results
)

cat("\n===== PROCESSING STATUS =====\n")
```

```
##
## ===== PROCESSING STATUS =====
```

```
if (!is.null(availability_summary)) {
  print(
    availability_summary %>%
      select(
        year, window, scene_count,
        valid_pixel_coverage_pct, coverage_status
      ),
    n = Inf
  )
}
```

```
## # A tibble: 16 x 5
##     year window  scene_count valid_pixel_coverage_pct coverage_status
##    <dbl> <chr>         <int>                    <dbl> <chr>
##  1  2000 dry_feb           4                     99.9 adequate_coverage
##  2  2000 dry_sep           3                     99.9 adequate_coverage
##  3  2000 wet_may           5                     92.7 adequate_coverage
##  4  2000 wet_dec           6                     88.9 adequate_coverage
##  5  2010 dry_feb           7                     95.5 adequate_coverage
##  6  2010 dry_sep           4                     98.5 adequate_coverage
##  7  2010 wet_may           6                     84.2 adequate_coverage
##  8  2010 wet_dec          17                     99.4 adequate_coverage
##  9  2020 dry_feb          16                     99.5 adequate_coverage
## 10  2020 dry_sep          17                     99.9 adequate_coverage
## 11  2020 wet_may          22                     99.9 adequate_coverage
## 12  2020 wet_dec          19                     99.8 adequate_coverage
## 13  2025 dry_feb          18                     99.9 adequate_coverage
## 14  2025 dry_sep          17                     99.9 adequate_coverage
## 15  2025 wet_may          21                     99.9 adequate_coverage
## 16  2025 wet_dec          19                     99.9 adequate_coverage
```

```
cat(
  "\nComposite images retained:",
  sum(!vapply(seasonal_composites, is.null, logical(1))),
```

```
  "of", length(seasonal_composites), "\n"
)
```

```
##
## Composite images retained: 16 of 16
```

```
cat(
  "Availability summary:",
  normalizePath(availability_file, mustWork = FALSE), "\n"
)
```

```
## Availability summary: C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Landsat_Expanded_Window_Summary.csv
```

```
cat(
  "Acquisition dates:",
  normalizePath(dates_file, mustWork = FALSE), "\n"
)
```

```
## Acquisition dates: C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Landsat_Acquisition_Dates.csv
```

```
# ============================================================
# EXPORT REPORTS WITHOUT REQUERYING EARTH ENGINE
# ============================================================

# Convert to a plain R tibble to remove any Python-backed values.
availability_summary_r <- as.data.frame(availability_summary)

# Export the full summary.
readr::write_csv(
  availability_summary_r,
  "Tharaka_Landsat_Expanded_Window_Summary.csv"
)

# Export the acquisition-date report.
acquisition_date_report <- availability_summary_r[
  ,
  c(
    "year", "window", "target_start", "target_end",
    "expanded_start", "expanded_end",
    "scene_count", "unique_acquisition_dates",
    "acquisition_dates", "coverage_status"
  ),
  drop = FALSE
]

readr::write_csv(
  acquisition_date_report,
  "Tharaka_Landsat_Acquisition_Dates.csv"
)

cat("Export completed.\n")
```

```
## Export completed.
```

```
cat("Summary rows:", nrow(availability_summary_r), "\n")
```

```
## Summary rows: 16
```

```r
cat("Acquisition report rows:", nrow(acquisition_date_report), "\n")
```

```
## Acquisition report rows: 16
```

9 9. Check scene availability

```r
# ============================================================
# SECTION 10: VALIDATE ORIGINAL TARGET WINDOWS
# ============================================================

library(dplyr)
library(tibble)
library(readr)

# Server-side callback to attach a readable acquisition date.
date_callback <- reticulate::py_eval(
  "lambda image: image.set('acq_date', image.date().format('YYYY-MM-dd'))",
  convert = FALSE
)

validate_one_window <- function(w) {

  message("Validating target window: ", w$key)

  target_start <- as.character(w$target_start)
  target_end_exclusive <- as.character(w$target_end + 1)

  target_scene_count <- 0L
  target_dates <- character(0)

  # Count and list acquisitions inside the ORIGINAL target dates.
  for (sensor in sensor_specs) {

    raw <- ee$ImageCollection(sensor$id)
    raw <- ee_method(
      raw, "filterDate",
      target_start, target_end_exclusive
    )
    raw <- ee_method(raw, "filterBounds", roi)

    n <- tryCatch(
      as.integer(ee_get_info(ee_method(raw, "size"))),
      error = function(e) {
        message("Target scene count failed for ",
                sensor$id, ": ", conditionMessage(e))
        0L
      }
    )

    target_scene_count <- target_scene_count + n

    if (n > 0L) {
      dated_raw <- ee_method(raw, "map", date_callback)
```

```
    dates_sensor <- tryCatch(
      ee_get_info(
        ee_method(dated_raw, "aggregate_array", "acq_date")
      ),
      error = function(e) {
        message("Target date extraction failed for ",
                sensor$id, ": ", conditionMessage(e))
        character(0)
      }
    )

    target_dates <- c(
      target_dates,
      as.character(unlist(dates_sensor))
    )
  }
}

target_dates <- sort(unique(target_dates))

# Recalculate coverage for the existing expanded-window composite
# at a fixed 30 m scale, without bestEffort.
composite <- seasonal_composites[[w$key]]

fixed_scale_coverage <- tryCatch({

  if (is.null(composite)) {
    stop("No expanded-window composite is available.")
  }

  valid_mask <- ee_method(
    composite, "select",
    c("blue", "green", "red", "nir", "swir1", "swir2")
  )

  valid_mask <- ee_method(valid_mask, "mask")
  valid_mask <- ee_method(valid_mask, "reduce", ee$Reducer$min())
  valid_mask <- ee_method(valid_mask, "rename", "valid")

  coverage_dictionary <- ee_method(
    valid_mask, "reduceRegion",
    reducer = ee$Reducer$mean(),
    geometry = roi,
    scale = 30,
    maxPixels = 1e10,
    tileScale = 4
  )

  coverage_values <- ee_get_info(coverage_dictionary)
  coverage_value <- coverage_values[["valid"]]

  if (is.null(coverage_value)) {
    NA_real_
```

```
    } else {
      as.numeric(coverage_value)
    }

  }, error = function(e) {
    message("Fixed-scale coverage failed for ", w$key, ": ",
            conditionMessage(e))
    NA_real_
  })

  target_status <- if (target_scene_count == 0L) {
    "buffer_only_no_target_scenes"
  } else {
    "target_window_has_scenes"
  }

  coverage_status <- if (is.na(fixed_scale_coverage)) {
    "coverage_unavailable"
  } else if (fixed_scale_coverage < coverage_threshold) {
    "low_valid_pixel_coverage"
  } else {
    "coverage_passes_threshold"
  }

  tibble(
    year = w$year,
    window = w$window,
    key = w$key,
    target_start = target_start,
    target_end = as.character(w$target_end),
    expanded_start = as.character(w$expanded_start),
    expanded_end = as.character(w$expanded_end),
    expanded_scene_count = as.integer(
      availability_summary$scene_count[
        match(w$key, availability_summary$key)
      ]
    ),
    target_scene_count = target_scene_count,
    unique_target_dates = length(target_dates),
    target_acquisition_dates = paste(target_dates, collapse = "; "),
    fixed_scale_coverage = fixed_scale_coverage,
    fixed_scale_coverage_pct = round(
      100 * fixed_scale_coverage, 2
    ),
    target_status = target_status,
    coverage_status = coverage_status
  )
}

# Validate all 16 windows.
target_validation <- bind_rows(
  lapply(seq_len(nrow(window_schedule)), function(i) {
    validate_one_window(window_schedule[i, ])
```

```
  })
) %>%
  arrange(year, match(window, windows$window))

print(target_validation, n = Inf, width = Inf)
```

```
## # A tibble: 16 x 15
##     year window  key          target_start target_end expanded_start
##    <dbl> <chr>   <chr>        <chr>        <chr>      <chr>
##  1  2000 dry_feb 2000_dry_feb 2000-02-10   2000-02-20 2000-01-11
##  2  2000 dry_sep 2000_dry_sep 2000-09-15   2000-09-25 2000-08-16
##  3  2000 wet_may 2000_wet_may 2000-05-05   2000-05-30 2000-04-05
##  4  2000 wet_dec 2000_wet_dec 2000-12-05   2000-12-30 2000-11-05
##  5  2010 dry_feb 2010_dry_feb 2010-02-10   2010-02-20 2010-01-11
##  6  2010 dry_sep 2010_dry_sep 2010-09-15   2010-09-25 2010-08-16
##  7  2010 wet_may 2010_wet_may 2010-05-05   2010-05-30 2010-04-05
##  8  2010 wet_dec 2010_wet_dec 2010-12-05   2010-12-30 2010-11-05
##  9  2020 dry_feb 2020_dry_feb 2020-02-10   2020-02-20 2020-01-11
## 10  2020 dry_sep 2020_dry_sep 2020-09-15   2020-09-25 2020-08-16
## 11  2020 wet_may 2020_wet_may 2020-05-05   2020-05-30 2020-04-05
## 12  2020 wet_dec 2020_wet_dec 2020-12-05   2020-12-30 2020-11-05
## 13  2025 dry_feb 2025_dry_feb 2025-02-10   2025-02-20 2025-01-11
## 14  2025 dry_sep 2025_dry_sep 2025-09-15   2025-09-25 2025-08-16
## 15  2025 wet_may 2025_wet_may 2025-05-05   2025-05-30 2025-04-05
## 16  2025 wet_dec 2025_wet_dec 2025-12-05   2025-12-30 2025-11-05
##    expanded_end expanded_scene_count target_scene_count unique_target_dates
##    <chr>                       <int>              <int>                <int>
##  1 2000-03-21                      4                  0                    0
##  2 2000-10-25                      3                  2                    2
##  3 2000-06-29                      5                  0                    0
##  4 2001-01-29                      6                  3                    3
##  5 2010-03-22                      7                  0                    0
##  6 2010-10-25                      4                  0                    0
##  7 2010-06-29                      6                  1                    1
##  8 2011-01-29                     17                  5                    5
##  9 2020-03-21                     16                  3                    3
## 10 2020-10-25                     17                  4                    4
## 11 2020-06-29                     22                  6                    6
## 12 2021-01-29                     19                  6                    6
## 13 2025-03-22                     18                  3                    3
## 14 2025-10-25                     17                  2                    2
## 15 2025-06-29                     21                  6                    6
## 16 2026-01-29                     19                  5                    5
##    target_acquisition_dates
##    <chr>
##  1 ""
##  2 "2000-09-16; 2000-09-25"
##  3 ""
##  4 "2000-12-05; 2000-12-14; 2000-12-30"
##  5 ""
##  6 ""
##  7 "2010-05-16"
##  8 "2010-12-09; 2010-12-17; 2010-12-18; 2010-12-25; 2010-12-26"
##  9 "2020-02-12; 2020-02-13; 2020-02-20"
```

```
## 10 "2020-09-15; 2020-09-16; 2020-09-23; 2020-09-24"
## 11 "2020-05-10; 2020-05-11; 2020-05-18; 2020-05-19; 2020-05-26; 2020-05-27"
## 12 "2020-12-05; 2020-12-12; 2020-12-13; 2020-12-20; 2020-12-28; 2020-12-29"
## 13 "2025-02-10; 2025-02-17; 2025-02-18"
## 14 "2025-09-21; 2025-09-22"
## 15 "2025-05-08; 2025-05-09; 2025-05-16; 2025-05-17; 2025-05-24; 2025-05-25"
## 16 "2025-12-10; 2025-12-11; 2025-12-18; 2025-12-26; 2025-12-27"
##    fixed_scale_coverage fixed_scale_coverage_pct target_status
##                   <dbl>                    <dbl> <chr>
##  1                0.999                     99.9 buffer_only_no_target_scenes
##  2                0.999                     99.9 target_window_has_scenes
##  3                0.927                     92.7 buffer_only_no_target_scenes
##  4                0.889                     88.9 target_window_has_scenes
##  5                0.955                     95.5 buffer_only_no_target_scenes
##  6                0.985                     98.5 buffer_only_no_target_scenes
##  7                0.842                     84.2 target_window_has_scenes
##  8                0.994                     99.4 target_window_has_scenes
##  9                0.995                     99.4 target_window_has_scenes
## 10                0.999                     99.9 target_window_has_scenes
## 11                0.999                     99.9 target_window_has_scenes
## 12                0.998                     99.8 target_window_has_scenes
## 13                0.999                     99.9 target_window_has_scenes
## 14                0.999                     99.9 target_window_has_scenes
## 15                0.999                     99.9 target_window_has_scenes
## 16                0.999                     99.9 target_window_has_scenes
##    coverage_status
##    <chr>
##  1 coverage_passes_threshold
##  2 coverage_passes_threshold
##  3 coverage_passes_threshold
##  4 coverage_passes_threshold
##  5 coverage_passes_threshold
##  6 coverage_passes_threshold
##  7 coverage_passes_threshold
##  8 coverage_passes_threshold
##  9 coverage_passes_threshold
## 10 coverage_passes_threshold
## 11 coverage_passes_threshold
## 12 coverage_passes_threshold
## 13 coverage_passes_threshold
## 14 coverage_passes_threshold
## 15 coverage_passes_threshold
## 16 coverage_passes_threshold
```

```r
# Export the validation report.
readr::write_csv(
  target_validation,
  "Tharaka_Landsat_Target_Window_Validation.csv"
)
```

10 10. Optional target-window composite analysis

The optional chunk below rebuilds composites using only acquisitions inside the exact target dates and compares them with the expanded-window results. It is disabled by default because it issues additional Earth

Engine queries for the 16 windows. Section 12 uses the expanded-window composites created in Section 9B and does not depend on this optional chunk.

```
# ========================================================
# SECTION 11: TARGET COMPOSITES AND FINAL DATA SELECTION
# ========================================================

library(dplyr)
library(tibble)
library(readr)

# Calculate valid-pixel coverage for a composite at fixed 30 m.
get_fixed_coverage <- function(image) {

  if (is.null(image)) return(NA_real_)

  tryCatch({

    valid <- ee_method(
      image, "select",
      c("blue", "green", "red", "nir", "swir1", "swir2")
    )

    valid <- ee_method(valid, "mask")
    valid <- ee_method(valid, "reduce", ee$Reducer$min())
    valid <- ee_method(valid, "rename", "valid")

    result <- ee_method(
      valid, "reduceRegion",
      reducer = ee$Reducer$mean(),
      geometry = roi,
      scale = 30,
      maxPixels = 1e10,
      tileScale = 4
    )

    values <- ee_get_info(result)
    x <- values[["valid"]]

    if (is.null(x)) NA_real_ else as.numeric(x)

  }, error = function(e) {
    message("Coverage calculation failed: ", conditionMessage(e))
    NA_real_
  })
}


# Build a composite using acquisitions inside the original target dates.
build_target_composite <- function(w) {

  message("Building target-only composite: ", w$key)

  start <- as.character(w$target_start)
```

```
  end_exclusive <- as.character(w$target_end + 1)

  collections <- list()
  total_scenes <- 0L

  for (sensor in sensor_specs) {

    raw <- ee$ImageCollection(sensor$id)
    raw <- ee_method(raw, "filterDate", start, end_exclusive)
    raw <- ee_method(raw, "filterBounds", roi)

    n <- tryCatch(
      as.integer(ee_get_info(ee_method(raw, "size"))),
      error = function(e) 0L
    )

    total_scenes <- total_scenes + n

    if (n > 0L) {
      collections[[length(collections) + 1L]] <-
        ee_method(raw, "map", sensor$callback)
    }
  }

  if (total_scenes == 0L || length(collections) == 0L) {
    return(list(
      image = NULL,
      scene_count = total_scenes,
      coverage = NA_real_
    ))
  }

  merged <- collections[[1]]

  if (length(collections) > 1L) {
    for (j in 2:length(collections)) {
      merged <- ee_method(merged, "merge", collections[[j]])
    }
  }

  composite <- ee_method(merged, "median")
  composite <- ee_method(composite, "clip", roi)

  list(
    image = composite,
    scene_count = total_scenes,
    coverage = get_fixed_coverage(composite)
  )
}


# Build target composites for all 16 combinations.
target_composites <- vector("list", nrow(window_schedule))
```

```r
names(target_composites) <- window_schedule$key

target_composite_results <- vector("list", nrow(window_schedule))

for (i in seq_len(nrow(window_schedule))) {

  w <- window_schedule[i, ]

  result <- build_target_composite(w)

  target_composites[[w$key]] <- result$image

  target_composite_results[[i]] <- tibble(
    key = w$key,
    target_scene_count_rechecked = result$scene_count,
    target_fixed_coverage = result$coverage,
    target_fixed_coverage_pct = round(100 * result$coverage, 2)
  )
}

target_composite_summary <- bind_rows(target_composite_results)


# Join target-only and expanded-window assessment results.
selection_table <- target_validation %>%
  left_join(
    target_composite_summary,
    by = "key"
  ) %>%
  left_join(
    availability_summary %>%
      select(
        key,
        expanded_coverage_original = valid_pixel_coverage,
        expanded_coverage_original_pct = valid_pixel_coverage_pct,
        expanded_acquisition_dates = acquisition_dates
      ),
    by = "key"
  ) %>%
  mutate(
    target_support = case_when(
      target_scene_count_rechecked == 0 ~ "buffer_dependent",
      target_scene_count_rechecked <= 2 ~ "limited_target_support",
      TRUE ~ "multiple_target_scenes"
    ),

    target_coverage_pass = !is.na(target_fixed_coverage) &
      target_fixed_coverage >= coverage_threshold,

    expanded_coverage_pass = !is.na(expanded_coverage_original) &
      expanded_coverage_original >= coverage_threshold,

    selection_recommendation = case_when(
```

```
      target_scene_count_rechecked > 0 &
        target_coverage_pass &
        target_scene_count_rechecked >= 3 ~
        "prefer_target_window",

      target_scene_count_rechecked > 0 &
        target_coverage_pass ~
        "target_window_candidate_limited_dates",

      target_scene_count_rechecked == 0 &
        expanded_coverage_pass ~
        "expanded_window_fallback_requires_review",

      expanded_coverage_pass ~
        "review_target_and_expanded_data",

      TRUE ~
        "manual_review_required"
    ),

    selection_reason = case_when(
      selection_recommendation == "prefer_target_window" ~
        "At least three target-window scenes and coverage passes screening threshold",

      selection_recommendation == "target_window_candidate_limited_dates" ~
        "Target-window scenes exist, but temporal support is limited",

      selection_recommendation == "expanded_window_fallback_requires_review" ~
        "No target-window scenes; expanded imagery is a possible substitute",

      selection_recommendation == "review_target_and_expanded_data" ~
        "Target coverage does not pass; inspect both composites",

      TRUE ~
        "Coverage is unavailable or inadequate"
    )
  ) %>%
  arrange(year, match(window, windows$window))


# Inspect the final acquisition-selection table.
print(
  selection_table %>%
    select(
      year, window,
      target_scene_count_rechecked,
      target_fixed_coverage_pct,
      expanded_scene_count,
      expanded_coverage_original_pct,
      target_support,
      selection_recommendation,
      selection_reason
    ),
```

```
  n = Inf,
  width = Inf
)

# Save the full decision table.
readr::write_csv(
  selection_table,
  "Tharaka_Landsat_Acquisition_Selection.csv"
)
```

```
# ============================================================
# SECTION 12: FINAL COMPOSITE SELECTION AND VALIDATION
# Tharaka Sub-County Land-Cover Analysis
# ============================================================

library(dplyr)
library(readr)
library(tibble)

# Remove any R-level time limit that could interrupt processing
setTimeLimit(
  cpu = Inf,
  elapsed = Inf,
  transient = FALSE
)

# ------------------------------------------------------------
# 12.1. Validate required objects
# ------------------------------------------------------------

# Recovery path: if a prior run built window_results but stopped before the
# composite registry was assembled, rebuild the registry from those results.
# This does not rerun Earth Engine queries.
if (
  !exists("seasonal_composites", inherits = TRUE) &&
  exists("window_results", inherits = TRUE) &&
  is.list(window_results)
) {
  seasonal_composites <- setNames(
    lapply(window_results, function(result) {
      if (is.list(result) && "image" %in% names(result)) {
        result[["image"]]
      } else {
        NULL
      }
    }),
    names(window_results)
  )
}

required_objects <- c(
  "window_schedule",
  "availability_summary",
  "seasonal_composites"
```

```
)

missing_objects <- required_objects[
  !vapply(
    required_objects,
    exists,
    logical(1),
    inherits = TRUE
  )
]

if (length(missing_objects) > 0L) {
  stop(
    "Section 12 cannot proceed because these prerequisites are missing: ",
    paste(missing_objects, collapse = ", "),
    ". Run the document from the beginning through Section 9B. " ,
    "Do not execute Section 12 by itself in a fresh R session."
  )
}

# Confirm that the schedule and composites have matching keys
expected_keys <- as.character(window_schedule$key)
composite_keys <- names(seasonal_composites)

if (is.null(composite_keys)) {
  stop("seasonal_composites must be a named list.")
}

missing_composites <- setdiff(expected_keys, composite_keys)

if (length(missing_composites) > 0L) {
  stop(
    "Missing composite keys: ",
    paste(missing_composites, collapse = ", ")
  )
}

# Check which composites contain live Earth Engine images
image_available <- vapply(
  expected_keys,
  function(k) {
    !is.null(seasonal_composites[[k]])
  },
  logical(1)
)

composite_status <- tibble(
  key = expected_keys,
  image_available = image_available
)

print(composite_status, n = Inf)
```

```
## # A tibble: 16 x 2
```

```
##    key           image_available
##    <chr>         <lgl>
##  1 2000_dry_feb TRUE
##  2 2000_dry_sep TRUE
##  3 2000_wet_dec TRUE
##  4 2000_wet_may TRUE
##  5 2010_dry_feb TRUE
##  6 2010_dry_sep TRUE
##  7 2010_wet_dec TRUE
##  8 2010_wet_may TRUE
##  9 2020_dry_feb TRUE
## 10 2020_dry_sep TRUE
## 11 2020_wet_dec TRUE
## 12 2020_wet_may TRUE
## 13 2025_dry_feb TRUE
## 14 2025_dry_sep TRUE
## 15 2025_wet_dec TRUE
## 16 2025_wet_may TRUE
```

```r
if (!all(image_available)) {
  stop(
    "Some seasonal composites are unavailable. ",
    "Review composite_status before proceeding."
  )
}

cat(
  "\nSection 12 prerequisites passed:",
  sum(image_available),
  "of",
  length(expected_keys),
  "composites available.\n"
)
```

```
##
## Section 12 prerequisites passed: 16 of 16 composites available.
```

```r
# ------------------------------------------------------------
# 12.2. Define output directory
# ------------------------------------------------------------

plot_dir <- "Tharaka_Landsat_Enhanced_Maps"

if (!dir.exists(plot_dir)) {
  dir.create(
    plot_dir,
    recursive = TRUE,
    showWarnings = FALSE
  )
}

# ------------------------------------------------------------
# 12.3. Validate schedule and availability keys
# ------------------------------------------------------------
```

```r
required_schedule_columns <- c(
  "year", "window", "season", "key",
  "target_start", "target_end",
  "expanded_start", "expanded_end"
)

required_availability_columns <- c(
  "key", "scene_count",
  "valid_pixel_coverage_pct",
  "coverage_status"
)

missing_schedule_columns <- setdiff(
  required_schedule_columns,
  names(window_schedule)
)

missing_availability_columns <- setdiff(
  required_availability_columns,
  names(availability_summary)
)

if (length(missing_schedule_columns) > 0) {
  stop(
    "Missing columns in window_schedule: ",
    paste(missing_schedule_columns, collapse = ", ")
  )
}

if (length(missing_availability_columns) > 0) {
  stop(
    "Missing columns in availability_summary: ",
    paste(missing_availability_columns, collapse = ", ")
  )
}

if (anyDuplicated(window_schedule$key)) {
  stop("Duplicate keys detected in window_schedule.")
}

if (anyDuplicated(availability_summary$key)) {
  stop("Duplicate keys detected in availability_summary.")
}

if (nrow(window_schedule) != 16) {
  stop(
    "Expected 16 year-window combinations, found ",
    nrow(window_schedule), "."
  )
}

# ----------------------------------------------------------
# 12.4. Rebuild clean source tables
```

```r
# Avoid .x and .y duplicate columns
# ------------------------------------------------------------

schedule_clean <- window_schedule %>%
  select(all_of(required_schedule_columns))

availability_clean <- availability_summary %>%
  select(all_of(required_availability_columns))

# Confirm every scheduled window has availability metadata
missing_availability_keys <- setdiff(
  schedule_clean$key,
  availability_clean$key
)

if (length(missing_availability_keys) > 0) {
  stop(
    "Availability information is missing for: ",
    paste(missing_availability_keys, collapse = ", ")
  )
}

# ------------------------------------------------------------
# 12.5. Match existing Earth Engine composites
# IMPORTANT: use seasonal_composites directly
# ------------------------------------------------------------

composite_keys <- names(seasonal_composites)

if (is.null(composite_keys)) {
  stop(
    "seasonal_composites must be a named list ",
    "with window keys as names."
  )
}

image_available <- vapply(
  schedule_clean$key,
  function(k) {
    if (!k %in% composite_keys) {
      return(FALSE)
    }

    !is.null(seasonal_composites[[k]])
  },
  logical(1)
)

# ------------------------------------------------------------
# 12.6. Construct the final selection manifest
# ------------------------------------------------------------

final_selection <- schedule_clean %>%
```

```r
  left_join(
    availability_clean,
    by = "key"
  ) %>%
  mutate(
    image_available = image_available,

    selected_source = if_else(
      image_available,
      "expanded_window_composite",
      "none"
    ),

    selection_status = case_when(
      !image_available ~ "image_unavailable",

      is.na(scene_count) | scene_count < 1 ~
        "review_required_no_scenes",

      is.na(valid_pixel_coverage_pct) ~
        "review_required_missing_coverage",

      is.na(coverage_status) |
        coverage_status != "adequate_coverage" ~
        "review_required_coverage",

      TRUE ~ "provisional_expanded_composite"
    ),

    selection_notes = paste(
      "Uses the expanded date window;",
      "not restricted to the exact target dates.",
      "Spatial coverage does not establish",
      "land-cover classification accuracy."
    )
  )

# The subsequent QC, mapping and spectral sections use selected_composites.
# At this stage the workflow selects available expanded-window composites;
# the status columns continue to flag composites requiring manual review.
selected_keys <- final_selection$key[final_selection$image_available]
selected_composites <- seasonal_composites[selected_keys]

# ------------------------------------------------------------
# 12.7. Run validation checks
# ------------------------------------------------------------

stopifnot(
  nrow(final_selection) == 16,
  !anyDuplicated(final_selection$key),
  all(final_selection$key %in% schedule_clean$key),
  setequal(final_selection$key, schedule_clean$key)
)
```

```r
# Report image availability
n_images_available <- sum(
  final_selection$image_available,
  na.rm = TRUE
)

cat(
  "\nCOMPOSITE SELECTION VALIDATION\n",
  "----------------------------------------\n",
  "Scheduled windows: ", nrow(final_selection), "\n",
  "Images available:  ", n_images_available, "\n",
  "Images missing:    ",
  sum(!final_selection$image_available), "\n",
  "Unique window keys: ",
  dplyr::n_distinct(final_selection$key), "\n",
  sep = ""
)
```

```
##
## COMPOSITE SELECTION VALIDATION
## ----------------------------------------
## Scheduled windows: 16
## Images available:  16
## Images missing:    0
## Unique window keys: 16
```

```r
# ------------------------------------------------------------
# 12.8. Print the complete selection manifest
# ------------------------------------------------------------

print(
  final_selection %>%
    select(
      year, window, key,
      target_start, target_end,
      expanded_start, expanded_end,
      scene_count,
      valid_pixel_coverage_pct,
      coverage_status,
      selected_source,
      selection_status,
      image_available
    ) %>%
    arrange(year, window),
  n = Inf,
  width = Inf
)
```

```
## # A tibble: 16 x 13
##     year window  key          target_start target_end expanded_start
##    <dbl> <chr>   <chr>        <date>       <date>     <date>
##  1  2000 dry_feb 2000_dry_feb 2000-02-10   2000-02-20 2000-01-11
##  2  2000 dry_sep 2000_dry_sep 2000-09-15   2000-09-25 2000-08-16
##  3  2000 wet_dec 2000_wet_dec 2000-12-05   2000-12-30 2000-11-05
```

```
##  4  2000 wet_may 2000_wet_may 2000-05-05   2000-05-30 2000-04-05
##  5  2010 dry_feb 2010_dry_feb 2010-02-10   2010-02-20 2010-01-11
##  6  2010 dry_sep 2010_dry_sep 2010-09-15   2010-09-25 2010-08-16
##  7  2010 wet_dec 2010_wet_dec 2010-12-05   2010-12-30 2010-11-05
##  8  2010 wet_may 2010_wet_may 2010-05-05   2010-05-30 2010-04-05
##  9  2020 dry_feb 2020_dry_feb 2020-02-10   2020-02-20 2020-01-11
## 10  2020 dry_sep 2020_dry_sep 2020-09-15   2020-09-25 2020-08-16
## 11  2020 wet_dec 2020_wet_dec 2020-12-05   2020-12-30 2020-11-05
## 12  2020 wet_may 2020_wet_may 2020-05-05   2020-05-30 2020-04-05
## 13  2025 dry_feb 2025_dry_feb 2025-02-10   2025-02-20 2025-01-11
## 14  2025 dry_sep 2025_dry_sep 2025-09-15   2025-09-25 2025-08-16
## 15  2025 wet_dec 2025_wet_dec 2025-12-05   2025-12-30 2025-11-05
## 16  2025 wet_may 2025_wet_may 2025-05-05   2025-05-30 2025-04-05
##    expanded_end scene_count valid_pixel_coverage_pct coverage_status
##    <date>             <int>                    <dbl> <chr>
##  1 2000-03-21             4                     99.9 adequate_coverage
##  2 2000-10-25             3                     99.9 adequate_coverage
##  3 2001-01-29             6                     88.9 adequate_coverage
##  4 2000-06-29             5                     92.7 adequate_coverage
##  5 2010-03-22             7                     95.5 adequate_coverage
##  6 2010-10-25             4                     98.5 adequate_coverage
##  7 2011-01-29            17                     99.4 adequate_coverage
##  8 2010-06-29             6                     84.2 adequate_coverage
##  9 2020-03-21            16                     99.5 adequate_coverage
## 10 2020-10-25            17                     99.9 adequate_coverage
## 11 2021-01-29            19                     99.8 adequate_coverage
## 12 2020-06-29            22                     99.9 adequate_coverage
## 13 2025-03-22            18                     99.9 adequate_coverage
## 14 2025-10-25            17                     99.9 adequate_coverage
## 15 2026-01-29            19                     99.9 adequate_coverage
## 16 2025-06-29            21                     99.9 adequate_coverage
##    selected_source           selection_status               image_available
##    <chr>                     <chr>                          <lgl>
##  1 expanded_window_composite provisional_expanded_composite TRUE
##  2 expanded_window_composite provisional_expanded_composite TRUE
##  3 expanded_window_composite provisional_expanded_composite TRUE
##  4 expanded_window_composite provisional_expanded_composite TRUE
##  5 expanded_window_composite provisional_expanded_composite TRUE
##  6 expanded_window_composite provisional_expanded_composite TRUE
##  7 expanded_window_composite provisional_expanded_composite TRUE
##  8 expanded_window_composite provisional_expanded_composite TRUE
##  9 expanded_window_composite provisional_expanded_composite TRUE
## 10 expanded_window_composite provisional_expanded_composite TRUE
## 11 expanded_window_composite provisional_expanded_composite TRUE
## 12 expanded_window_composite provisional_expanded_composite TRUE
## 13 expanded_window_composite provisional_expanded_composite TRUE
## 14 expanded_window_composite provisional_expanded_composite TRUE
## 15 expanded_window_composite provisional_expanded_composite TRUE
## 16 expanded_window_composite provisional_expanded_composite TRUE
```

```
# ------------------------------------------------------------
# 12.9. Save the manifest
# ------------------------------------------------------------
```

```r
manifest_file <- file.path(
  plot_dir,
  "Tharaka_Landsat_Final_Selection_Manifest.csv"
)

write.csv(
  final_selection,
  manifest_file,
  row.names = FALSE,
  na = ""
)

# ----------------------------------------------------------
# 12.10. Save a diagnostic summary
# ----------------------------------------------------------

selection_summary <- final_selection %>%
  group_by(
    year,
    selection_status,
    coverage_status
  ) %>%
  summarise(
    number_of_windows = n(),

    images_available = sum(
      image_available,
      na.rm = TRUE
    ),

    minimum_coverage_pct = if (
      all(is.na(valid_pixel_coverage_pct))
    ) {
      NA_real_
    } else {
      min(
        valid_pixel_coverage_pct,
        na.rm = TRUE
      )
    },

    maximum_coverage_pct = if (
      all(is.na(valid_pixel_coverage_pct))
    ) {
      NA_real_
    } else {
      max(
        valid_pixel_coverage_pct,
        na.rm = TRUE
      )
    },

    .groups = "drop"
```

```
  )

summary_file <- file.path(
  plot_dir,
  "Tharaka_Landsat_Selection_Diagnostic_Summary.csv"
)

write.csv(
  selection_summary,
  summary_file,
  row.names = FALSE,
  na = ""
)

# ------------------------------------------------------------
# 12.11. Final confirmation
# ------------------------------------------------------------

cat(
  "\nSECTION 12 COMPLETED\n",
  "----------------------------------------\n",
  "Manifest: ",
  normalizePath(
    manifest_file,
    winslash = "/",
    mustWork = TRUE
  ), "\n",
  "Diagnostic summary: ",
  normalizePath(
    summary_file,
    winslash = "/",
    mustWork = TRUE
  ), "\n",
  "----------------------------------------\n",
  sep = ""
)
```

```
##
## SECTION 12 COMPLETED
## ----------------------------------------
## Manifest: C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Landsat_Enhanced_Maps/Tharaka_Landsat_Final_Selecti
## Diagnostic summary: C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Landsat_Enhanced_Maps/Tharaka_Landsat_Sel
## ----------------------------------------
```

```
if (
  all(final_selection$image_available) &&
  all(
    final_selection$selection_status ==
      "provisional_expanded_composite"
  )
) {
  message(
    "SUCCESS: All 16 windows have available ",
    "expanded-window composites and passed ",
```

```
    "the current coverage-status checks."
  )
} else {
  warning(
    "Some windows require review. ",
    "Check selection_status in final_selection."
  )
}
```

```
# ============================================================
# SECTION 13A: VISUAL QUALITY CONTROL
# ============================================================

library(rgee)
library(dplyr)
library(tibble)
library(readr)

# Centre the interactive map on Tharaka Sub-county.
Map$centerObject(roi, 10)

# Add the ROI boundary for orientation.
Map$addLayer(
  roi,
  list(color = "yellow"),
  name = "Tharaka ROI"
)
```

```
for (key in names(selected_composites)) {

  image <- selected_composites[[key]]

  if (is.null(image)) {
    message("Skipping unavailable composite: ", key)
    next
  }

  message("Adding visual QC layers: ", key)

  # True-colour RGB: red, green, blue.
  Map$addLayer(
    image,
    list(
      bands = c("red", "green", "blue"),
      min = 0,
      max = 0.30,
      gamma = 1.2
    ),
    name = paste0(key, "_RGB"),
    shown = FALSE
  )

  # False colour: near infrared, red, green.
  Map$addLayer(
```

```
    image,
    list(
      bands = c("nir", "red", "green"),
      min = 0,
      max = 0.40,
      gamma = 1.2
    ),
    name = paste0(key, "_FalseColour"),
    shown = FALSE
  )

  # NDVI: vegetation contrast.
  ndvi <- image$
    normalizedDifference(c("nir", "red"))$
    rename("NDVI")

  Map$addLayer(
    ndvi,
    list(
      min = -0.2,
      max = 0.8,
      palette = c("8c510a", "f6e8c3", "80cdc1", "01665e")
    ),
    name = paste0(key, "_NDVI"),
    shown = FALSE
  )
}

cat(
  "Visual QC layers added. Use the map layer selector to inspect ",
  "each year and season.\n"
)
```

```
## Visual QC layers added. Use the map layer selector to inspect  each year and season.
```

```
# ========================================================
# SECTION 13B: SPECTRAL QUALITY-CONTROL SUMMARY
# ========================================================

get_spectral_summary <- function(key, image, source) {

  if (is.null(image)) {
    return(tibble(
      key = key,
      source = source,
      mean_ndvi = NA_real_,
      mean_blue = NA_real_,
      mean_green = NA_real_,
      mean_red = NA_real_,
      mean_nir = NA_real_,
      mean_swir1 = NA_real_,
      mean_swir2 = NA_real_,
      qc_status = "no_composite"
    ))
```

```
}

stats <- tryCatch({

  ndvi <- image$
    normalizedDifference(c("nir", "red"))$
    rename("NDVI")

  combined <- image$select(
    c("blue", "green", "red", "nir", "swir1", "swir2")
  )$addBands(ndvi)

  result <- combined$reduceRegion(
    reducer = ee$Reducer$mean(),
    geometry = roi,
    scale = 30,
    maxPixels = 1e10,
    tileScale = 4
  )$getInfo()

  result <- reticulate::py_to_r(result)

  get_stat <- function(name) {
    value <- result[[name]]
    if (is.null(value)) NA_real_ else as.numeric(value)
  }

  tibble(
    key = key,
    source = source,
    mean_ndvi = get_stat("NDVI"),
    mean_blue = get_stat("blue"),
    mean_green = get_stat("green"),
    mean_red = get_stat("red"),
    mean_nir = get_stat("nir"),
    mean_swir1 = get_stat("swir1"),
    mean_swir2 = get_stat("swir2"),
    qc_status = "summary_calculated"
  )

}, error = function(e) {
  message("Spectral summary failed for ", key, ": ",
          conditionMessage(e))

  tibble(
    key = key,
    source = source,
    mean_ndvi = NA_real_,
    mean_blue = NA_real_,
    mean_green = NA_real_,
    mean_red = NA_real_,
    mean_nir = NA_real_,
    mean_swir1 = NA_real_,
```

```
      mean_swir2 = NA_real_,
      qc_status = "summary_failed"
    )
  })

  stats
}

spectral_summary <- bind_rows(
  lapply(names(selected_composites), function(key) {

    source <- final_selection$selected_source[
      match(key, final_selection$key)
    ]

    get_spectral_summary(
      key = key,
      image = selected_composites[[key]],
      source = source
    )
  })
) %>%
  left_join(
    final_selection %>% select(key, year, window),
    by = "key"
  ) %>%
  select(
    year, window, key, source,
    mean_ndvi, mean_blue, mean_green, mean_red,
    mean_nir, mean_swir1, mean_swir2, qc_status
  ) %>%
  arrange(year, window)

print(spectral_summary, n = Inf, width = Inf)
```

```
## # A tibble: 16 x 12
##     year window  key          source                     mean_ndvi mean_blue
##    <dbl> <chr>   <chr>        <chr>                          <dbl>     <dbl>
##  1  2000 dry_feb 2000_dry_feb expanded_window_composite     0.410    0.0496
##  2  2000 dry_sep 2000_dry_sep expanded_window_composite     0.237    0.0596
##  3  2000 wet_dec 2000_wet_dec expanded_window_composite     0.446    0.0551
##  4  2000 wet_may 2000_wet_may expanded_window_composite     0.402    0.0499
##  5  2010 dry_feb 2010_dry_feb expanded_window_composite     0.399    0.0563
##  6  2010 dry_sep 2010_dry_sep expanded_window_composite     0.234    0.0656
##  7  2010 wet_dec 2010_wet_dec expanded_window_composite     0.403    0.0615
##  8  2010 wet_may 2010_wet_may expanded_window_composite     0.523    0.0525
##  9  2020 dry_feb 2020_dry_feb expanded_window_composite     0.564    0.0389
## 10  2020 dry_sep 2020_dry_sep expanded_window_composite     0.296    0.0539
## 11  2020 wet_dec 2020_wet_dec expanded_window_composite     0.568    0.0426
## 12  2020 wet_may 2020_wet_may expanded_window_composite     0.552    0.0390
## 13  2025 dry_feb 2025_dry_feb expanded_window_composite     0.389    0.0465
## 14  2025 dry_sep 2025_dry_sep expanded_window_composite     0.321    0.0525
## 15  2025 wet_dec 2025_wet_dec expanded_window_composite     0.478    0.0441
## 16  2025 wet_may 2025_wet_may expanded_window_composite     0.628    0.0348
```

```
##    mean_green mean_red mean_nir mean_swir1 mean_swir2 qc_status
##         <dbl>    <dbl>    <dbl>      <dbl>      <dbl> <chr>
##  1     0.0747   0.0980    0.229      0.245      0.156 summary_calculated
##  2     0.0870   0.128     0.206      0.295      0.222 summary_calculated
##  3     0.0831   0.0988    0.255      0.256      0.174 summary_calculated
##  4     0.0768   0.0963    0.223      0.245      0.161 summary_calculated
##  5     0.0865   0.106     0.242      0.258      0.176 summary_calculated
##  6     0.0990   0.141     0.223      0.303      0.218 summary_calculated
##  7     0.0941   0.113     0.261      0.278      0.182 summary_calculated
##  8     0.0832   0.0916    0.288      0.243      0.150 summary_calculated
##  9     0.0691   0.0757    0.267      0.235      0.145 summary_calculated
## 10     0.0871   0.130     0.235      0.312      0.227 summary_calculated
## 11     0.0767   0.0835    0.299      0.247      0.157 summary_calculated
## 12     0.0727   0.0818    0.277      0.240      0.149 summary_calculated
## 13     0.0832   0.117     0.258      0.292      0.205 summary_calculated
## 14     0.0902   0.135     0.258      0.320      0.230 summary_calculated
## 15     0.0813   0.100     0.275      0.276      0.192 summary_calculated
## 16     0.0735   0.0753    0.323      0.234      0.143 summary_calculated
```

```
readr::write_csv(
  spectral_summary,
  "Tharaka_Landsat_Spectral_QC.csv"
)
```

```
# ============================================================
# SECTION 13C: CORRECTED NDVI AND SEASONAL COMPARISONS
# ============================================================

library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)

get_spectral_summary_v2 <- function(key, image, source) {

  if (is.null(image)) {
    return(tibble(
      key = key,
      source = source,
      mean_ndvi = NA_real_,
      mean_blue = NA_real_,
      mean_green = NA_real_,
      mean_red = NA_real_,
      mean_nir = NA_real_,
      mean_swir1 = NA_real_,
      mean_swir2 = NA_real_,
      qc_status = "no_composite"
    ))
  }

  tryCatch({

    nir <- image$select("nir")
    red <- image$select("red")
```

```
  denominator <- nir$add(red)

  # Explicit NDVI calculation; mask only zero denominators.
  ndvi <- nir$subtract(red)$
    divide(denominator)$
    updateMask(denominator$neq(0))$
    rename("NDVI")

  combined <- image$select(
    c("blue", "green", "red", "nir", "swir1", "swir2")
  )$addBands(ndvi)

  result <- combined$reduceRegion(
    reducer = ee$Reducer$mean(),
    geometry = roi,
    scale = 30,
    maxPixels = 1e10,
    tileScale = 4
  )$getInfo()

  result <- reticulate::py_to_r(result)

  get_stat <- function(name) {
    value <- result[[name]]
    if (is.null(value)) NA_real_ else as.numeric(value)
  }

  tibble(
    key = key,
    source = source,
    mean_ndvi = get_stat("NDVI"),
    mean_blue = get_stat("blue"),
    mean_green = get_stat("green"),
    mean_red = get_stat("red"),
    mean_nir = get_stat("nir"),
    mean_swir1 = get_stat("swir1"),
    mean_swir2 = get_stat("swir2"),
    qc_status = "summary_calculated"
  )

}, error = function(e) {

  message("Corrected summary failed for ", key, ": ",
          conditionMessage(e))

  tibble(
    key = key,
    source = source,
    mean_ndvi = NA_real_,
    mean_blue = NA_real_,
    mean_green = NA_real_,
    mean_red = NA_real_,
    mean_nir = NA_real_,
```

```
      mean_swir1 = NA_real_,
      mean_swir2 = NA_real_,
      qc_status = "summary_failed"
    )
  })
}


spectral_summary_v2 <- bind_rows(
  lapply(names(selected_composites), function(key) {

    source <- final_selection$selected_source[
      match(key, final_selection$key)
    ]

    get_spectral_summary_v2(
      key = key,
      image = selected_composites[[key]],
      source = source
    )
  })
) %>%
  left_join(
    final_selection %>% select(key, year, window),
    by = "key"
  ) %>%
  mutate(
    season = case_when(
      grepl("dry", window) ~ "Dry",
      grepl("wet", window) ~ "Wet",
      TRUE ~ NA_character_
    )
  ) %>%
  select(
    year, window, season, key, source,
    mean_ndvi, mean_blue, mean_green, mean_red,
    mean_nir, mean_swir1, mean_swir2, qc_status
  ) %>%
  arrange(year, match(window, windows$window))

print(spectral_summary_v2, n = Inf, width = Inf)
```

```
## # A tibble: 16 x 13
##     year window  season key          source                     mean_ndvi
##    <dbl> <chr>   <chr>  <chr>        <chr>                          <dbl>
##  1  2000 dry_feb Dry    2000_dry_feb expanded_window_composite      0.410
##  2  2000 dry_sep Dry    2000_dry_sep expanded_window_composite      0.237
##  3  2000 wet_may Wet    2000_wet_may expanded_window_composite      0.402
##  4  2000 wet_dec Wet    2000_wet_dec expanded_window_composite      0.446
##  5  2010 dry_feb Dry    2010_dry_feb expanded_window_composite      0.399
##  6  2010 dry_sep Dry    2010_dry_sep expanded_window_composite      0.234
##  7  2010 wet_may Wet    2010_wet_may expanded_window_composite      0.523
##  8  2010 wet_dec Wet    2010_wet_dec expanded_window_composite      0.403
##  9  2020 dry_feb Dry    2020_dry_feb expanded_window_composite      0.564
```

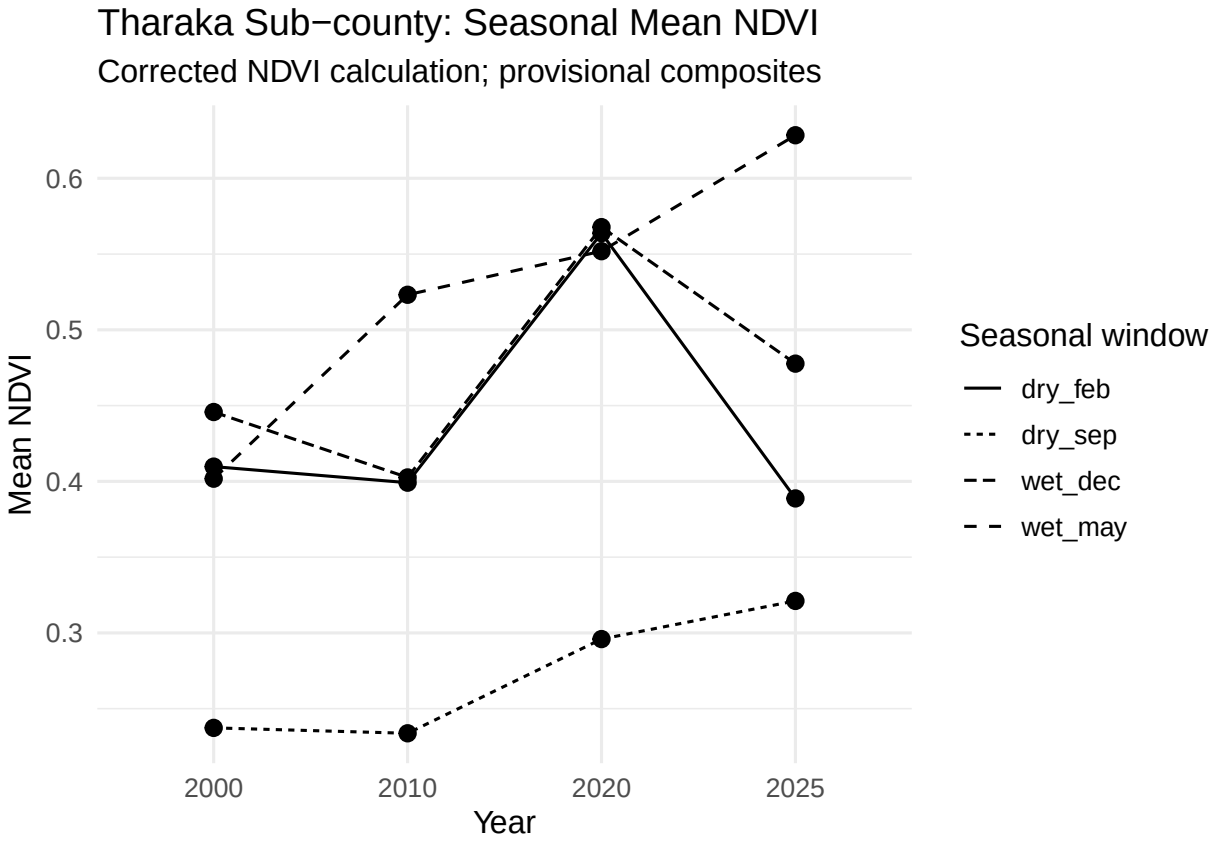
```
## 10  2020 dry_sep Dry    2020_dry_sep expanded_window_composite     0.296
## 11  2020 wet_may Wet    2020_wet_may expanded_window_composite     0.552
## 12  2020 wet_dec Wet    2020_wet_dec expanded_window_composite     0.568
## 13  2025 dry_feb Dry    2025_dry_feb expanded_window_composite     0.389
## 14  2025 dry_sep Dry    2025_dry_sep expanded_window_composite     0.321
## 15  2025 wet_may Wet    2025_wet_may expanded_window_composite     0.628
## 16  2025 wet_dec Wet    2025_wet_dec expanded_window_composite     0.478
##    mean_blue mean_green mean_red mean_nir mean_swir1 mean_swir2
##        <dbl>      <dbl>    <dbl>    <dbl>      <dbl>      <dbl>
##  1    0.0496     0.0747   0.0980    0.229      0.245      0.156
##  2    0.0596     0.0870   0.128     0.206      0.295      0.222
##  3    0.0499     0.0768   0.0963    0.223      0.245      0.161
##  4    0.0551     0.0831   0.0988    0.255      0.256      0.174
##  5    0.0563     0.0865   0.106     0.242      0.258      0.176
##  6    0.0656     0.0990   0.141     0.223      0.303      0.218
##  7    0.0525     0.0832   0.0916    0.288      0.243      0.150
##  8    0.0615     0.0941   0.113     0.261      0.278      0.182
##  9    0.0389     0.0691   0.0757    0.267      0.235      0.145
## 10    0.0539     0.0871   0.130     0.235      0.312      0.227
## 11    0.0390     0.0727   0.0818    0.277      0.240      0.149
## 12    0.0426     0.0767   0.0835    0.299      0.247      0.157
## 13    0.0465     0.0832   0.117     0.258      0.292      0.205
## 14    0.0525     0.0902   0.135     0.258      0.320      0.230
## 15    0.0348     0.0735   0.0753    0.323      0.234      0.143
## 16    0.0441     0.0813   0.100     0.275      0.276      0.192
##    qc_status
##    <chr>
##  1 summary_calculated
##  2 summary_calculated
##  3 summary_calculated
##  4 summary_calculated
##  5 summary_calculated
##  6 summary_calculated
##  7 summary_calculated
##  8 summary_calculated
##  9 summary_calculated
## 10 summary_calculated
## 11 summary_calculated
## 12 summary_calculated
## 13 summary_calculated
## 14 summary_calculated
## 15 summary_calculated
## 16 summary_calculated
```

```r
readr::write_csv(
  spectral_summary_v2,
  "Tharaka_Landsat_Spectral_QC_Corrected.csv"
)


# Plot mean NDVI by year and seasonal window.
p_ndvi <- ggplot(
  spectral_summary_v2,
  aes(
```

```
    x = factor(year),
    y = mean_ndvi,
    group = window,
    linetype = window
  )
) +
  geom_line() +
  geom_point(size = 2.5) +
  labs(
    title = "Tharaka Sub-county: Seasonal Mean NDVI",
    subtitle = "Corrected NDVI calculation; provisional composites",
    x = "Year",
    y = "Mean NDVI",
    linetype = "Seasonal window"
  ) +
  theme_minimal(base_size = 12)

print(p_ndvi)
```

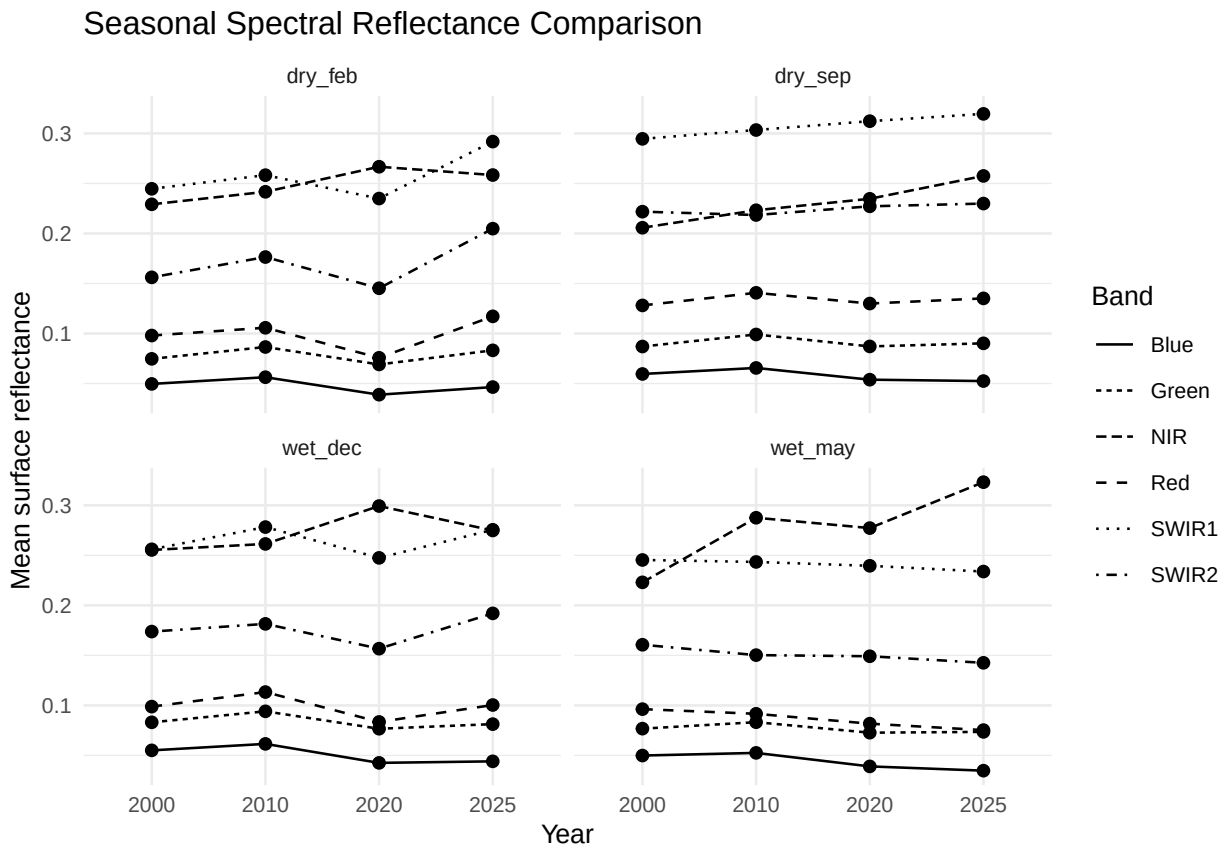



```
ggsave(
  "Tharaka_Mean_NDVI_Seasonal_Comparison.png",
  p_ndvi,
  width = 10,
  height = 6,
  dpi = 300
)
```

```r
# Plot the six reflectance bands as a compact diagnostic.
reflectance_long <- spectral_summary_v2 %>%
  pivot_longer(
    cols = c(
      mean_blue, mean_green, mean_red,
      mean_nir, mean_swir1, mean_swir2
    ),
    names_to = "band",
    values_to = "mean_reflectance"
  ) %>%
  mutate(
    band = recode(
      band,
      mean_blue = "Blue",
      mean_green = "Green",
      mean_red = "Red",
      mean_nir = "NIR",
      mean_swir1 = "SWIR1",
      mean_swir2 = "SWIR2"
    )
  )

p_reflectance <- ggplot(
  reflectance_long,
  aes(
    x = factor(year),
    y = mean_reflectance,
    group = band,
    linetype = band
  )
) +
  geom_line() +
  geom_point(size = 1.8) +
  facet_wrap(~ window, ncol = 2) +
  labs(
    title = "Seasonal Spectral Reflectance Comparison",
    x = "Year",
    y = "Mean surface reflectance",
    linetype = "Band"
  ) +
  theme_minimal(base_size = 10)

print(p_reflectance)
```




```
ggsave(
  "Tharaka_Spectral_Reflectance_Comparison.png",
  p_reflectance,
  width = 12,
  height = 9,
  dpi = 300
)
```

```
# ============================================================
# SECTION 14: REFERENCE-SAMPLING FRAMEWORK
# ============================================================

library(dplyr)
library(tibble)
library(readr)
library(reticulate)

# ------------------------------------------------------------
# 14.1 Sampling settings
# ------------------------------------------------------------

n_candidates <- 500
sampling_seed <- 2026

# Confirm that roi is the intended Tharaka Sub-county boundary
stopifnot(!is.null(roi))
```

```r
# ------------------------------------------------------------
# 14.2 Generate random candidate locations in the ROI
# ------------------------------------------------------------

candidate_points <- ee$FeatureCollection$randomPoints(
  region = roi,
  points = n_candidates,
  seed = sampling_seed,
  maxError = 30
)

# Attach an identifier and geographic coordinates.
coordinate_callback <- reticulate::py_eval(
  paste0(
    "lambda f: f.set({",
    "'point_id': f.get('system:index'),",
    "'longitude': f.geometry().coordinates().get(0),",
    "'latitude': f.geometry().coordinates().get(1)",
    "})"
  ),
  convert = FALSE
)

candidate_points <- ee_method(
  candidate_points, "map", coordinate_callback
)

# Retrieve the coordinates as ordinary R vectors.
point_ids <- ee_get_info(
  ee_method(candidate_points, "aggregate_array", "point_id")
)

longitudes <- ee_get_info(
  ee_method(candidate_points, "aggregate_array", "longitude")
)

latitudes <- ee_get_info(
  ee_method(candidate_points, "aggregate_array", "latitude")
)

reference_points <- tibble(
  point_id = as.character(unlist(point_ids)),
  longitude = as.numeric(unlist(longitudes)),
  latitude = as.numeric(unlist(latitudes))
) %>%
  arrange(point_id)

stopifnot(nrow(reference_points) > 0)

cat("Candidate points generated:", nrow(reference_points), "\n")
```

```
## Candidate points generated: 500
```

```r
# ------------------------------------------------------------
# 14.3 Create a manual reference-label template
# ------------------------------------------------------------

reference_template <- reference_points

study_years <- c(2000, 2010, 2020, 2025)

for (yr in study_years) {
  reference_template[[paste0("class_", yr)]] <- NA_character_
  reference_template[[paste0("confidence_", yr)]] <- NA_character_
  reference_template[[paste0("evidence_", yr)]] <- NA_character_
  reference_template[[paste0("notes_", yr)]] <- NA_character_
}

reference_template <- reference_template %>%
  mutate(
    class_2000 = as.character(class_2000),
    class_2010 = as.character(class_2010),
    class_2020 = as.character(class_2020),
    class_2025 = as.character(class_2025)
  )

readr::write_csv(
  reference_template,
  "Tharaka_Reference_Sampling_Template.csv"
)

# Save the unlabelled coordinates separately as well.
readr::write_csv(
  reference_points,
  "Tharaka_Reference_Candidate_Points.csv"
)

# ------------------------------------------------------------
# 14.4 Optional: display candidate locations in the map
# ------------------------------------------------------------

Map$centerObject(roi, 10)

Map$addLayer(
  candidate_points,
  list(color = "cyan"),
  name = "Reference candidate points",
  shown = TRUE
)
```

```r
print(head(reference_template, 10))
```

```
## # A tibble: 10 x 19
##    point_id longitude latitude class_2000 confidence_2000 evidence_2000
##    <chr>        <dbl>    <dbl> <chr>      <chr>           <chr>
##  1 0             38.1  -0.214  <NA>       <NA>            <NA>
##  2 1             38.1  -0.268  <NA>       <NA>            <NA>
```

```
##  3 10            38.0  -0.304  <NA>       <NA>            <NA>
##  4 100           38.1  -0.0717 <NA>       <NA>            <NA>
##  5 101           38.1  -0.152  <NA>       <NA>            <NA>
##  6 102           38.1  -0.209  <NA>       <NA>            <NA>
##  7 103           38.1  -0.136  <NA>       <NA>            <NA>
##  8 104           38.0   0.0388 <NA>       <NA>            <NA>
##  9 105           37.8  -0.246  <NA>       <NA>            <NA>
## 10 106           38.2  -0.0487 <NA>       <NA>            <NA>
## # i 13 more variables: notes_2000 <chr>, class_2010 <chr>,
## #   confidence_2010 <chr>, evidence_2010 <chr>, notes_2010 <chr>,
## #   class_2020 <chr>, confidence_2020 <chr>, evidence_2020 <chr>,
## #   notes_2020 <chr>, class_2025 <chr>, confidence_2025 <chr>,
## #   evidence_2025 <chr>, notes_2025 <chr>
```

```
# ============================================================
# SECTION 14A: VERIFY AND ORDER REFERENCE POINTS
# ============================================================

reference_template <- reference_template %>%
  mutate(
    point_number = suppressWarnings(as.integer(point_id))
  ) %>%
  arrange(point_number) %>%
  select(-point_number)

# Check the number of points and coordinate validity.
sampling_qc <- tibble(
  total_points = nrow(reference_template),
  unique_ids = n_distinct(reference_template$point_id),
  missing_longitude = sum(is.na(reference_template$longitude)),
  missing_latitude = sum(is.na(reference_template$latitude)),
  duplicate_coordinates = sum(
    duplicated(reference_template[c("longitude", "latitude")])
  )
)

print(sampling_qc)
```

```
## # A tibble: 1 x 5
##   total_points unique_ids missing_longitude missing_latitude
##          <int>      <int>             <int>            <int>
## 1          500        500                 0                0
## # i 1 more variable: duplicate_coordinates <int>
```

```
# Save the ordered template.
readr::write_csv(
  reference_template,
  "Tharaka_Reference_Sampling_Template.csv"
)
```

```
# ============================================================
# SECTION 15: PILOT REFERENCE SAMPLE
# ============================================================

set.seed(2026)
```

```r
pilot_n <- min(50, nrow(reference_template))

pilot_reference <- reference_template %>%
  slice_sample(n = pilot_n) %>%
  arrange(point_id)

readr::write_csv(
  pilot_reference,
  "Tharaka_Reference_Pilot_50.csv"
)

cat("Pilot sample size:", nrow(pilot_reference), "\n")
```

```
## Pilot sample size: 50
```

```r
# ============================================================
# SECTION 15: PILOT REFERENCE SAMPLE QUALITY CONTROL
# ============================================================

library(dplyr)
library(readr)

# Confirm sample size
stopifnot(nrow(pilot_reference) == 50)

# Check unique IDs and coordinates
pilot_qc <- pilot_reference %>%
  summarise(
    total_points = n(),
    unique_ids = n_distinct(point_id),
    missing_longitude = sum(is.na(longitude)),
    missing_latitude = sum(is.na(latitude)),
    duplicate_coordinates = sum(duplicated(
      paste(longitude, latitude, sep = "_")
    ))
  )

print(pilot_qc)
```

```
## # A tibble: 1 x 5
##   total_points unique_ids missing_longitude missing_latitude
##          <int>      <int>             <int>            <int>
## 1           50         50                 0                0
## # i 1 more variable: duplicate_coordinates <int>
```

```r
# Inspect the first records and columns
print(head(pilot_reference, 6))
```

```
## # A tibble: 6 x 19
##   point_id longitude latitude class_2000 confidence_2000 evidence_2000
##   <chr>        <dbl>    <dbl> <chr>      <chr>           <chr>
## 1 107           38.2 -0.0731  <NA>       <NA>            <NA>
## 2 109           38.2 -0.112   <NA>       <NA>            <NA>
## 3 11            38.1 -0.0745  <NA>       <NA>            <NA>
## 4 110           38.1 -0.0273  <NA>       <NA>            <NA>
```

```
## 5 120           38.2 -0.0842  <NA>       <NA>            <NA>
## 6 132           37.9  0.00419 <NA>       <NA>            <NA>
## # i 13 more variables: notes_2000 <chr>, class_2010 <chr>,
## #   confidence_2010 <chr>, evidence_2010 <chr>, notes_2010 <chr>,
## #   class_2020 <chr>, confidence_2020 <chr>, evidence_2020 <chr>,
## #   notes_2020 <chr>, class_2025 <chr>, confidence_2025 <chr>,
## #   evidence_2025 <chr>, notes_2025 <chr>
```

```
print(names(pilot_reference))
```

```
##  [1] "point_id"        "longitude"       "latitude"        "class_2000"
##  [5] "confidence_2000" "evidence_2000"   "notes_2000"      "class_2010"
##  [9] "confidence_2010" "evidence_2010"   "notes_2010"      "class_2020"
## [13] "confidence_2020" "evidence_2020"   "notes_2020"      "class_2025"
## [17] "confidence_2025" "evidence_2025"   "notes_2025"
```

```
# Save a clean pilot copy
write_csv(
  pilot_reference,
  "Tharaka_Reference_Pilot_50.csv"
)
```

```
# ============================================================
# SECTION 16: PREPARE HISTORICAL REFERENCE LABELS
# ============================================================

library(dplyr)
library(tidyr)
library(readr)

# 1. Sort point IDs numerically
pilot_reference <- pilot_reference %>%
  mutate(point_id_numeric = as.integer(point_id)) %>%
  arrange(point_id_numeric)

# 2. Create one row per point per target year
pilot_review <- pilot_reference %>%
  select(point_id, longitude, latitude) %>%
  crossing(year = c(2000, 2010, 2020, 2025)) %>%
  mutate(
    land_cover = NA_character_,
    confidence = NA_character_,
    evidence = NA_character_,
    notes = NA_character_
  ) %>%
  arrange(point_id, year)

# 3. Check the review worksheet
cat("Review records:", nrow(pilot_review), "\n")
```

```
## Review records: 200
```

```
cat("Expected records:", 50 * 4, "\n")
```

```
## Expected records: 200
```

```r
print(head(pilot_review, 8))
```

```
## # A tibble: 8 x 8
##   point_id longitude latitude  year land_cover confidence evidence notes
##   <chr>        <dbl>    <dbl> <dbl> <chr>      <chr>      <chr>    <chr>
## 1 107           38.2  -0.0731  2000 <NA>       <NA>       <NA>     <NA>
## 2 107           38.2  -0.0731  2010 <NA>       <NA>       <NA>     <NA>
## 3 107           38.2  -0.0731  2020 <NA>       <NA>       <NA>     <NA>
## 4 107           38.2  -0.0731  2025 <NA>       <NA>       <NA>     <NA>
## 5 109           38.2  -0.112   2000 <NA>       <NA>       <NA>     <NA>
## 6 109           38.2  -0.112   2010 <NA>       <NA>       <NA>     <NA>
## 7 109           38.2  -0.112   2020 <NA>       <NA>       <NA>     <NA>
## 8 109           38.2  -0.112   2025 <NA>       <NA>       <NA>     <NA>
```

```r
# 4. Save the review worksheet
write_csv(
  pilot_review,
  "Tharaka_Historical_Reference_Review_Template.csv"
)
```

```r
# ============================================================
# SECTION 16A: VALIDATE AND SAVE REVIEW WORKSHEET
# ============================================================

library(dplyr)
library(readr)

stopifnot(nrow(pilot_review) == 200)

# Confirm each point has four years
year_check <- pilot_review %>%
  count(point_id, name = "n_years")

stopifnot(all(year_check$n_years == 4))

# Confirm the expected years
stopifnot(
  setequal(unique(pilot_review$year), c(2000, 2010, 2020, 2025))
)

# Save the worksheet
write_csv(
  pilot_review,
  "Tharaka_Historical_Reference_Review_Template.csv"
)

cat("Blank template saved separately; reviewed labels are preserved in Tharaka_Historical_Reference_Revi
```

```
## Blank template saved separately; reviewed labels are preserved in Tharaka_Historical_Reference_Revie
```

```r
cat("Worksheet saved successfully.\n")
```

```
## Worksheet saved successfully.
```

```r
cat("Records:", nrow(pilot_review), "\n")
```

```
## Records: 200
```

```
cat("Unique points:", n_distinct(pilot_review$point_id), "\n")
```

```
## Unique points: 50
```

```
# Run only if these packages are not installed
# install.packages(c("leaflet", "htmlwidgets", "htmltools"))
```

```
# ========================================================
# SECTION 17.1: INTERACTIVE HISTORICAL IMAGERY MAP
# ========================================================

library(leaflet)
library(htmlwidgets)
library(dplyr)
library(sf)
library(reticulate)

# Confirm the required objects exist
stopifnot(
  exists("ee"),
  exists("selected_composites"),
  exists("pilot_reference")
)

stopifnot(nrow(pilot_reference) > 0)

# Convert pilot coordinates to an sf object
pilot_sf <- st_as_sf(
  pilot_reference,
  coords = c("longitude", "latitude"),
  crs = 4326,
  remove = FALSE
)

# Visualisation parameters for Landsat surface reflectance
vis_params <- list(
  bands = list("red", "green", "blue"),
  min = 0.0,
  max = 0.30,
  gamma = 1.2
)

# Obtain Earth Engine tile URLs for each selected composite
composite_names <- names(selected_composites)

if (is.null(composite_names) || length(composite_names) == 0) {
  stop("selected_composites must be a named list of images.")
}

tile_urls <- list()

for (nm in composite_names) {
```

```
  img <- selected_composites[[nm]]

  map_info <- ee_method(
    img,
    "getMapId",
    vis_params
  )

  tile_fetcher <- map_info[["tile_fetcher"]]

  tile_url <- reticulate::py_to_r(
    reticulate::py_get_attr(
      tile_fetcher,
      "url_format"
    )
  )

  tile_urls[[nm]] <- tile_url
}

# Build the interactive map
m <- leaflet(
  options = leafletOptions(preferCanvas = TRUE)
) %>%
  addProviderTiles("CartoDB.Positron", group = "Base map") %>%
  addCircleMarkers(
    data = pilot_sf,
    lng = ~longitude,
    lat = ~latitude,
    radius = 4,
    stroke = TRUE,
    weight = 1,
    fillOpacity = 0.9,
    label = ~paste0("Point ", point_id),
    popup = ~paste0(
      "<b>Point ID:</b> ", point_id,
      "<br><b>Longitude:</b> ", round(longitude, 5),
      "<br><b>Latitude:</b> ", round(latitude, 5)
    ),
    group = "Pilot points"
  )

# Add each composite as a switchable imagery layer
for (nm in names(tile_urls)) {
  m <- m %>%
    addTiles(
      urlTemplate = tile_urls[[nm]],
      group = nm,
      options = tileOptions(opacity = 1)
    )
}

# Add layer controls
```

```r
m <- m %>%
  addLayersControl(
    baseGroups = "Base map",
    overlayGroups = c("Pilot points", names(tile_urls)),
    options = layersControlOptions(collapsed = FALSE)
  ) %>%
  hideGroup(names(tile_urls)) %>%
  fitBounds(
    lng1 = min(pilot_reference$longitude, na.rm = TRUE),
    lat1 = min(pilot_reference$latitude, na.rm = TRUE),
    lng2 = max(pilot_reference$longitude, na.rm = TRUE),
    lat2 = max(pilot_reference$latitude, na.rm = TRUE)
  )

# Display in RStudio Viewer
m
```

```r
# Save the interactive map
htmlwidgets::saveWidget(
  m,
  "Tharaka_Historical_Imagery_Review.html",
  selfcontained = TRUE
)
```

```r
# ============================================================
# SECTION 18: REFERENCE-LABEL VALIDATION
# ============================================================

library(dplyr)
library(readr)
library(tidyr)

# Read the reviewed worksheet
reference_file <- "Tharaka_Historical_Reference_Review.csv"

reference_review <- read_csv(
  reference_file,
  show_col_types = FALSE
)

# Allowed values
allowed_years <- c(2000, 2010, 2020, 2025)

allowed_classes <- c(
  "forest", "grassland", "bushland", "farmland",
  "settlement", "other", "uncertain"
)

allowed_confidence <- c("high", "medium", "low")

allowed_evidence <- c(
  "historical_imagery", "existing_map",
  "field", "other"
)
```

```r
# Standardise text without assigning any labels
reference_review <- reference_review %>%
  mutate(
    point_id = as.character(point_id),
    year = as.integer(year),
    land_cover = tolower(trimws(land_cover)),
    confidence = tolower(trimws(confidence)),
    evidence = tolower(trimws(evidence)),
    notes = trimws(notes)
  )

# Identify invalid values and missing entries
reference_review <- reference_review %>%
  mutate(
    invalid_year = is.na(year) | !year %in% allowed_years,

    missing_class = is.na(land_cover) | land_cover == "",

    invalid_class = !missing_class &
      !land_cover %in% allowed_classes,

    missing_confidence = is.na(confidence) | confidence == "",

    invalid_confidence = !missing_confidence &
      !confidence %in% allowed_confidence,

    missing_evidence = is.na(evidence) | evidence == "",

    invalid_evidence = !missing_evidence &
      !evidence %in% allowed_evidence,

    needs_review =
      invalid_year |
      missing_class |
      invalid_class |
      missing_confidence |
      invalid_confidence |
      missing_evidence |
      invalid_evidence |
      land_cover == "uncertain" |
      confidence == "low"
  )

# Summary of the validation
validation_summary <- reference_review %>%
  summarise(
    total_records = n(),
    unique_points = n_distinct(point_id),
    missing_classes = sum(missing_class),
    invalid_classes = sum(invalid_class),
    missing_confidence = sum(missing_confidence),
    invalid_confidence = sum(invalid_confidence),
    missing_evidence = sum(missing_evidence),
```

```
    invalid_evidence = sum(invalid_evidence),
    records_needing_review = sum(needs_review)
  )

print(validation_summary)
```

```
## # A tibble: 1 x 9
##   total_records unique_points missing_classes invalid_classes missing_confidence
##           <int>         <int>           <int>           <int>              <int>
## 1           200            50             200               0                200
## # i 4 more variables: invalid_confidence <int>, missing_evidence <int>,
## #   invalid_evidence <int>, records_needing_review <int>
```

```
# Label counts by year
class_counts <- reference_review %>%
  filter(
    !missing_class,
    !invalid_class,
    land_cover != "uncertain"
  ) %>%
  count(year, land_cover, name = "n") %>%
  arrange(year, land_cover)

print(class_counts)
```

```
## # A tibble: 0 x 3
## # i 3 variables: year <int>, land_cover <chr>, n <int>
```

```
# Export all records needing attention
review_flags <- reference_review %>%
  filter(needs_review)

write_csv(
  review_flags,
  "Tharaka_Reference_Review_Flags.csv"
)

# Export the validation summary and class counts
write_csv(
  validation_summary,
  "Tharaka_Reference_Validation_Summary.csv"
)

write_csv(
  class_counts,
  "Tharaka_Reference_Class_Counts.csv"
)

cat("Validation completed.\n")
```

```
## Validation completed.
```

```
cat("Records flagged for review:", nrow(review_flags), "\n")
```

```
## Records flagged for review: 200
```

```r
# ============================================================
# SECTION 18B: PREPARE CANDIDATE TRAINING LABELS
# ============================================================

reference_training <- reference_review %>%
  filter(
    !invalid_year,
    !missing_class,
    !invalid_class,
    !missing_confidence,
    !invalid_confidence,
    !missing_evidence,
    !invalid_evidence,
    land_cover != "uncertain",
    confidence %in% c("high", "medium"),
    evidence %in% allowed_evidence
  ) %>%
  select(
    point_id,
    longitude,
    latitude,
    year,
    land_cover,
    confidence,
    evidence,
    notes
  )

# Summarise the candidate training records
training_summary <- reference_training %>%
  count(year, land_cover, name = "n") %>%
  arrange(year, land_cover)

print(training_summary)
```

```
## # A tibble: 0 x 3
## # i 3 variables: year <int>, land_cover <chr>, n <int>
```

```r
write_csv(
  reference_training,
  "Tharaka_Reference_Training_Candidates.csv"
)

cat(
  "Candidate training records:",
  nrow(reference_training),
  "\n"
)
```

```
## Candidate training records: 0
```

```r
# ============================================================
# SECTION 19: EXCEL REFERENCE-LABELLING WORKBOOK
# ============================================================
```

```r
# Install once if necessary
if (!requireNamespace("openxlsx", quietly = TRUE)) {
  install.packages("openxlsx")
}

library(openxlsx)
library(dplyr)

# Prepare a clean labelling sheet
labelling_sheet <- pilot_review %>%
  select(point_id, longitude, latitude, year) %>%
  mutate(
    land_cover = "",
    confidence = "",
    evidence = "",
    notes = ""
  ) %>%
  arrange(year, point_id)

# Create workbook
wb <- createWorkbook()

addWorksheet(wb, "Labelling_Sheet")
addWorksheet(wb, "Progress_Summary")
addWorksheet(wb, "Instructions")

# Write the labelling sheet
writeData(
  wb, "Labelling_Sheet",
  labelling_sheet,
  withFilter = TRUE
)

# Add drop-down validation lists
class_values <- c(
  "forest", "grassland", "bushland", "farmland",
  "settlement", "other", "uncertain"
)

confidence_values <- c("high", "medium", "low")

evidence_values <- c(
  "historical_imagery", "existing_map",
  "field", "other"
)

n_rows <- nrow(labelling_sheet) + 1

dataValidation(
  wb, "Labelling_Sheet",
  cols = 5, rows = 2:n_rows,
  type = "list",
  value = paste0('"', paste(class_values, collapse = ","), '"')
```

```
)

dataValidation(
  wb, "Labelling_Sheet",
  cols = 6, rows = 2:n_rows,
  type = "list",
  value = paste0('"', paste(confidence_values, collapse = ","), '"')
)

dataValidation(
  wb, "Labelling_Sheet",
  cols = 7, rows = 2:n_rows,
  type = "list",
  value = paste0('"', paste(evidence_values, collapse = ","), '"')
)

# Instructions sheet
instructions <- data.frame(
  Item = c(
    "Purpose",
    "Land-cover classes",
    "Confidence",
    "Evidence",
    "Uncertain labels",
    "Important"
  ),
  Guidance = c(
    "Interpret each point separately for each target year.",
    paste(class_values, collapse = ", "),
    paste(confidence_values, collapse = ", "),
    paste(evidence_values, collapse = ", "),
    "Use uncertain when evidence does not support a confident class.",
    "Do not guess labels. Record the evidence and relevant notes."
  )
)

writeData(wb, "Instructions", instructions)

# Create progress summary formulas
progress <- data.frame(
  Year = c(2000, 2010, 2020, 2025),
  Total_Records = 50,
  Labelled = NA_integer_,
  Remaining = NA_integer_
)

writeData(wb, "Progress_Summary", progress)

for (i in 2:5) {
  wb_formula_row <- i
  wb_sheet_start <- 2
  wb_sheet_end <- n_rows
```

```
  writeFormula(
    wb,
    "Progress_Summary",
    x = sprintf(
      'COUNTIFS(Labelling_Sheet!$D$%d:$D$%d,A%d,Labelling_Sheet!$E$%d:$E$%d,"<>")',
      wb_sheet_start, wb_sheet_end, wb_formula_row,
      wb_sheet_start, wb_sheet_end
    ),
    startCol = 3,
    startRow = wb_formula_row
  )

  writeFormula(
    wb,
    "Progress_Summary",
    x = sprintf("B%d-C%d", wb_formula_row, wb_formula_row),
    startCol = 4,
    startRow = wb_formula_row
  )
}

# Formatting
header_style <- createStyle(
  textDecoration = "bold",
  fgFill = "#D9EAF7",
  border = "Bottom"
)

for (sheet in c(
  "Labelling_Sheet",
  "Progress_Summary",
  "Instructions"
)) {
  addStyle(
    wb, sheet, header_style,
    rows = 1, cols = 1:8,
    gridExpand = TRUE
  )
  freezePane(wb, sheet, firstRow = TRUE)
  setColWidths(wb, sheet, cols = 1:8, widths = "auto")
}

# Save workbook
output_file <- "Tharaka_Reference_Labelling_Workbook_Template.xlsx"

saveWorkbook(
  wb,
  output_file,
  overwrite = TRUE
)

cat("Workbook created:", output_file, "\n")
```

```
## Workbook created: Tharaka_Reference_Labelling_Workbook_Template.xlsx
```

```r
# ========================================================
# SECTION 20: IMPORT REFERENCE-LABELLING WORKBOOK
# ========================================================

library(openxlsx)
library(dplyr)
library(readr)

# Select the completed workbook from your computer
workbook_path <- file.choose()

# Import the labelling sheet
reference_review <- read.xlsx(
  workbook_path,
  sheet = "Labelling_Sheet",
  detectDates = FALSE
) %>%
  mutate(
    point_id = as.character(point_id),
    year = as.integer(year),
    longitude = as.numeric(longitude),
    latitude = as.numeric(latitude),
    land_cover = tolower(trimws(as.character(land_cover))),
    confidence = tolower(trimws(as.character(confidence))),
    evidence = tolower(trimws(as.character(evidence))),
    notes = as.character(notes)
  )

# Convert empty Excel cells to NA
reference_review <- reference_review %>%
  mutate(
    across(
      c(land_cover, confidence, evidence, notes),
      ~ na_if(trimws(.x), "")
    )
  )

# Inspect the imported data
cat("Imported records:", nrow(reference_review), "\n")
cat("Unique points:", n_distinct(reference_review$point_id), "\n")

print(head(reference_review, 10))
print(names(reference_review))

# Check labelling progress by year
import_progress <- reference_review %>%
  group_by(year) %>%
  summarise(
    total_records = n(),
    labelled = sum(!is.na(land_cover)),
    remaining = sum(is.na(land_cover)),
    .groups = "drop"
  )
```

```r
print(import_progress)

# Save an R-ready CSV copy
write_csv(
  reference_review,
  "Tharaka_Reference_Review_Imported.csv"
)

cat("Import completed.\n")
```

```r
# ============================================================
# SECTION 21: FIRST BATCH OF REFERENCE LABELS
# ============================================================

library(dplyr)
library(readr)

# Start with 10 locations for the year 2000
review_batch <- reference_review %>%
  filter(year == 2000) %>%
  arrange(as.integer(point_id)) %>%
  slice_head(n = 10) %>%
  select(
    point_id,
    longitude,
    latitude,
    year,
    land_cover,
    confidence,
    evidence,
    notes
  )

# Save a separate review sheet
write_csv(
  review_batch,
  "Tharaka_Reference_Batch01_Year2000.csv"
)

print(review_batch)
```

```
## # A tibble: 10 x 8
##    point_id longitude latitude  year land_cover confidence evidence notes
##    <chr>        <dbl>    <dbl> <int> <chr>      <chr>      <chr>    <chr>
##  1 11            38.1  -0.0745  2000 <NA>       <NA>       <NA>     <NA>
##  2 34            38.0  -0.113   2000 <NA>       <NA>       <NA>     <NA>
##  3 41            37.9  -0.0611  2000 <NA>       <NA>       <NA>     <NA>
##  4 44            38.2  -0.149   2000 <NA>       <NA>       <NA>     <NA>
##  5 78            38.3  -0.0262  2000 <NA>       <NA>       <NA>     <NA>
##  6 99            37.9  -0.286   2000 <NA>       <NA>       <NA>     <NA>
##  7 107           38.2  -0.0731  2000 <NA>       <NA>       <NA>     <NA>
##  8 109           38.2  -0.112   2000 <NA>       <NA>       <NA>     <NA>
##  9 110           38.1  -0.0273  2000 <NA>       <NA>       <NA>     <NA>
## 10 120           38.2  -0.0842  2000 <NA>       <NA>       <NA>     <NA>
```

```r
cat("Records in review batch:", nrow(review_batch), "\n")
```

```
## Records in review batch: 10
```

```r
# ============================================================
# SECTION 21A: POINT REVIEW LINKS
# ============================================================

review_batch <- review_batch %>%
  mutate(
    map_url = sprintf(
      "https://www.google.com/maps?q=%.6f,%.6f",
      latitude,
      longitude
    )
  )

print(
  review_batch %>%
    select(point_id, year, longitude, latitude, map_url)
)
```

```
## # A tibble: 10 x 5
##    point_id  year longitude latitude map_url
##    <chr>    <int>     <dbl>    <dbl> <chr>
##  1 11        2000      38.1  -0.0745 https://www.google.com/maps?q=-0.074518,38~
##  2 34        2000      38.0  -0.113  https://www.google.com/maps?q=-0.113223,38~
##  3 41        2000      37.9  -0.0611 https://www.google.com/maps?q=-0.061073,37~
##  4 44        2000      38.2  -0.149  https://www.google.com/maps?q=-0.149283,38~
##  5 78        2000      38.3  -0.0262 https://www.google.com/maps?q=-0.026201,38~
##  6 99        2000      37.9  -0.286  https://www.google.com/maps?q=-0.285646,37~
##  7 107       2000      38.2  -0.0731 https://www.google.com/maps?q=-0.073115,38~
##  8 109       2000      38.2  -0.112  https://www.google.com/maps?q=-0.112405,38~
##  9 110       2000      38.1  -0.0273 https://www.google.com/maps?q=-0.027291,38~
## 10 120       2000      38.2  -0.0842 https://www.google.com/maps?q=-0.084180,38~
```

```r
write_csv(
  review_batch,
  "Tharaka_Reference_Batch01_Year2000.csv"
)
```

```r
# ============================================================
# SECTION 22A: GENERATE YEAR-SPECIFIC REVIEW BATCHES
# ============================================================

library(dplyr)
library(readr)

years <- c(2000, 2010, 2020, 2025)

# Output directory
review_dir <- "Tharaka_Reference_Review_Batches"

if (!dir.exists(review_dir)) {
  dir.create(review_dir, recursive = TRUE)
```

```
}

# Ensure point IDs are sorted numerically
reference_review <- reference_review %>%
  mutate(
    point_id = as.character(point_id),
    point_id_numeric = suppressWarnings(as.integer(point_id))
  )

# Export a separate review queue for each year
for (yr in years) {

  year_batch <- reference_review %>%
    filter(year == yr) %>%
    arrange(point_id_numeric) %>%
    select(
      point_id,
      longitude,
      latitude,
      year,
      land_cover,
      confidence,
      evidence,
      notes
    )

  output_file <- file.path(
    review_dir,
    paste0("Tharaka_Review_", yr, ".csv")
  )

  write_csv(year_batch, output_file)

  cat(
    "Year:", yr,
    "| Records:", nrow(year_batch),
    "| Saved:", output_file, "\n"
  )
}
```

```
## Year: 2000 | Records: 50 | Saved: Tharaka_Reference_Review_Batches/Tharaka_Review_2000.csv
## Year: 2010 | Records: 50 | Saved: Tharaka_Reference_Review_Batches/Tharaka_Review_2010.csv
## Year: 2020 | Records: 50 | Saved: Tharaka_Reference_Review_Batches/Tharaka_Review_2020.csv
## Year: 2025 | Records: 50 | Saved: Tharaka_Reference_Review_Batches/Tharaka_Review_2025.csv
```

```
# ========================================================
# SECTION 22B: LABELLING PROGRESS MONITOR
# ========================================================

progress_summary <- reference_review %>%
  group_by(year) %>%
  summarise(
    total_records = n(),
    labelled = sum(
```

```
      !is.na(land_cover) & land_cover != ""
    ),
    remaining = total_records - labelled,
    percent_complete = round(
      100 * labelled / total_records, 1
    ),
    .groups = "drop"
  ) %>%
  arrange(year)

print(progress_summary)
```

```
## # A tibble: 4 x 5
##    year total_records labelled remaining percent_complete
##   <int>         <int>    <int>     <int>            <dbl>
## 1  2000            50        0        50                0
## 2  2010            50        0        50                0
## 3  2020            50        0        50                0
## 4  2025            50        0        50                0
```

```
write_csv(
  progress_summary,
  "Tharaka_Reference_Progress.csv"
)
```

```
# ============================================================
# SECTION 23: EXTRACT PILOT-POINT SPECTRAL INDICATORS
# ============================================================

library(dplyr)
library(readr)
library(reticulate)

# ------------------------------------------------------------
# 23.1 Confirm required objects
# ------------------------------------------------------------

stopifnot(
  exists("ee"),
  exists("selected_composites"),
  exists("pilot_reference"),
  exists("ee_method"),
  exists("ee_get_info")
)

stopifnot(
  nrow(pilot_reference) > 0,
  length(selected_composites) > 0
)

# ------------------------------------------------------------
# 23.2 Create Earth Engine point features
# ------------------------------------------------------------
```

```
pilot_features <- lapply(
  seq_len(nrow(pilot_reference)),
  function(i) {

    lon <- as.numeric(pilot_reference$longitude[i])
    lat <- as.numeric(pilot_reference$latitude[i])
    pid <- as.character(pilot_reference$point_id[i])

    geometry <- ee$Geometry$Point(
      reticulate::tuple(lon, lat)
    )

    ee$Feature(
      geometry,
      list(
        point_id = pid,
        longitude = lon,
        latitude = lat
      )
    )
  }
)

# Build the FeatureCollection using a Python list
pilot_list <- reticulate::r_to_py(
  pilot_features,
  convert = FALSE
)

pilot_fc <- ee$FeatureCollection(pilot_list)

cat(
  "Pilot locations prepared:",
  nrow(pilot_reference), "\n"
)
```

```
## Pilot locations prepared: 50
```

```
# ------------------------------------------------------------
# 23.3 Define output containers
# ------------------------------------------------------------

spectral_results <- list()

# ------------------------------------------------------------
# 23.4 Sample each selected composite
# ------------------------------------------------------------

for (nm in names(selected_composites)) {

  cat("Processing composite:", nm, "\n")

  img <- selected_composites[[nm]]
```

```r
# Calculate NDVI.
# IMPORTANT: pass band names as a flat character vector.
ndvi <- ee_method(
  img,
  "normalizedDifference",
  c("nir", "red")
)

ndvi <- ee_method(
  ndvi,
  "rename",
  "NDVI"
)

# Add NDVI to the six reflectance bands
img_with_ndvi <- ee_method(
  img,
  "addBands",
  ndvi
)

# Sample the image at each point
sampled <- ee_method(
  img_with_ndvi,
  "sampleRegions",
  collection = pilot_fc,
  properties = reticulate::tuple(
    "point_id", "longitude", "latitude"
  ),
  scale = 30,
  geometries = FALSE,
  tileScale = 4
)

# Retrieve sampled records from Earth Engine
sampled_info <- ee_get_info(sampled)

# Convert the feature properties to an R data frame
if (
  is.null(sampled_info$features) ||
  length(sampled_info$features) == 0
) {
  warning(
    "No sampled records returned for composite: ",
    nm
  )
  next
}

feature_rows <- lapply(
  sampled_info$features,
  function(f) {
```

```
      props <- f$properties

      # Convert each property list into a one-row data frame
      as.data.frame(
        lapply(props, function(x) {
          if (is.null(x) || length(x) == 0) {
            NA
          } else {
            unlist(x, use.names = FALSE)[1]
          }
        }),
        stringsAsFactors = FALSE
      )
    }
  )

  result <- bind_rows(feature_rows) %>%
    mutate(
      composite = nm,
      year = as.integer(
        sub("_.*$", "", nm)
      )
    )

  spectral_results[[nm]] <- result

  cat(
    "  Sampled records:",
    nrow(result), "\n"
  )
}
```

```
## Processing composite: 2000_dry_feb
##   Sampled records: 50
## Processing composite: 2000_dry_sep
##   Sampled records: 50
## Processing composite: 2000_wet_dec
##   Sampled records: 44
## Processing composite: 2000_wet_may
##   Sampled records: 44
## Processing composite: 2010_dry_feb
##   Sampled records: 47
## Processing composite: 2010_dry_sep
##   Sampled records: 48
## Processing composite: 2010_wet_dec
##   Sampled records: 50
## Processing composite: 2010_wet_may
##   Sampled records: 46
## Processing composite: 2020_dry_feb
##   Sampled records: 50
## Processing composite: 2020_dry_sep
##   Sampled records: 50
## Processing composite: 2020_wet_dec
##   Sampled records: 50
```

```
## Processing composite: 2020_wet_may
##   Sampled records: 50
## Processing composite: 2025_dry_feb
##   Sampled records: 50
## Processing composite: 2025_dry_sep
##   Sampled records: 50
## Processing composite: 2025_wet_dec
##   Sampled records: 50
## Processing composite: 2025_wet_may
##   Sampled records: 50
```

```
# ------------------------------------------------------------
# 23.5 Combine, inspect and export results
# ------------------------------------------------------------

if (length(spectral_results) == 0) {

  stop(
    "No spectral data were returned. Check the selected ",
    "composites, band names and Earth Engine errors."
  )

}

pilot_spectral_data <- bind_rows(spectral_results)

# Reorder important identifier columns first
pilot_spectral_data <- pilot_spectral_data %>%
  relocate(
    composite,
    year,
    point_id,
    longitude,
    latitude
  )

# Summarise extraction coverage
spectral_qc <- pilot_spectral_data %>%
  group_by(composite, year) %>%
  summarise(
    records_returned = n(),
    unique_points = n_distinct(point_id),
    missing_NDVI = sum(is.na(NDVI)),
    .groups = "drop"
  )

print(spectral_qc)
```

```
## # A tibble: 16 x 5
##    composite     year records_returned unique_points missing_NDVI
##    <chr>        <int>            <int>         <int>        <int>
##  1 2000_dry_feb  2000               50            50            0
##  2 2000_dry_sep  2000               50            50            0
##  3 2000_wet_dec  2000               44            44            0
##  4 2000_wet_may  2000               44            44            0
```

```
##  5 2010_dry_feb  2010               47            47            0
##  6 2010_dry_sep  2010               48            48            0
##  7 2010_wet_dec  2010               50            50            0
##  8 2010_wet_may  2010               46            46            0
##  9 2020_dry_feb  2020               50            50            0
## 10 2020_dry_sep  2020               50            50            0
## 11 2020_wet_dec  2020               50            50            0
## 12 2020_wet_may  2020               50            50            0
## 13 2025_dry_feb  2025               50            50            0
## 14 2025_dry_sep  2025               50            50            0
## 15 2025_wet_dec  2025               50            50            0
## 16 2025_wet_may  2025               50            50            0
```

```
print(head(pilot_spectral_data, 10))
```

```
##       composite year point_id longitude    latitude      NDVI       blue
## 1  2000_dry_feb 2000       11  38.09017 -0.07451829 0.3803372 0.04835250
## 2  2000_dry_feb 2000       34  38.01882 -0.11322331 0.2675060 0.04777500
## 3  2000_dry_feb 2000       41  37.94450 -0.06107348 0.5151975 0.03328250
## 4  2000_dry_feb 2000       44  38.17350 -0.14928311 0.2776928 0.06780875
## 5  2000_dry_feb 2000       78  38.26546 -0.02620120 0.3727477 0.05652000
## 6  2000_dry_feb 2000       99  37.93689 -0.28564570 0.7679752 0.01738750
## 7  2000_dry_feb 2000      107  38.21331 -0.07311465 0.4339065 0.04200000
## 8  2000_dry_feb 2000      109  38.15066 -0.11240454 0.3356782 0.05944875
## 9  2000_dry_feb 2000      110  38.12836 -0.02729136 0.5124759 0.04649625
## 10 2000_dry_feb 2000      120  38.16277 -0.08418035 0.2903175 0.06130500
##         green       nir        red     swir1     swir2
## 1  0.07538500 0.2179725 0.09785250 0.2436025 0.1754575
## 2  0.08360750 0.2226200 0.12865250 0.3118575 0.2250950
## 3  0.06075500 0.2299350 0.07357000 0.1933600 0.1081100
## 4  0.09279250 0.2381300 0.13462000 0.3215375 0.2107125
## 5  0.06870250 0.1688025 0.07713125 0.2036862 0.1282400
## 6  0.03529000 0.2225925 0.02921250 0.1184500 0.0516800
## 7  0.05547500 0.1638525 0.06468750 0.1960000 0.1257375
## 8  0.08454250 0.2335375 0.11615375 0.2963887 0.1834325
## 9  0.06769875 0.2412513 0.07776375 0.2387763 0.1356925
## 10 0.08666000 0.2155112 0.11853250 0.2829000 0.1916137
```

```
# Export data and QC report
write_csv(
  pilot_spectral_data,
  "Tharaka_Pilot_Spectral_Indicators.csv"
)

write_csv(
  spectral_qc,
  "Tharaka_Pilot_Spectral_QC.csv"
)

cat(
  "Spectral extraction complete.\n",
  "Total sampled records: ",
  nrow(pilot_spectral_data), "\n",
  sep = ""
)
```

```
## Spectral extraction complete.
## Total sampled records: 779
```

```
# ======================================================
# SECTION 24: SPECTRAL EXTRACTION COMPLETENESS
# ======================================================

library(dplyr)
library(tidyr)
library(readr)

# Expected combinations: every pilot point in every composite
expected_records <- pilot_reference %>%
  transmute(
    point_id = as.character(point_id),
    longitude,
    latitude
  ) %>%
  crossing(composite = names(selected_composites)) %>%
  mutate(
    year = as.integer(sub("_.*$", "", composite))
  )

# Compare expected combinations with actual extracted records
observed_records <- pilot_spectral_data %>%
  distinct(point_id, composite) %>%
  mutate(point_id = as.character(point_id))

missing_spectral_records <- expected_records %>%
  anti_join(
    observed_records,
    by = c("point_id", "composite")
  ) %>%
  arrange(composite, point_id)

# Completeness summary
completeness_summary <- expected_records %>%
  left_join(
    observed_records %>%
      mutate(returned = TRUE),
    by = c("point_id", "composite")
  ) %>%
  group_by(composite, year) %>%
  summarise(
    expected = n(),
    returned = sum(!is.na(returned)),
    missing = expected - returned,
    completeness_pct = round(100 * returned / expected, 1),
    .groups = "drop"
  )

print(completeness_summary)
```

```
## # A tibble: 16 x 6
##    composite     year expected returned missing completeness_pct
```

```
##    <chr>        <int>    <int>    <int>   <int>           <dbl>
##  1 2000_dry_feb  2000       50       50       0             100
##  2 2000_dry_sep  2000       50       50       0             100
##  3 2000_wet_dec  2000       50       44       6              88
##  4 2000_wet_may  2000       50       44       6              88
##  5 2010_dry_feb  2010       50       47       3              94
##  6 2010_dry_sep  2010       50       48       2              96
##  7 2010_wet_dec  2010       50       50       0             100
##  8 2010_wet_may  2010       50       46       4              92
##  9 2020_dry_feb  2020       50       50       0             100
## 10 2020_dry_sep  2020       50       50       0             100
## 11 2020_wet_dec  2020       50       50       0             100
## 12 2020_wet_may  2020       50       50       0             100
## 13 2025_dry_feb  2025       50       50       0             100
## 14 2025_dry_sep  2025       50       50       0             100
## 15 2025_wet_dec  2025       50       50       0             100
## 16 2025_wet_may  2025       50       50       0             100
```

```
print(missing_spectral_records)
```

```
## # A tibble: 21 x 5
##    point_id longitude latitude composite     year
##    <chr>        <dbl>    <dbl> <chr>        <int>
##  1 175           38.1  -0.105  2000_wet_dec  2000
##  2 299           37.9  -0.184  2000_wet_dec  2000
##  3 316           37.9  -0.348  2000_wet_dec  2000
##  4 343           37.9   0.0101 2000_wet_dec  2000
##  5 499           38.0  -0.295  2000_wet_dec  2000
##  6 99            37.9  -0.286  2000_wet_dec  2000
##  7 172           38.1   0.0207 2000_wet_may  2000
##  8 232           38.3  -0.0472 2000_wet_may  2000
##  9 337           38.2  -0.0797 2000_wet_may  2000
## 10 353           38.2  -0.0400 2000_wet_may  2000
## # i 11 more rows
```

```
write_csv(
  completeness_summary,
  "Tharaka_Spectral_Completeness.csv"
)

write_csv(
  missing_spectral_records,
  "Tharaka_Missing_Spectral_Records.csv"
)
```

```
# ============================================================
# SECTION 25: CALCULATE ADDITIONAL SPECTRAL INDICES
# ============================================================

library(dplyr)
library(readr)

pilot_spectral_indices <- pilot_spectral_data %>%
  mutate(
    NDMI = if_else(
```

```
      is.finite(nir + swir1) & (nir + swir1) != 0,
      (nir - swir1) / (nir + swir1),
      NA_real_
    ),

    NDBI = if_else(
      is.finite(swir1 + nir) & (swir1 + nir) != 0,
      (swir1 - nir) / (swir1 + nir),
      NA_real_
    ),

    SAVI = if_else(
      is.finite(nir + red + 0.5) &
        (nir + red + 0.5) != 0,
      1.5 * (nir - red) / (nir + red + 0.5),
      NA_real_
    )
  )

# Check index ranges and missing values
index_qc <- pilot_spectral_indices %>%
  summarise(
    records = n(),
    missing_NDVI = sum(is.na(NDVI)),
    missing_NDMI = sum(is.na(NDMI)),
    missing_NDBI = sum(is.na(NDBI)),
    missing_SAVI = sum(is.na(SAVI)),
    max_abs_NDMI_NDBI_difference = max(
      abs(NDMI + NDBI),
      na.rm = TRUE
    )
  )

print(index_qc)
```

```
##   records missing_NDVI missing_NDMI missing_NDBI missing_SAVI
## 1     779            0            0            0            0
##   max_abs_NDMI_NDBI_difference
## 1                            0
```

```
# Summarise indicators by year and seasonal window
index_summary <- pilot_spectral_indices %>%
  group_by(composite, year) %>%
  summarise(
    n = n(),
    mean_NDVI = mean(NDVI, na.rm = TRUE),
    mean_NDMI = mean(NDMI, na.rm = TRUE),
    mean_NDBI = mean(NDBI, na.rm = TRUE),
    mean_SAVI = mean(SAVI, na.rm = TRUE),
    .groups = "drop"
  )

print(index_summary)
```

```
## # A tibble: 16 x 7
##    composite     year     n mean_NDVI mean_NDMI mean_NDBI mean_SAVI
##    <chr>        <int> <int>     <dbl>     <dbl>     <dbl>     <dbl>
##  1 2000_dry_feb  2000    50     0.380  -0.0506    0.0506      0.218
##  2 2000_dry_sep  2000    50     0.227  -0.177     0.177       0.133
##  3 2000_wet_dec  2000    44     0.408  -0.0208    0.0208      0.250
##  4 2000_wet_may  2000    44     0.381  -0.0728    0.0728      0.217
##  5 2010_dry_feb  2010    47     0.361  -0.0603    0.0603      0.214
##  6 2010_dry_sep  2010    48     0.222  -0.162     0.162       0.136
##  7 2010_wet_dec  2010    50     0.372  -0.0421    0.0421      0.231
##  8 2010_wet_may  2010    46     0.486   0.0604   -0.0604      0.310
##  9 2020_dry_feb  2020    50     0.543   0.0542   -0.0542      0.328
## 10 2020_dry_sep  2020    50     0.281  -0.145     0.145       0.175
## 11 2020_wet_dec  2020    50     0.524   0.0623   -0.0623      0.336
## 12 2020_wet_may  2020    50     0.521   0.0542   -0.0542      0.323
## 13 2025_dry_feb  2025    50     0.360  -0.0650    0.0650      0.229
## 14 2025_dry_sep  2025    50     0.298  -0.115     0.115       0.193
## 15 2025_wet_dec  2025    50     0.447  -0.00569   0.00569     0.285
## 16 2025_wet_may  2025    50     0.594   0.142    -0.142       0.396
```

```r
# Export indices and summary
write_csv(
  pilot_spectral_indices,
  "Tharaka_Pilot_Spectral_Indices.csv"
)

write_csv(
  index_summary,
  "Tharaka_Pilot_Spectral_Index_Summary.csv"
)
```

```r
# ============================================================
# SECTION 26: VISUALISE SPECTRAL INDEX PATTERNS
# ============================================================

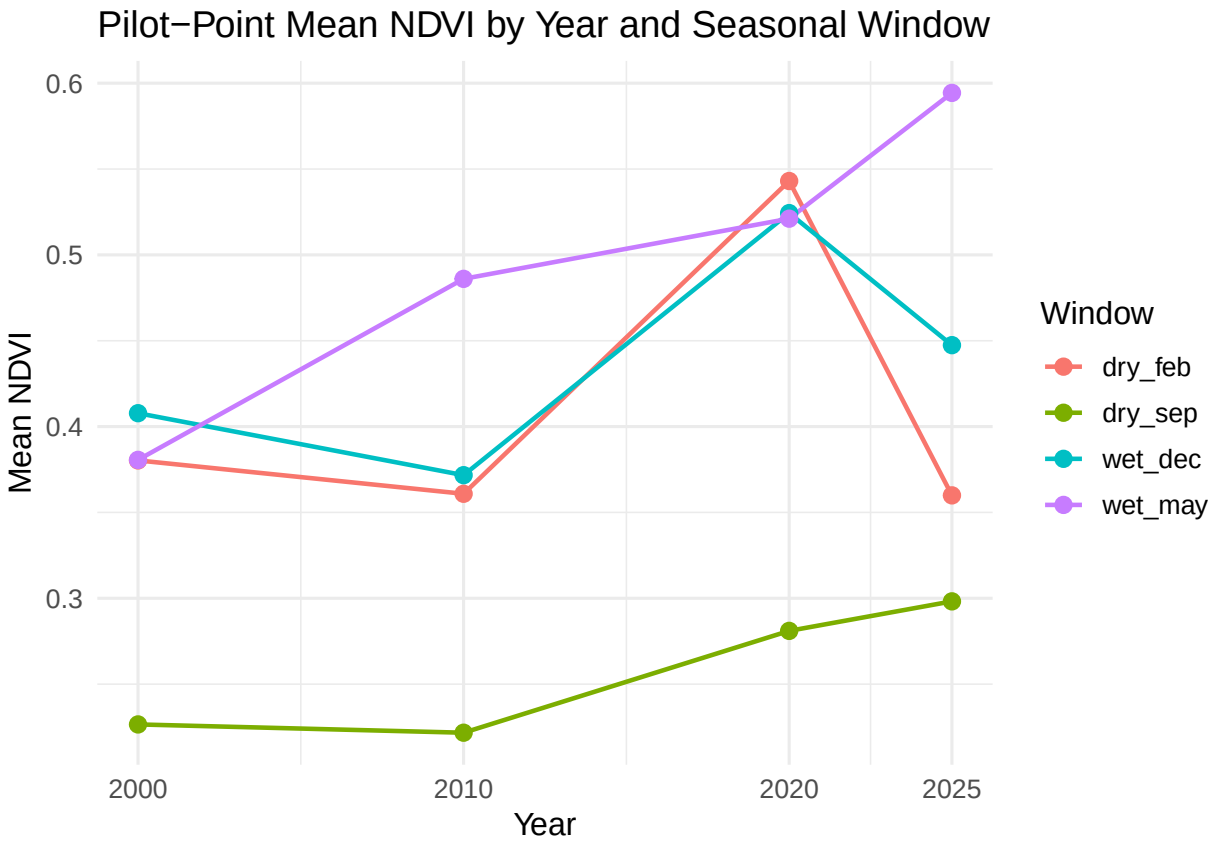
library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)

# Create season labels from composite names
index_plot_data <- pilot_spectral_indices %>%
  mutate(
    season_window = sub("^\\d{4}_", "", composite),
    season = case_when(
      grepl("^dry_", season_window) ~ "Dry",
      grepl("^wet_", season_window) ~ "Wet",
      TRUE ~ "Unknown"
    ),
    month_window = sub("^(dry|wet)_", "", season_window)
  )

# Plot 1: Mean NDVI by year and seasonal window
p_ndvi <- index_plot_data %>%
  group_by(year, season_window) %>%
```

```
  summarise(
    mean_NDVI = mean(NDVI, na.rm = TRUE),
    sd_NDVI = sd(NDVI, na.rm = TRUE),
    n = n(),
    se_NDVI = sd_NDVI / sqrt(n),
    .groups = "drop"
  ) %>%
  ggplot(
    aes(
      x = year,
      y = mean_NDVI,
      colour = season_window,
      group = season_window
    )
  ) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2.5) +
  scale_x_continuous(breaks = c(2000, 2010, 2020, 2025)) +
  labs(
    title = "Pilot-Point Mean NDVI by Year and Seasonal Window",
    x = "Year",
    y = "Mean NDVI",
    colour = "Window"
  ) +
  theme_minimal(base_size = 12)

print(p_ndvi)
```



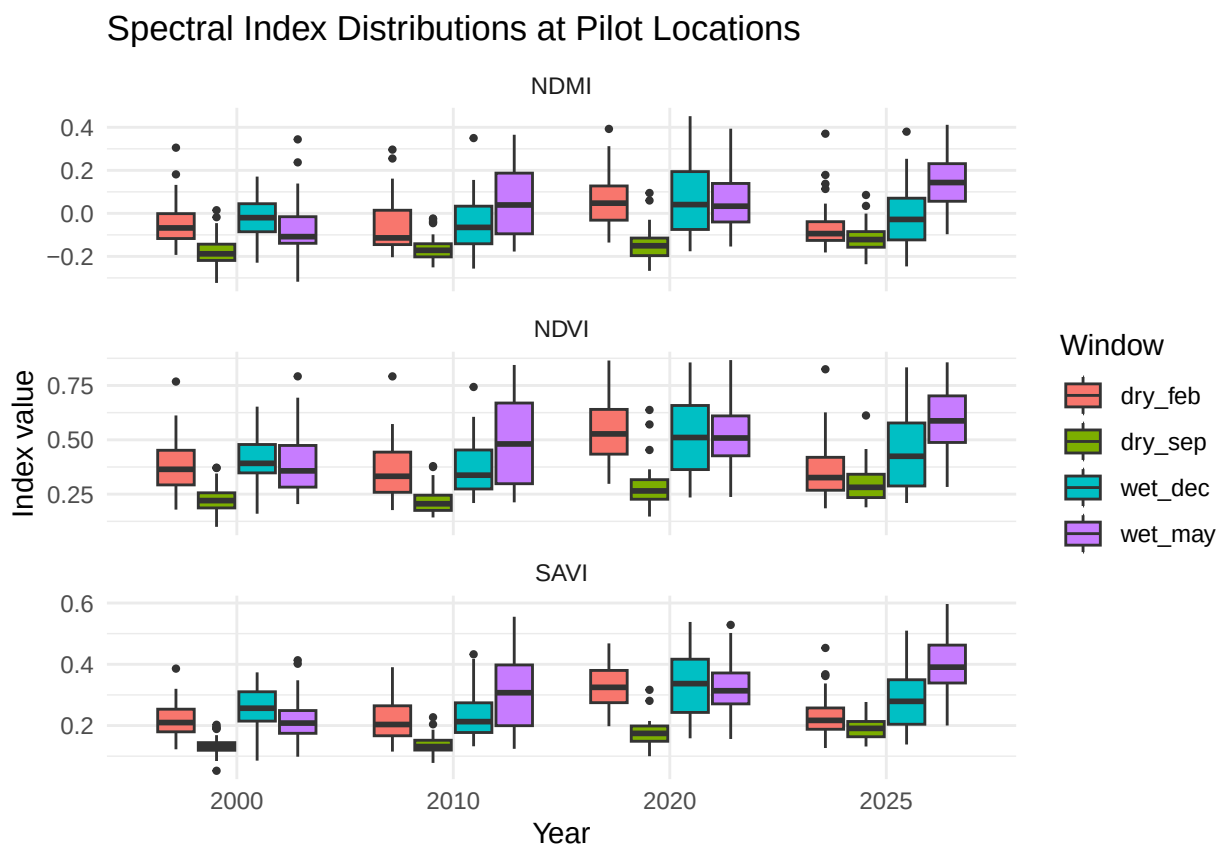

```
ggsave(
  "Tharaka_Pilot_NDVI_Trends.png",
  p_ndvi,
  width = 9,
  height = 5.5,
  dpi = 300
)

# Plot 2: Compare all index distributions
index_long <- index_plot_data %>%
  select(
    year, season_window, NDVI, NDMI, SAVI
  ) %>%
  pivot_longer(
    cols = c(NDVI, NDMI, SAVI),
    names_to = "index",
    values_to = "value"
  )

p_indices <- ggplot(
  index_long,
  aes(
    x = factor(year),
    y = value,
    group = interaction(year, season_window)
  )
```

```
) +
  geom_boxplot(aes(fill = season_window), outlier.size = 0.8) +
  facet_wrap(~ index, scales = "free_y", ncol = 1) +
  labs(
    title = "Spectral Index Distributions at Pilot Locations",
    x = "Year",
    y = "Index value",
    fill = "Window"
  ) +
  theme_minimal(base_size = 11)

print(p_indices)
```




```
ggsave(
  "Tharaka_Pilot_Index_Distributions.png",
  p_indices,
  width = 10,
  height = 10,
  dpi = 300
)

cat("Section 26 plots saved successfully.\n")
```

```
## Section 26 plots saved successfully.
```

```r
# ========================================================
# SECTION 27: SUMMARISE PILOT SPECTRAL PATTERNS
# ========================================================

library(dplyr)
library(tidyr)
library(readr)
library(stringr)

# 1. Extract seasonal identifiers from composite
pilot_index_summary <- pilot_spectral_indices %>%
  mutate(
    year = as.integer(year),
    window = str_remove(composite, "^\\d{4}_"),
    season = case_when(
      str_starts(window, "dry") ~ "Dry",
      str_starts(window, "wet") ~ "Wet",
      TRUE ~ NA_character_
    )
  ) %>%
  group_by(year, composite, window, season) %>%
  summarise(
    n = n(),
    mean_NDVI = mean(NDVI, na.rm = TRUE),
    sd_NDVI = sd(NDVI, na.rm = TRUE),
    mean_NDMI = mean(NDMI, na.rm = TRUE),
    sd_NDMI = sd(NDMI, na.rm = TRUE),
    mean_SAVI = mean(SAVI, na.rm = TRUE),
    sd_SAVI = sd(SAVI, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(year, season, window)

# 2. Display results
print(pilot_index_summary, n = Inf)
```

```
## # A tibble: 16 x 11
##     year composite    window  season     n mean_NDVI sd_NDVI mean_NDMI sd_NDMI
##    <int> <chr>        <chr>   <chr>  <int>     <dbl>   <dbl>     <dbl>   <dbl>
##  1  2000 2000_dry_feb dry_feb Dry       50     0.380  0.118   -0.0506   0.0990
##  2  2000 2000_dry_sep dry_sep Dry       50     0.227  0.0582  -0.177    0.0688
##  3  2000 2000_wet_dec wet_dec Wet       44     0.408  0.126   -0.0208   0.101
##  4  2000 2000_wet_may wet_may Wet       44     0.381  0.134   -0.0728   0.123
##  5  2010 2010_dry_feb dry_feb Dry       47     0.361  0.128   -0.0603   0.118
##  6  2010 2010_dry_sep dry_sep Dry       48     0.222  0.0595  -0.162    0.0572
##  7  2010 2010_wet_dec wet_dec Wet       50     0.372  0.122   -0.0421   0.121
##  8  2010 2010_wet_may wet_may Wet       46     0.486  0.186    0.0604   0.159
##  9  2020 2020_dry_feb dry_feb Dry       50     0.543  0.128    0.0542   0.117
## 10  2020 2020_dry_sep dry_sep Dry       50     0.281  0.0906  -0.145    0.0696
## 11  2020 2020_wet_dec wet_dec Wet       50     0.524  0.163    0.0623   0.157
## 12  2020 2020_wet_may wet_may Wet       50     0.521  0.142    0.0542   0.128
## 13  2025 2025_dry_feb dry_feb Dry       50     0.360  0.128   -0.0650   0.102
## 14  2025 2025_dry_sep dry_sep Dry       50     0.298  0.0843  -0.115    0.0628
## 15  2025 2025_wet_dec wet_dec Wet       50     0.447  0.164   -0.00569  0.142
```

```
## 16  2025 2025_wet_may wet_may Wet       50     0.594  0.149    0.142    0.129
## # i 2 more variables: mean_SAVI <dbl>, sd_SAVI <dbl>
```

```
# 3. Export summary
write_csv(
  pilot_index_summary,
  "Tharaka_Pilot_Spectral_Patterns_Summary.csv"
)

# 4. Check number of seasonal windows per year
combination_check <- pilot_index_summary %>%
  count(year, name = "number_of_windows")

print(combination_check)
```

```
## # A tibble: 4 x 2
##    year number_of_windows
##   <int>             <int>
## 1  2000                 4
## 2  2010                 4
## 3  2020                 4
## 4  2025                 4
```

```
# 5. Quality-control flags
qc_flags <- pilot_index_summary %>%
  mutate(
    low_sample_size = n < 40,
    missing_index_summary =
      is.na(mean_NDVI) |
      is.na(mean_NDMI) |
      is.na(mean_SAVI)
  )

print(qc_flags, n = Inf)
```

```
## # A tibble: 16 x 13
##     year composite    window  season     n mean_NDVI sd_NDVI mean_NDMI sd_NDMI
##    <int> <chr>        <chr>   <chr>  <int>     <dbl>   <dbl>     <dbl>   <dbl>
##  1  2000 2000_dry_feb dry_feb Dry       50     0.380  0.118   -0.0506   0.0990
##  2  2000 2000_dry_sep dry_sep Dry       50     0.227  0.0582  -0.177    0.0688
##  3  2000 2000_wet_dec wet_dec Wet       44     0.408  0.126   -0.0208   0.101
##  4  2000 2000_wet_may wet_may Wet       44     0.381  0.134   -0.0728   0.123
##  5  2010 2010_dry_feb dry_feb Dry       47     0.361  0.128   -0.0603   0.118
##  6  2010 2010_dry_sep dry_sep Dry       48     0.222  0.0595  -0.162    0.0572
##  7  2010 2010_wet_dec wet_dec Wet       50     0.372  0.122   -0.0421   0.121
##  8  2010 2010_wet_may wet_may Wet       46     0.486  0.186    0.0604   0.159
##  9  2020 2020_dry_feb dry_feb Dry       50     0.543  0.128    0.0542   0.117
## 10  2020 2020_dry_sep dry_sep Dry       50     0.281  0.0906  -0.145    0.0696
## 11  2020 2020_wet_dec wet_dec Wet       50     0.524  0.163    0.0623   0.157
## 12  2020 2020_wet_may wet_may Wet       50     0.521  0.142    0.0542   0.128
## 13  2025 2025_dry_feb dry_feb Dry       50     0.360  0.128   -0.0650   0.102
## 14  2025 2025_dry_sep dry_sep Dry       50     0.298  0.0843  -0.115    0.0628
## 15  2025 2025_wet_dec wet_dec Wet       50     0.447  0.164   -0.00569  0.142
## 16  2025 2025_wet_may wet_may Wet       50     0.594  0.149    0.142    0.129
## # i 4 more variables: mean_SAVI <dbl>, sd_SAVI <dbl>, low_sample_size <lgl>,
## #   missing_index_summary <lgl>
```

```r
write_csv(
  qc_flags,
  "Tharaka_Pilot_Spectral_Patterns_QC.csv"
)

cat("Section 27 completed successfully.\n")
```

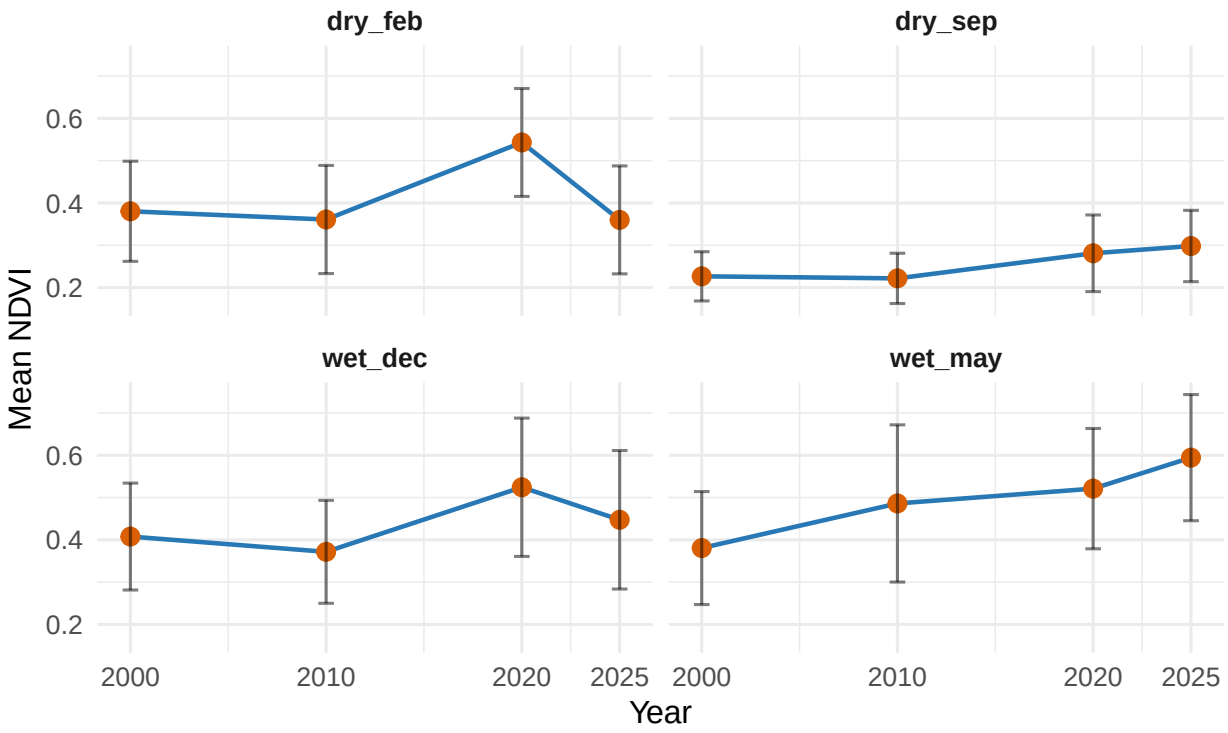
```
## Section 27 completed successfully.
```

```r
# ============================================================
# SECTION 28: NDVI TRENDS BY SEASONAL WINDOW
# ============================================================

library(ggplot2)
library(dplyr)

p_ndvi_seasonal <- ggplot(
  pilot_index_summary,
  aes(
    x = year,
    y = mean_NDVI,
    group = window
  )
) +
  geom_line(linewidth = 0.8, colour = "#2878B5") +
  geom_point(size = 2.8, colour = "#D95F02") +
  geom_errorbar(
    aes(
      ymin = mean_NDVI - sd_NDVI,
      ymax = mean_NDVI + sd_NDVI
    ),
    width = 0.8,
    alpha = 0.5
  ) +
  facet_wrap(~ window, ncol = 2) +
  scale_x_continuous(
    breaks = c(2000, 2010, 2020, 2025)
  ) +
  labs(
    title = "Seasonal NDVI Patterns in Tharaka Sub-county",
    subtitle = "Mean NDVI at pilot locations; error bars show ±1 SD",
    x = "Year",
    y = "Mean NDVI"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold"),
    strip.text = element_text(face = "bold")
  )

print(p_ndvi_seasonal)
```




```
ggsave(
  "Tharaka_Seasonal_NDVI_Patterns.png",
  plot = p_ndvi_seasonal,
  width = 10,
  height = 7,
  dpi = 300
)

cat("Section 28 completed successfully.\n")
```

```
## Section 28 completed successfully.
```

```
# ============================================================
# SECTION 29: REFERENCE LABEL READINESS CHECK
# ============================================================

library(dplyr)
library(readr)

# Confirm reference-review object exists
if (exists("reference_review")) {

  cat("Reference review records:", nrow(reference_review), "\n")

  print(
    reference_review %>%
      summarise(
```

```
        total_records = n(),
        labelled_classes = sum(
          !is.na(land_cover) & land_cover != ""
        ),
        labelled_confidence = sum(
          !is.na(confidence) & confidence != ""
        ),
        labelled_evidence = sum(
          !is.na(evidence) & evidence != ""
        )
      )
  )

  print(
    reference_review %>%
      count(land_cover, .drop = FALSE)
  )

} else {
  cat(
    "reference_review is not currently loaded. ",
    "Please import the completed reference workbook first.\n"
  )
}
```

```
## Reference review records: 200
## # A tibble: 1 x 4
##   total_records labelled_classes labelled_confidence labelled_evidence
##           <int>            <int>               <int>             <int>
## 1           200                0                   0                 0
## # A tibble: 1 x 2
##   land_cover     n
##   <chr>      <int>
## 1 <NA>         200
```

```
# ============================================================
# SECTION 30: PREPARE PILOT REFERENCE-LABELING SHEET
# ============================================================

library(dplyr)
library(readr)

pilot_label_sheet <- reference_review %>%
  select(
    point_id,
    year,
    longitude,
    latitude,
    land_cover,
    confidence,
    evidence,
    notes
  ) %>%
  arrange(point_id, year)
```

```r
# Export a CSV for manual labeling
write_csv(
  pilot_label_sheet,
  "Tharaka_Pilot_Reference_Labelling.csv"
)

# Show labeling requirements
cat("Records requiring review:", nrow(pilot_label_sheet), "\n")
```

```
## Records requiring review: 200
```

```r
cat("Unique locations:", n_distinct(pilot_label_sheet$point_id), "\n")
```

```
## Unique locations: 50
```

```r
cat("Years:", paste(sort(unique(pilot_label_sheet$year)),
                    collapse = ", "), "\n")
```

```
## Years: 2000, 2010, 2020, 2025
```

```r
cat("\nPermitted land-cover labels:\n")
```

```
##
## Permitted land-cover labels:
```

```r
print(c(
  "forest",
  "grassland",
  "bushland",
  "farmland",
  "settlement",
  "other",
  "uncertain"
))
```

```
## [1] "forest"     "grassland"  "bushland"   "farmland"   "settlement"
## [6] "other"      "uncertain"
```

```r
cat("\nPermitted confidence ratings: high, medium, low\n")
```

```
##
## Permitted confidence ratings: high, medium, low
```

```r
cat("Evidence: field, historical_imagery, existing_map, other\n")
```

```
## Evidence: field, historical_imagery, existing_map, other
```

```r
# ============================================================
# SECTION 31: OPEN HISTORICAL IMAGERY REVIEW MAP
# ============================================================

review_map <- file.path(
  getwd(),
  "Tharaka_Historical_Imagery_Review.html"
)

if (file.exists(review_map)) {
  browseURL(review_map)
```

```
  cat("Review map opened in your browser.\n")
  cat("Map location:", normalizePath(review_map), "\n")
} else {
  cat("Review map was not found in the current working directory.\n")
  cat("Current directory:", getwd(), "\n")
  cat("Check where Tharaka_Historical_Imagery_Review.html was saved.\n")
}
```

```
## Review map opened in your browser.
## Map location: C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Historical_Imagery_Review.html
```

```
# Check which plots exist in the working directory
list.files(
  getwd(),
  pattern = "\\.png$",
  full.names = TRUE
)
```

```
## [1] "C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Mean_NDVI_Seasonal_Comparison.png"
## [2] "C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Pilot_Index_Distributions.png"
## [3] "C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Pilot_NDVI_Trends.png"
## [4] "C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Seasonal_NDVI_Patterns.png"
## [5] "C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Spectral_Reflectance_Comparison.png"
```

```
# Create a results folder
dir.create(
  "Tharaka_Land_Cover_Results",
  showWarnings = FALSE
)

# Copy the review map if it exists in the current directory
if (file.exists("Tharaka_Historical_Imagery_Review.html")) {
  file.copy(
    "Tharaka_Historical_Imagery_Review.html",
    "Tharaka_Land_Cover_Results/Tharaka_Historical_Imagery_Review.html",
    overwrite = TRUE
  )
}
```

```
## [1] TRUE
```

```
# ============================================================
# SECTION 32: GENERATE ALL 16 LANDSAT SCENARIO PLOTS
# ============================================================

library(reticulate)
library(ggplot2)
library(png)

# ------------------------------------------------------------
# 1. Earth Engine session
# ------------------------------------------------------------
ee <- reticulate::import("ee", convert = FALSE)

# Reuse the working Earth Engine authentication
ee$Initialize(project = "thinking-leaf-305215")
```

```
## None
```

```r
# ------------------------------------------------------------
# 2. Output directory and scenario labels
# ------------------------------------------------------------
plot_dir <- "Tharaka_Landsat_Scenario_Plots"

dir.create(
  plot_dir,
  showWarnings = FALSE,
  recursive = TRUE
)

scenario_labels <- c(
  dry_feb = "February Dry Season",
  dry_sep = "September Dry Season",
  wet_may = "May Wet Season",
  wet_dec = "December Wet Season"
)

# ------------------------------------------------------------
# 3. Confirm the 16 composites are available
# ------------------------------------------------------------
scenario_names <- names(selected_composites)

if (length(scenario_names) != 16) {
  stop(
    "Expected 16 composites, but found ",
    length(scenario_names),
    ". Check selected_composites."
  )
}

# ------------------------------------------------------------
# 4. Plot each scenario
# ------------------------------------------------------------
plot_results <- data.frame(
  composite = scenario_names,
  status = "Pending",
  file = NA_character_,
  stringsAsFactors = FALSE
)

for (i in seq_along(scenario_names)) {

  scenario <- scenario_names[i]

  message(
    "\nProcessing scenario ",
    i, " of ", length(scenario_names),
    ": ", scenario
  )

  tryCatch({
```

```r
# Retrieve existing Earth Engine image
img <- selected_composites[[scenario]]

# Create true-colour visualisation in Earth Engine
visualized_img <- ee$Image$visualize(
  img,
  bands = c("red", "green", "blue"),
  min = 0.02,
  max = 0.30,
  gamma = 1.3
)

# Request thumbnail URL
thumb <- visualized_img$getThumbURL(
  reticulate::dict(
    region = roi,
    dimensions = 1200,
    format = "png"
  )
)

thumb_url <- reticulate::py_to_r(thumb)

# Download thumbnail to a temporary PNG
temp_png <- tempfile(fileext = ".png")

download.file(
  url = thumb_url,
  destfile = temp_png,
  mode = "wb",
  quiet = TRUE
)

# Read the image
raster_img <- png::readPNG(temp_png)

# Extract year and seasonal window
year <- sub("_.*$", "", scenario)
window <- sub("^\\d{4}_", "", scenario)

if (!window %in% names(scenario_labels)) {
  stop("Unrecognised seasonal window: ", window)
}

# Create title
title_text <- paste(
  "Tharaka Sub-county -",
  scenario_labels[[window]],
  year
)

# Construct framed figure
p <- ggplot() +
```

```
    annotation_raster(
      raster_img,
      xmin = 0, xmax = 1,
      ymin = 0, ymax = 1
    ) +
    coord_fixed(
      xlim = c(0, 1),
      ylim = c(0, 1),
      expand = FALSE
    ) +
    labs(
      title = title_text,
      subtitle = paste(
        "Landsat Collection 2 Level-2",
        "| True-colour RGB composite"
      ),
      caption = paste(
        "Display stretch: RGB bands,",
        "min = 0.02, max = 0.30, gamma = 1.3"
      )
    ) +
    theme_void(base_size = 12) +
    theme(
      plot.title = element_text(
        face = "bold",
        size = 15,
        hjust = 0.5,
        margin = margin(b = 8)
      ),
      plot.subtitle = element_text(
        size = 10,
        hjust = 0.5
      ),
      plot.caption = element_text(
        size = 8,
        hjust = 0.5
      ),
      panel.border = element_rect(
        colour = "black",
        fill = NA,
        linewidth = 0.9
      ),
      plot.margin = margin(12, 12, 12, 12)
    )

  # Save plot
  output_file <- file.path(
    plot_dir,
    paste0("Tharaka_", scenario, ".png")
  )

  ggsave(
    filename = output_file,
```

```
      plot = p,
      width = 9,
      height = 7,
      dpi = 300,
      bg = "white"
    )

    # Record success
    plot_results$status[i] <- "Success"
    plot_results$file[i] <- output_file

    unlink(temp_png)

    message("Saved: ", output_file)

  }, error = function(e) {

    plot_results$status[i] <<- paste(
      "Failed:", conditionMessage(e)
    )

    message(
      "Failed for ", scenario, ": ",
      conditionMessage(e)
    )
  })
}

# ------------------------------------------------------------
# 5. Save processing log
# ------------------------------------------------------------
write.csv(
  plot_results,
  file.path(plot_dir, "Scenario_Plot_Processing_Log.csv"),
  row.names = FALSE
)

cat("\n==========================================\n")
```

```
##
## ==========================================
```

```
cat("SECTION 32 FINISHED\n")
```

```
## SECTION 32 FINISHED
```

```
cat("Successful plots:",
    sum(plot_results$status == "Success"), "\n")
```

```
## Successful plots: 16
```

```
cat("Failed plots:",
    sum(plot_results$status != "Success"), "\n")
```

```
## Failed plots: 0
```

```r
cat("Output folder:",
    normalizePath(plot_dir), "\n")
```

```
## Output folder: C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Landsat_Scenario_Plots
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
print(plot_results)
```

```
##       composite  status                                                    file
## 1  2000_dry_feb Success Tharaka_Landsat_Scenario_Plots/Tharaka_2000_dry_feb.png
## 2  2000_dry_sep Success Tharaka_Landsat_Scenario_Plots/Tharaka_2000_dry_sep.png
## 3  2000_wet_dec Success Tharaka_Landsat_Scenario_Plots/Tharaka_2000_wet_dec.png
## 4  2000_wet_may Success Tharaka_Landsat_Scenario_Plots/Tharaka_2000_wet_may.png
## 5  2010_dry_feb Success Tharaka_Landsat_Scenario_Plots/Tharaka_2010_dry_feb.png
## 6  2010_dry_sep Success Tharaka_Landsat_Scenario_Plots/Tharaka_2010_dry_sep.png
## 7  2010_wet_dec Success Tharaka_Landsat_Scenario_Plots/Tharaka_2010_wet_dec.png
## 8  2010_wet_may Success Tharaka_Landsat_Scenario_Plots/Tharaka_2010_wet_may.png
## 9  2020_dry_feb Success Tharaka_Landsat_Scenario_Plots/Tharaka_2020_dry_feb.png
## 10 2020_dry_sep Success Tharaka_Landsat_Scenario_Plots/Tharaka_2020_dry_sep.png
## 11 2020_wet_dec Success Tharaka_Landsat_Scenario_Plots/Tharaka_2020_wet_dec.png
## 12 2020_wet_may Success Tharaka_Landsat_Scenario_Plots/Tharaka_2020_wet_may.png
## 13 2025_dry_feb Success Tharaka_Landsat_Scenario_Plots/Tharaka_2025_dry_feb.png
## 14 2025_dry_sep Success Tharaka_Landsat_Scenario_Plots/Tharaka_2025_dry_sep.png
## 15 2025_wet_dec Success Tharaka_Landsat_Scenario_Plots/Tharaka_2025_wet_dec.png
## 16 2025_wet_may Success Tharaka_Landsat_Scenario_Plots/Tharaka_2025_wet_may.png
```

```r
# ============================================================
# SECTION 32: PUBLICATION-STYLE LANDSAT SCENARIO MAPS
# ============================================================

library(reticulate)
library(ggplot2)
library(png)
library(grid)

# ------------------------------------------------------------
# 1. Earth Engine session and output directory
# ------------------------------------------------------------

ee <- reticulate::import("ee", convert = FALSE)
ee$Initialize(project = "thinking-leaf-305215")
```

```
## None
```

```r
plot_dir <- "Tharaka_Landsat_Enhanced_Maps"

dir.create(
  plot_dir,
  showWarnings = FALSE,
  recursive = TRUE
)

# ------------------------------------------------------------
# 2. Seasonal labels
```

```r
# ------------------------------------------------------------

scenario_labels <- c(
  dry_feb = "February Dry Season (10-20 February)",
  dry_sep = "September Dry Season (15-25 September)",
  wet_may = "May Wet Season (5-30 May)",
  wet_dec = "December Wet Season (5-30 December)"
)

scenario_names <- names(selected_composites)

if (length(scenario_names) != 16) {
  stop("Expected 16 scenarios; found ",
       length(scenario_names), ".")
}

# ------------------------------------------------------------
# 3. Processing log
# ------------------------------------------------------------

plot_results <- data.frame(
  composite = scenario_names,
  status = "Pending",
  file = NA_character_,
  stringsAsFactors = FALSE
)

# ------------------------------------------------------------
# 4. Generate enhanced maps
# ------------------------------------------------------------

for (i in seq_along(scenario_names)) {

  scenario <- scenario_names[i]

  message(
    "\nProcessing ", i, " of ",
    length(scenario_names), ": ", scenario
  )

  temp_png <- tempfile(fileext = ".png")

  tryCatch({

    img <- selected_composites[[scenario]]

    year <- sub("_.*$", "", scenario)
    window <- sub("^\\d{4}_", "", scenario)

    if (!window %in% names(scenario_labels)) {
      stop("Unknown seasonal window: ", window)
    }
```

```r
# ------------------------------------------------------
# A. True-colour RGB visualisation
# ------------------------------------------------------

# Estimate robust per-band display limits from the 2nd and 98th
# reflectance percentiles. This improves contrast across bright and
# dark scenes without changing the underlying reflectance values.
rgb_stats_image <- ee_method(
  img, "select", c("red", "green", "blue")
)

rgb_stats <- ee_get_info(ee_method(
  rgb_stats_image, "reduceRegion",
  reducer = ee$Reducer$percentile(c(2, 98)),
  geometry = roi,
  scale = 90,
  maxPixels = 1e9,
  bestEffort = TRUE,
  tileScale = 4
))

get_rgb_percentile <- function(band, percentile, fallback) {
  key <- paste0(band, "_p", percentile)
  value <- rgb_stats[[key]]
  if (is.null(value) || length(value) != 1L ||
      !is.finite(as.numeric(value))) {
    return(fallback)
  }
  as.numeric(value)
}

rgb_min <- c(
  get_rgb_percentile("red", 2, 0.02),
  get_rgb_percentile("green", 2, 0.02),
  get_rgb_percentile("blue", 2, 0.02)
)
rgb_max <- c(
  get_rgb_percentile("red", 98, 0.30),
  get_rgb_percentile("green", 98, 0.30),
  get_rgb_percentile("blue", 98, 0.30)
)

# Guard against flat or degenerate percentile ranges.
rgb_max <- pmax(rgb_max, rgb_min + 0.03)

rgb_img <- ee$Image$visualize(
  img,
  bands = c("red", "green", "blue"),
  min = rgb_min,
  max = rgb_max,
  gamma = 1.15
)
```

```
   # ------------------------------------------------------
# B. Add ONLY the ROI boundary, keeping the image visible
# ------------------------------------------------------

roi_fc <- ee$FeatureCollection(
  ee$Feature(roi)
)

# Paint the boundary into a blank image
boundary_img <- ee$Image$constant(0)$byte()$paint(
  roi_fc, 1, 2
)

# Mask out all pixels except the painted boundary
boundary_masked <- boundary_img$updateMask(
  boundary_img
)

# Visualise the boundary in white
boundary_vis <- boundary_masked$visualize(
  palette = c("FFFFFF"),
  min = 0,
  max = 1,
  opacity = 1
)

# Overlay the boundary without covering the RGB imagery
map_img <- rgb_img$blend(boundary_vis)

    # ------------------------------------------------------
    # C. Download thumbnail
    # ------------------------------------------------------

    thumb <- map_img$getThumbURL(
      reticulate::dict(
        region = roi,
        dimensions = 1200,
        format = "png"
      )
    )

    thumb_url <- reticulate::py_to_r(thumb)

    download.file(
      url = thumb_url,
      destfile = temp_png,
      mode = "wb",
      quiet = TRUE
    )

    raster_img <- png::readPNG(temp_png)

    # ------------------------------------------------------
```

```
# D. Create framed map
# ------------------------------------------------------

p <- ggplot() +

  annotation_raster(
    raster_img,
    xmin = 0, xmax = 1,
    ymin = 0, ymax = 1
  ) +

  # North arrow
  annotate(
    "segment",
    x = 0.93, xend = 0.93,
    y = 0.77, yend = 0.89,
    linewidth = 0.8,
    colour = "black",
    arrow = grid::arrow(
      length = grid::unit(0.16, "cm"),
      type = "closed"
    )
  ) +

  annotate(
    "text",
    x = 0.93, y = 0.93,
    label = "N",
    fontface = "bold",
    size = 4
  ) +

  # RGB key
  annotate(
    "label",
    x = 0.03, y = 0.97,
    label = "RGB KEY\nR: Red\nG: Green\nB: Blue",
    hjust = 0, vjust = 1,
    size = 3,
    label.size = 0.3,
    fill = "white",
    alpha = 0.92
  ) +

  coord_fixed(
    xlim = c(0, 1),
    ylim = c(0, 1),
    expand = FALSE
  ) +

  labs(
    title = paste(
      "THARAKA SUB-COUNTY -", year
```

```
    ),
    subtitle = paste(
      scenario_labels[[window]],
      "| Landsat Collection 2 Level-2"
    ),
    caption = paste(
      "True-colour RGB composite | Nominal pixel size: 30 m",
      "\nWhite line: study-area boundary | adaptive 2nd-98th percentile RGB stretch"
    )
  ) +

  theme_void(base_size = 12) +

  theme(
    plot.title = element_text(
      face = "bold",
      size = 16,
      hjust = 0.5,
      margin = margin(b = 8)
    ),
    plot.subtitle = element_text(
      size = 10,
      hjust = 0.5
    ),
    plot.caption = element_text(
      size = 8,
      hjust = 0.5
    ),
    panel.border = element_rect(
      colour = "black",
      fill = NA,
      linewidth = 1
    ),
    plot.background = element_rect(
      fill = "white",
      colour = NA
    ),
    plot.margin = margin(12, 12, 12, 12)
  )

# ----------------------------------------------------
# E. Save high-resolution PNG
# ----------------------------------------------------

output_file <- file.path(
  plot_dir,
  paste0("Tharaka_", scenario, ".png")
)

ggsave(
  filename = output_file,
  plot = p,
  width = 9,
```

```
      height = 7,
      units = "in",
      dpi = 300,
      bg = "white"
    )

    plot_results$status[i] <- "Success"
    plot_results$file[i] <- output_file

    message("Saved: ", output_file)

  }, error = function(e) {

    plot_results$status[i] <<- paste(
      "Failed:", conditionMessage(e)
    )

    message(
      "Error for ", scenario, ": ",
      conditionMessage(e)
    )

  }, finally = {

    if (file.exists(temp_png)) {
      unlink(temp_png)
    }

  })
}
```

```
## Warning in annotate("label", x = 0.03, y = 0.97, label = "RGB KEY\nR: Red\nG: Green\nB: Blue", : Ign
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
## Ignoring unknown parameters: `label.size`
```

```
## Warning in download.file(url = thumb_url, destfile = temp_png, mode = "wb", :
## downloaded length 1162217 != reported length 1804304
```

```
## Warning in download.file(url = thumb_url, destfile = temp_png, mode = "wb", :
## URL
## 'https://earthengine.googleapis.com/v1/projects/thinking-leaf-305215/thumbnails/c65cfa1f3c219fe00e74
## status was 'Failure when receiving data from the peer'
```

```r
# ------------------------------------------------------------
# 5. Save processing log and report
# ------------------------------------------------------------

write.csv(
  plot_results,
  file.path(plot_dir, "Scenario_Plot_Processing_Log.csv"),
  row.names = FALSE
)

cat("\n====================================\n")
```

```
##
## ====================================
```

```r
cat("ENHANCED MAP EXPORT COMPLETE\n")
```

```
## ENHANCED MAP EXPORT COMPLETE
```

```r
cat("Successful:", sum(plot_results$status == "Success"), "\n")
```

```
## Successful: 15
```

```r
cat("Failed:", sum(plot_results$status != "Success"), "\n")
```

```
## Failed: 1
```

```r
cat("Output folder:", normalizePath(plot_dir), "\n")
```

```
## Output folder: C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Landsat_Enhanced_Maps
```

```r
cat("====================================\n")
```

```
## ====================================
```

```r
print(plot_results)
```

```
##       composite
## 1  2000_dry_feb
## 2  2000_dry_sep
## 3  2000_wet_dec
## 4  2000_wet_may
## 5  2010_dry_feb
## 6  2010_dry_sep
## 7  2010_wet_dec
## 8  2010_wet_may
## 9  2020_dry_feb
## 10 2020_dry_sep
## 11 2020_wet_dec
## 12 2020_wet_may
## 13 2025_dry_feb
## 14 2025_dry_sep
## 15 2025_wet_dec
## 16 2025_wet_may
##
## 1
## 2
## 3
```

```
## 4
## 5
## 6
## 7
## 8
## 9
## 10
## 11
## 12
## 13
## 14
## 15
## 16 Failed: download from 'https://earthengine.googleapis.com/v1/projects/thinking-leaf-305215/thumbna
##                                                    file
## 1  Tharaka_Landsat_Enhanced_Maps/Tharaka_2000_dry_feb.png
## 2  Tharaka_Landsat_Enhanced_Maps/Tharaka_2000_dry_sep.png
## 3  Tharaka_Landsat_Enhanced_Maps/Tharaka_2000_wet_dec.png
## 4  Tharaka_Landsat_Enhanced_Maps/Tharaka_2000_wet_may.png
## 5  Tharaka_Landsat_Enhanced_Maps/Tharaka_2010_dry_feb.png
## 6  Tharaka_Landsat_Enhanced_Maps/Tharaka_2010_dry_sep.png
## 7  Tharaka_Landsat_Enhanced_Maps/Tharaka_2010_wet_dec.png
## 8  Tharaka_Landsat_Enhanced_Maps/Tharaka_2010_wet_may.png
## 9  Tharaka_Landsat_Enhanced_Maps/Tharaka_2020_dry_feb.png
## 10 Tharaka_Landsat_Enhanced_Maps/Tharaka_2020_dry_sep.png
## 11 Tharaka_Landsat_Enhanced_Maps/Tharaka_2020_wet_dec.png
## 12 Tharaka_Landsat_Enhanced_Maps/Tharaka_2020_wet_may.png
## 13 Tharaka_Landsat_Enhanced_Maps/Tharaka_2025_dry_feb.png
## 14 Tharaka_Landsat_Enhanced_Maps/Tharaka_2025_dry_sep.png
## 15 Tharaka_Landsat_Enhanced_Maps/Tharaka_2025_wet_dec.png
## 16                                                    <NA>
```

```r
# ============================================================
# SECTION 33: STUDY-AREA LOCATION MAP
# Kenya -> Tharaka-Nithi County -> Tharaka study area
# ============================================================

library(sf)
library(ggplot2)
library(geodata)
library(patchwork)

# ------------------------------------------------------------
# 1. Create output directories
# ------------------------------------------------------------

locator_dir <- "Tharaka_Land_Cover_Results"

boundary_dir <- file.path(
  locator_dir,
  "Administrative_Boundaries"
)

plot_dir <- "Tharaka_Landsat_Enhanced_Maps"
```

```r
dir.create(
  boundary_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

dir.create(
  plot_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

# ------------------------------------------------------------
# 2. Load Kenya administrative boundaries
# ------------------------------------------------------------

message("Loading Kenya administrative boundaries...")

kenya_counties <- sf::st_as_sf(
  geodata::gadm(
    country = "KEN",
    level = 1,
    path = boundary_dir
  )
)

kenya_country <- sf::st_as_sf(
  geodata::gadm(
    country = "KEN",
    level = 0,
    path = boundary_dir
  )
)

stopifnot(
  nrow(kenya_counties) > 0,
  "NAME_1" %in% names(kenya_counties)
)

# ------------------------------------------------------------
# 3. Select Tharaka-Nithi County
# ------------------------------------------------------------

county <- kenya_counties[
  grepl(
    "tharaka",
    kenya_counties$NAME_1,
    ignore.case = TRUE
  ) &
    grepl(
      "nithi",
      kenya_counties$NAME_1,
      ignore.case = TRUE
```

```
    ),
]

if (nrow(county) != 1) {
  stop(
    "Expected exactly one Tharaka-Nithi County feature; found ",
    nrow(county),
    ". Check unique(kenya_counties$NAME_1)."
  )
}

message("Selected county: ", county$NAME_1)

# ------------------------------------------------------------
# 4. Recover the Tharaka study-area boundary
#
# Prefer the existing study_area_sf or study_area object.
# If neither is available, reconstruct it from GADM level 2.
# No Earth Engine query is needed for this location map.
# ------------------------------------------------------------

if (exists("study_area_sf", inherits = TRUE)) {

  roi_sf <- get("study_area_sf", inherits = TRUE)

  if (!inherits(roi_sf, "sf")) {
    roi_sf <- sf::st_as_sf(roi_sf)
  }

} else if (exists("study_area", inherits = TRUE)) {

  roi_sf <- sf::st_as_sf(
    get("study_area", inherits = TRUE)
  )

} else {

  message("Recovering Tharaka boundary from GADM level 2...")

  kenya_adm2 <- sf::st_as_sf(
    geodata::gadm(
      country = "KEN",
      level = 2,
      path = boundary_dir
    )
  )

  roi_sf <- kenya_adm2[
    kenya_adm2$NAME_2 == "Tharaka" &
      kenya_adm2$NAME_1 == "Tharaka-Nithi",
  ]

  if (nrow(roi_sf) != 1) {
```

```
    stop(
      "Could not identify exactly one Tharaka study-area polygon."
    )
  }
}

# ------------------------------------------------------------
# 5. Standardize and validate geometries
# ------------------------------------------------------------

kenya_counties <- sf::st_transform(
  kenya_counties, 4326
)

kenya_country <- sf::st_transform(
  kenya_country, 4326
)

county <- sf::st_transform(
  county, 4326
)

roi_sf <- sf::st_transform(
  roi_sf, 4326
)

roi_sf <- sf::st_make_valid(roi_sf)

if (nrow(roi_sf) == 0 ||
    any(sf::st_is_empty(roi_sf))) {
  stop("The Tharaka study-area geometry is empty.")
}

# Retain only polygon components if validity repair
# has produced a geometry collection.
geometry_types <- as.character(
  sf::st_geometry_type(roi_sf)
)

if (any(geometry_types == "GEOMETRY")) {
  roi_sf <- sf::st_collection_extract(
    roi_sf,
    "POLYGON",
    warn = FALSE
  )
}

if (nrow(roi_sf) == 0 ||
    !all(sf::st_is_valid(roi_sf))) {
  stop("The study-area boundary is not a valid polygon.")
}

message("Study-area geometry validated successfully.")
```

```r
# Confirm the study area intersects the intended county.
overlap <- sf::st_intersects(
  roi_sf,
  county,
  sparse = FALSE
)

if (!all(overlap)) {
  stop(
    "The study-area boundary does not intersect ",
    "Tharaka-Nithi County. Check the selected boundaries."
  )
}

# Calculate percentage of the study area inside the county.
roi_area <- sf::st_transform(roi_sf, 6933)
county_area <- sf::st_transform(county, 6933)

inside_area <- sf::st_intersection(
  roi_area,
  county_area
)
```

```
## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries
```

```r
overlap_pct <- 100 * as.numeric(
  sum(sf::st_area(inside_area)) /
    sum(sf::st_area(roi_area))
)

cat(
  "Percentage of study area inside Tharaka-Nithi County:",
  round(overlap_pct, 2), "%\n"
)
```

```
## Percentage of study area inside Tharaka-Nithi County: 100 %
```

```r
# ------------------------------------------------------------
# 6. Map 1: Kenya, with county boundaries
# ------------------------------------------------------------

p_kenya <- ggplot() +
  geom_sf(
    data = kenya_country,
    fill = "#F7F7F7",
    colour = "#444444",
    linewidth = 0.35
  ) +
  geom_sf(
    data = kenya_counties,
    fill = NA,
    colour = "#B7B7B7",
    linewidth = 0.2
  ) +
```

```
  geom_sf(
    data = county,
    fill = "#E76F51",
    colour = "#8C2D04",
    linewidth = 0.65
  ) +
  labs(
    title = "KENYA",
    subtitle = "Tharaka-Nithi County highlighted"
  ) +
  coord_sf(expand = FALSE) +
  theme_void(base_size = 11) +
  theme(
    plot.title = element_text(
      face = "bold", hjust = 0.5
    ),
    plot.subtitle = element_text(
      hjust = 0.5, size = 9
    ),
    panel.border = element_rect(
      colour = "black",
      fill = NA,
      linewidth = 0.7
    )
  )

# ------------------------------------------------------------
# 7. Map 2: Tharaka-Nithi County and study area
# ------------------------------------------------------------

county_bbox <- sf::st_bbox(county)

county_x <- c(
  unname(county_bbox["xmin"]) - 0.12,
  unname(county_bbox["xmax"]) + 0.12
)

county_y <- c(
  unname(county_bbox["ymin"]) - 0.08,
  unname(county_bbox["ymax"]) + 0.08
)

p_county <- ggplot() +
  geom_sf(
    data = county,
    fill = "#E9E9E9",
    colour = "#333333",
    linewidth = 0.8
  ) +
  geom_sf(
    data = roi_sf,
    fill = "#65A765",
    colour = "#185C37",
```

```
    linewidth = 0.8
  ) +
  labs(
    title = "THARAKA-NITHI COUNTY",
    subtitle = "Tharaka study area highlighted"
  ) +
  coord_sf(
    xlim = county_x,
    ylim = county_y,
    expand = FALSE
  ) +
  theme_void(base_size = 11) +
  theme(
    plot.title = element_text(
      face = "bold", hjust = 0.5
    ),
    plot.subtitle = element_text(
      hjust = 0.5, size = 9
    ),
    panel.border = element_rect(
      colour = "black",
      fill = NA,
      linewidth = 0.7
    )
  )

# ------------------------------------------------------------
# 8. Map 3: Detailed Tharaka study-area boundary
# ------------------------------------------------------------

roi_bbox <- sf::st_bbox(roi_sf)

roi_x <- c(
  unname(roi_bbox["xmin"]) - 0.025,
  unname(roi_bbox["xmax"]) + 0.025
)

roi_y <- c(
  unname(roi_bbox["ymin"]) - 0.025,
  unname(roi_bbox["ymax"]) + 0.025
)

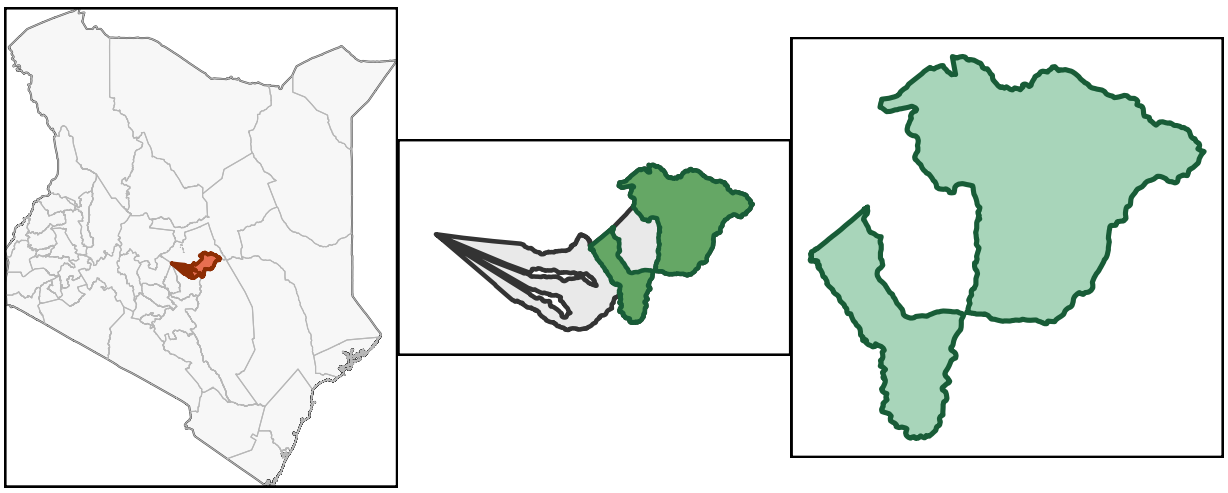
p_detail <- ggplot() +
  geom_sf(
    data = roi_sf,
    fill = "#A8D5BA",
    colour = "#185C37",
    linewidth = 0.9
  ) +
  labs(
    title = "THARAKA STUDY AREA",
    subtitle = "Boundary used in the Landsat analysis"
  ) +
```

```
  coord_sf(
    xlim = roi_x,
    ylim = roi_y,
    expand = FALSE
  ) +
  theme_void(base_size = 11) +
  theme(
    plot.title = element_text(
      face = "bold", hjust = 0.5
    ),
    plot.subtitle = element_text(
      hjust = 0.5, size = 9
    ),
    panel.border = element_rect(
      colour = "black",
      fill = NA,
      linewidth = 0.7
    )
  )

# ------------------------------------------------------------
# 9. Combine the three maps
# ------------------------------------------------------------

p_locator_0 <- (p_kenya | p_county | p_detail) +
  plot_layout(widths = c(1, 1, 1.1)) +
  plot_annotation(
    title = "Location of the Tharaka Study Area, Kenya",
    subtitle = paste(
      "Geographic context for the 2000, 2010, 2020 and 2025",
      "seasonal Landsat analysis"
    ),
    caption = paste(
      "Administrative boundaries: GADM.",
      "Study boundary: Tharaka Sub-county."
    ),
    theme = theme(
      plot.title = element_text(
        size = 17, face = "bold", hjust = 0.5
      ),
      plot.subtitle = element_text(
        size = 11, hjust = 0.5
      ),
      plot.caption = element_text(
        size = 8, hjust = 0
      )
    )
  )

print(p_locator_0)
```



Administrative boundaries: GADM. Study boundary: Tharaka Sub−county.

```
# ------------------------------------------------------------
# 10. Export the location map
# ------------------------------------------------------------

png_file <- file.path(
  plot_dir,
  "Tharaka_Study_Area_Location_Map.png"
)

pdf_file <- file.path(
  plot_dir,
  "Tharaka_Study_Area_Location_Map.pdf"
)

ggsave(
  filename = png_file,
  plot = p_locator_0,
  width = 15,
  height = 7,
  units = "in",
  dpi = 300,
  bg = "white"
)

ggsave(
  filename = pdf_file,
```

```
  plot = p_locator_0,
  width = 15,
  height = 7,
  units = "in",
  bg = "white"
)

cat(
  "\nSECTION 33 COMPLETED\n",
  "County: ", county$NAME_1, "\n",
  "Study-area overlap: ", round(overlap_pct, 2), "%\n",
  "PNG: ", normalizePath(png_file, winslash = "/"), "\n",
  "PDF: ", normalizePath(pdf_file, winslash = "/"), "\n",
  sep = ""
)
```

```
##
## SECTION 33 COMPLETED
## County: Tharaka-Nithi
## Study-area overlap: 100%
## PNG: C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Landsat_Enhanced_Maps/Tharaka_Study_Area_Location_Map.pn
## PDF: C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Landsat_Enhanced_Maps/Tharaka_Study_Area_Location_Map.pd
```

```
# Select Tharaka-Nithi County robustly
county <- kenya_counties[
  grepl(
    "tharaka",
    kenya_counties$NAME_1,
    ignore.case = TRUE
  ) &
  grepl(
    "nithi",
    kenya_counties$NAME_1,
    ignore.case = TRUE
  ),
]

if (nrow(county) != 1) {
  stop(
    "Expected one Tharaka-Nithi County feature, found ",
    nrow(county),
    ". Check kenya_counties$NAME_1."
  )
}

cat("Selected county:", county$NAME_1, "\n")
```

```
## Selected county: Tharaka-Nithi
```

```
# ============================================================
# FIX: VALIDATE AND REPAIR STUDY-AREA GEOMETRY
# ============================================================

# Check the current geometry
print(sf::st_is_valid(roi_sf, reason = TRUE))
```

```
## [1] "Valid Geometry"
```

```
# Repair invalid geometry
roi_sf <- sf::st_make_valid(roi_sf)

# Keep polygonal components only
roi_sf <- sf::st_collection_extract(
  roi_sf,
  "POLYGON",
  warn = FALSE
)
```

```
## Warning in st_collection_extract.sf(roi_sf, "POLYGON", warn = FALSE): x is
## already of type POLYGON.
```

```
# Confirm validity
print(sf::st_is_valid(roi_sf))
```

```
## [1] TRUE
```

```
# Stop only if the geometry remains invalid
if (!all(sf::st_is_valid(roi_sf))) {
  stop("The study-area geometry is still invalid. Inspect the geometry.")
}

cat("Study-area geometry validated successfully.\n")
```

```
## Study-area geometry validated successfully.
```

```
# ============================================================
# SECTION 33A: CHECK STUDY-AREA LOCATION
# ============================================================

# Check whether the ROI intersects Tharaka-Nithi County
overlap <- sf::st_intersects(
  roi_sf,
  county,
  sparse = FALSE
)

print(overlap)
```

```
##      [,1]
## [1,] TRUE
```

```
if (all(overlap)) {
  cat("The study area intersects Tharaka-Nithi County.\n")
} else {
  warning(
    "The study area does not intersect Tharaka-Nithi County. ",
    "Check the ROI and coordinate reference system."
  )
}
```

```
## The study area intersects Tharaka-Nithi County.
```

```
# Estimate how much of the ROI lies inside the county
roi_area <- sf::st_transform(roi_sf, 6933)
county_area <- sf::st_transform(county, 6933)
```

```r
inside_area <- sf::st_intersection(
  roi_area,
  county_area
)
```

```
## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries
```

```r
overlap_pct <- 100 * sum(sf::st_area(inside_area)) /
  sum(sf::st_area(roi_area))

cat(
  "Percentage of study area inside Tharaka-Nithi County:",
  round(as.numeric(overlap_pct), 2), "%\n"
)
```

```
## Percentage of study area inside Tharaka-Nithi County: 100 %
```

```r
# ============================================================
# SECTION 33: STUDY-AREA LOCATION MAP
# Kenya -> Tharaka-Nithi County -> Tharaka study area
# ============================================================

library(sf)
library(ggplot2)
library(geodata)
library(reticulate)
library(jsonlite)
library(geojsonsf)
library(patchwork)

# ------------------------------------------------------------
# 1. Output folder
# ------------------------------------------------------------

locator_dir <- "Tharaka_Land_Cover_Results"
dir.create(locator_dir, showWarnings = FALSE, recursive = TRUE)

boundary_dir <- file.path(locator_dir, "Administrative_Boundaries")
dir.create(boundary_dir, showWarnings = FALSE, recursive = TRUE)

# ------------------------------------------------------------
# 2. Download GADM administrative boundaries
# ------------------------------------------------------------

kenya_counties <- sf::st_as_sf(
  geodata::gadm(
    country = "KEN",
    level = 1,
    path = boundary_dir
  )
)

kenya_country <- sf::st_as_sf(
  geodata::gadm(
```

```
    country = "KEN",
    level = 0,
    path = boundary_dir
  )
)

# Inspect county names if necessary
print(sort(unique(kenya_counties$NAME_1)))
```

```
##  [1] "Baringo"         "Bomet"           "Bungoma"         "Busia"
##  [5] "Elgeyo-Marakwet" "Embu"            "Garissa"         "Homa Bay"
##  [9] "Isiolo"          "Kajiado"         "Kakamega"        "Kericho"
## [13] "Kiambu"          "Kilifi"          "Kirinyaga"       "Kisii"
## [17] "Kisumu"          "Kitui"           "Kwale"           "Laikipia"
## [21] "Lamu"            "Machakos"        "Makueni"         "Mandera"
## [25] "Marsabit"        "Meru"            "Migori"          "Mombasa"
## [29] "Murang'a"        "Nairobi"         "Nakuru"          "Nandi"
## [33] "Narok"           "Nyamira"         "Nyandarua"       "Nyeri"
## [37] "Samburu"         "Siaya"           "Taita Taveta"    "Tana River"
## [41] "Tharaka-Nithi"   "Trans Nzoia"     "Turkana"         "Uasin Gishu"
## [45] "Vihiga"          "Wajir"           "West Pokot"
```

```
# Select Tharaka-Nithi County robustly
county <- kenya_counties[
  grepl(
    "tharaka",
    kenya_counties$NAME_1,
    ignore.case = TRUE
  ) &
  grepl(
    "nithi",
    kenya_counties$NAME_1,
    ignore.case = TRUE
  ),
]

if (nrow(county) != 1) {
  stop(
    "Expected one Tharaka-Nithi County feature, found ",
    nrow(county),
    ". Check kenya_counties$NAME_1."
  )
}

cat("Selected county:", county$NAME_1, "\n")
```

```
## Selected county: Tharaka-Nithi
```

```
# ------------------------------------------------------------
# 3. Convert Earth Engine ROI to sf and repair its geometry
# ------------------------------------------------------------

ee <- reticulate::import("ee", convert = FALSE)

# Extract the ROI already used in the Landsat analysis
```

```r
roi_info <- reticulate::py_to_r(
  ee$Feature(roi)$getInfo()
)

roi_feature <- list(
  type = "Feature",
  properties = list(name = "Tharaka study area"),
  geometry = roi_info$geometry
)

roi_json <- jsonlite::toJSON(
  roi_feature,
  auto_unbox = TRUE,
  null = "null"
)

roi_sf <- geojsonsf::geojson_sf(roi_json)

# Set coordinate reference systems consistently
county <- sf::st_transform(county, 4326)
kenya_counties <- sf::st_transform(kenya_counties, 4326)
kenya_country <- sf::st_transform(kenya_country, 4326)
roi_sf <- sf::st_transform(roi_sf, 4326)

# Repair the geometry AFTER recreating roi_sf
cat("Original geometry validity:\n")
```

```
## Original geometry validity:
```

```r
print(sf::st_is_valid(roi_sf, reason = TRUE))
```

```
## [1] "Loop 0: Edge 84 is degenerate (duplicate vertex)"
```

```r
roi_sf <- sf::st_make_valid(roi_sf)

# Keep polygonal geometry; if already polygonal, no extraction is needed
if (!all(sf::st_is_valid(roi_sf))) {
  roi_sf <- sf::st_collection_extract(
    roi_sf, "POLYGON", warn = FALSE
  )
}

# Confirm the final geometry
valid_roi <- sf::st_is_valid(roi_sf)

if (!all(valid_roi)) {
  stop("ROI geometry remains invalid after repair.")
}

if (nrow(roi_sf) == 0) {
  stop("No valid polygon remains in roi_sf.")
}

cat("Study-area geometry validated successfully.\n")
```

```
## Study-area geometry validated successfully.
```

```
# Verify the study area still overlaps the intended county
overlap <- sf::st_intersects(
  roi_sf, county, sparse = FALSE
)

if (!all(overlap)) {
  warning("The ROI does not intersect Tharaka-Nithi County.")
}

# Calculate percentage of ROI inside the county
roi_area <- sf::st_transform(roi_sf, 6933)
county_area <- sf::st_transform(county, 6933)

inside_area <- sf::st_intersection(
  roi_area, county_area
)
```

```
## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries
```

```
overlap_pct <- 100 *
  sum(sf::st_area(inside_area)) /
  sum(sf::st_area(roi_area))

cat(
  "Percentage of study area inside Tharaka-Nithi County:",
  round(as.numeric(overlap_pct), 2), "%\n"
)
```

```
## Percentage of study area inside Tharaka-Nithi County: 99.99 %
```

```
# ------------------------------------------------------------
# 4. Create map 1: Kenya with county highlighted
# ------------------------------------------------------------

p_kenya <- ggplot() +
  geom_sf(
    data = kenya_country,
    fill = "#F7F7F7",
    colour = "#444444",
    linewidth = 0.35
  ) +
  geom_sf(
    data = kenya_counties,
    fill = NA,
    colour = "#B7B7B7",
    linewidth = 0.2
  ) +
  geom_sf(
    data = county,
    fill = "#E76F51",
    colour = "#8C2D04",
    linewidth = 0.65
  ) +
```

```
  labs(
    title = "KENYA",
    subtitle = "Tharaka-Nithi County highlighted"
  ) +
  coord_sf(expand = FALSE) +
  theme_void(base_size = 11) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5, size = 9),
    panel.border = element_rect(
      colour = "black", fill = NA, linewidth = 0.7
    )
  )

# ------------------------------------------------------------
# 5. Create map 2: County with study area highlighted
# ------------------------------------------------------------

county_bbox <- sf::st_bbox(county)

county_x <- c(
  unname(county_bbox["xmin"]) - 0.12,
  unname(county_bbox["xmax"]) + 0.12
)

county_y <- c(
  unname(county_bbox["ymin"]) - 0.08,
  unname(county_bbox["ymax"]) + 0.08
)

p_county <- ggplot() +
  geom_sf(
    data = county,
    fill = "#E9E9E9",
    colour = "#333333",
    linewidth = 0.8
  ) +
  geom_sf(
    data = roi_sf,
    fill = "#65A765",
    colour = "#185C37",
    linewidth = 0.8
  ) +
  labs(
    title = "THARAKA-NITHI COUNTY",
    subtitle = "Tharaka study area highlighted"
  ) +
  coord_sf(
    xlim = county_x,
    ylim = county_y,
    expand = FALSE
  ) +
  theme_void(base_size = 11) +
```

```r
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5, size = 9),
    panel.border = element_rect(
      colour = "black", fill = NA, linewidth = 0.7
    )
  )

# ------------------------------------------------------------
# 6. Create map 3: Detailed study-area boundary
# ------------------------------------------------------------

roi_bbox <- sf::st_bbox(roi_sf)

roi_x <- c(
  unname(roi_bbox["xmin"]) - 0.025,
  unname(roi_bbox["xmax"]) + 0.025
)

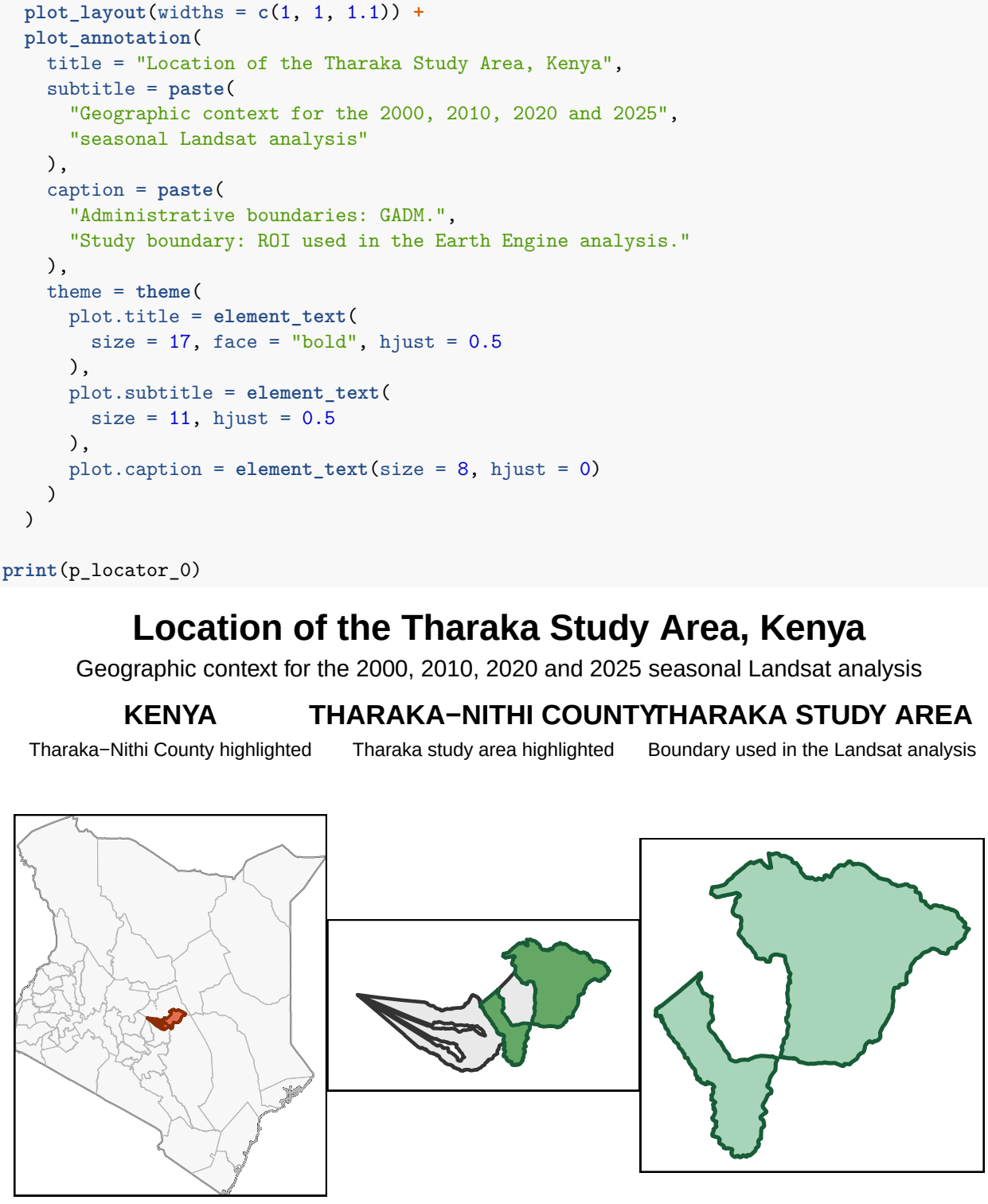
roi_y <- c(
  unname(roi_bbox["ymin"]) - 0.025,
  unname(roi_bbox["ymax"]) + 0.025
)

p_detail <- ggplot() +
  geom_sf(
    data = roi_sf,
    fill = "#A8D5BA",
    colour = "#185C37",
    linewidth = 0.9
  ) +
  labs(
    title = "THARAKA STUDY AREA",
    subtitle = "Boundary used in the Landsat analysis"
  ) +
  coord_sf(
    xlim = roi_x,
    ylim = roi_y,
    expand = FALSE
  ) +
  theme_void(base_size = 11) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5, size = 9),
    panel.border = element_rect(
      colour = "black", fill = NA, linewidth = 0.7
    )
  )

# ------------------------------------------------------------
# 7. Combine the three maps
# ------------------------------------------------------------
```

```r
p_locator_0 <- (p_kenya | p_county | p_detail) +
  plot_layout(widths = c(1, 1, 1.1)) +
  plot_annotation(
    title = "Location of the Tharaka Study Area, Kenya",
    subtitle = paste(
      "Geographic context for the 2000, 2010, 2020 and 2025",
      "seasonal Landsat analysis"
    ),
    caption = paste(
      "Administrative boundaries: GADM.",
      "Study boundary: ROI used in the Earth Engine analysis."
    ),
    theme = theme(
      plot.title = element_text(
        size = 17, face = "bold", hjust = 0.5
      ),
      plot.subtitle = element_text(
        size = 11, hjust = 0.5
      ),
      plot.caption = element_text(size = 8, hjust = 0)
    )
  )

print(p_locator_0)
```



```r
# ------------------------------------------------------------
# 8. Export publication-quality files
# ------------------------------------------------------------


# ------------------------------------------------------------
# SAVE LOCATOR MAP IN THE EXISTING MAP DIRECTORY
# ------------------------------------------------------------

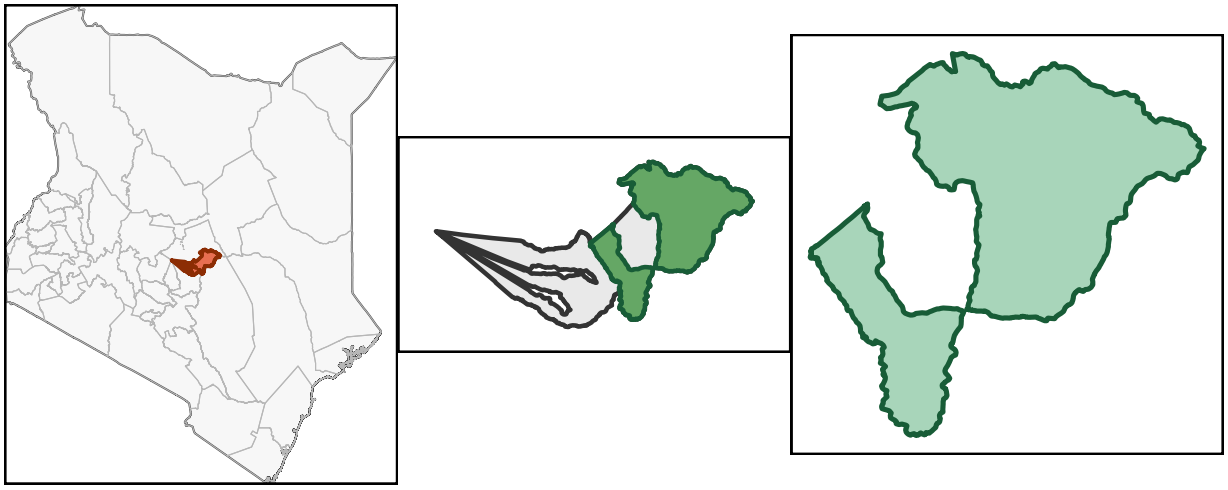
plot_dir <- "Tharaka_Landsat_Enhanced_Maps"

dir.create(
  plot_dir,
  showWarnings = FALSE,
  recursive = TRUE
)

# Save high-resolution PNG
ggsave(
  filename = file.path(
    plot_dir,
    "Tharaka_Study_Area_Location_Map.png"
  ),
  plot = p_locator_0,
  width = 15,
  height = 7,
  units = "in",
  dpi = 300,
  bg = "white"
)

# Save publication-quality PDF
ggsave(
  filename = file.path(
    plot_dir,
    "Tharaka_Study_Area_Location_Map.pdf"
  ),
  plot = p_locator_0,
  width = 15,
  height = 7,
  units = "in",
  bg = "white"
)

# Display the map
print(p_locator_0)
```



Administrative boundaries: GADM. Study boundary: ROI used in the Earth Engine analysis.

```
cat(
  "\nLocator maps saved successfully in:\n",
  normalizePath(plot_dir),
  "\n"
)
```

```
##
## Locator maps saved successfully in:
##  C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Landsat_Enhanced_Maps
```

```
# ============================================================
# SECTION 33B: LOCATOR MAP WITH KEY AND NORTH COMPASS
# ============================================================

library(ggplot2)
library(sf)
library(patchwork)
library(ggspatial)

# ------------------------------------------------------------
# 1. Kenya overview
# ------------------------------------------------------------

p_kenya <- ggplot() +
  geom_sf(
    data = kenya_country,
    fill = "#F7F7F7",
```

```
    colour = "#444444",
    linewidth = 0.4
  ) +
  geom_sf(
    data = kenya_counties,
    fill = NA,
    colour = "#B7B7B7",
    linewidth = 0.2
  ) +
  geom_sf(
    data = county,
    aes(fill = "Tharaka-Nithi County"),
    colour = "#8C2D04",
    linewidth = 0.7
  ) +
  scale_fill_manual(
    name = "Map key",
    values = c("Tharaka-Nithi County" = "#E76F51")
  ) +
  labs(
    title = "KENYA",
    subtitle = "Study county highlighted"
  ) +
  coord_sf(expand = FALSE) +
  theme_void() +
  theme(
    plot.title = element_text(
      face = "bold", size = 12, hjust = 0.5
    ),
    plot.subtitle = element_text(
      size = 9, hjust = 0.5
    ),
    legend.position = "bottom",
    legend.title = element_text(face = "bold", size = 8),
    legend.text = element_text(size = 8),
    legend.key.size = grid::unit(0.35, "cm")
  )

# ------------------------------------------------------------
# 2. County-level view
# ------------------------------------------------------------

county_bbox <- sf::st_bbox(county)

county_x <- c(
  as.numeric(county_bbox["xmin"]) - 0.12,
  as.numeric(county_bbox["xmax"]) + 0.12
)

county_y <- c(
  as.numeric(county_bbox["ymin"]) - 0.08,
  as.numeric(county_bbox["ymax"]) + 0.08
)
```

```r
p_county <- ggplot() +
  geom_sf(
    data = county,
    fill = "#E9E9E9",
    colour = "#333333",
    linewidth = 0.8
  ) +
  geom_sf(
    data = roi_sf,
    aes(fill = "Tharaka study area"),
    colour = "#185C37",
    linewidth = 0.8
  ) +
  scale_fill_manual(
    name = "Map key",
    values = c("Tharaka study area" = "#65A765")
  ) +
  labs(
    title = "THARAKA-NITHI COUNTY",
    subtitle = "Study boundary highlighted"
  ) +
  coord_sf(
    xlim = county_x,
    ylim = county_y,
    expand = FALSE
  ) +
  theme_void() +
  theme(
    plot.title = element_text(
      face = "bold", size = 12, hjust = 0.5
    ),
    plot.subtitle = element_text(
      size = 9, hjust = 0.5
    ),
    legend.position = "bottom",
    legend.title = element_text(face = "bold", size = 8),
    legend.text = element_text(size = 8),
    legend.key.size = grid::unit(0.35, "cm")
  )

# ------------------------------------------------------------
# 3. Detailed study-area view with north compass
# ------------------------------------------------------------

roi_bbox <- sf::st_bbox(roi_sf)

roi_x <- c(
  as.numeric(roi_bbox["xmin"]) - 0.025,
  as.numeric(roi_bbox["xmax"]) + 0.025
)

roi_y <- c(
  as.numeric(roi_bbox["ymin"]) - 0.025,
```

```
  as.numeric(roi_bbox["ymax"]) + 0.025
)

p_detail <- ggplot() +
  geom_sf(
    data = roi_sf,
    aes(fill = "Study-area boundary"),
    colour = "#185C37",
    linewidth = 0.9
  ) +
  scale_fill_manual(
    name = "Map key",
    values = c("Study-area boundary" = "#A8D5BA")
  ) +
  # North arrow / compass
  ggspatial::annotation_north_arrow(
    location = "tr",
    which_north = "true",
    height = grid::unit(1.0, "cm"),
    width = grid::unit(0.8, "cm"),
    style = ggspatial::north_arrow_fancy_orienteering
  ) +
  labs(
    title = "THARAKA STUDY AREA",
    subtitle = "Boundary used in the Landsat analysis"
  ) +
  coord_sf(
    xlim = roi_x,
    ylim = roi_y,
    expand = FALSE
  ) +
  theme_void() +
  theme(
    plot.title = element_text(
      face = "bold", size = 12, hjust = 0.5
    ),
    plot.subtitle = element_text(
      size = 9, hjust = 0.5
    ),
    legend.position = "bottom",
    legend.title = element_text(face = "bold", size = 8),
    legend.text = element_text(size = 8),
    legend.key.size = grid::unit(0.35, "cm")
  )

# ------------------------------------------------------------
# 4. Combine the three panels
# ------------------------------------------------------------

p_locator <- (p_kenya | p_county | p_detail) +
  plot_layout(widths = c(1, 1, 1.1), guides = "collect") +
  plot_annotation(
    title = "Location of the Tharaka Study Area, Kenya",
```

```
    subtitle = paste(
      "Geographic context for the 2000, 2010, 2020",
      "and 2025 seasonal Landsat analysis"
    ),
    caption = paste(
      "Administrative boundaries: GADM.",
      "Study boundary: ROI used in Google Earth Engine."
    ),
    theme = theme(
      plot.title = element_text(
        face = "bold", size = 16, hjust = 0.5
      ),
      plot.subtitle = element_text(
        size = 10, hjust = 0.5
      ),
      plot.caption = element_text(
        size = 8, hjust = 0.5
      ),
      legend.position = "bottom"
    )
  )

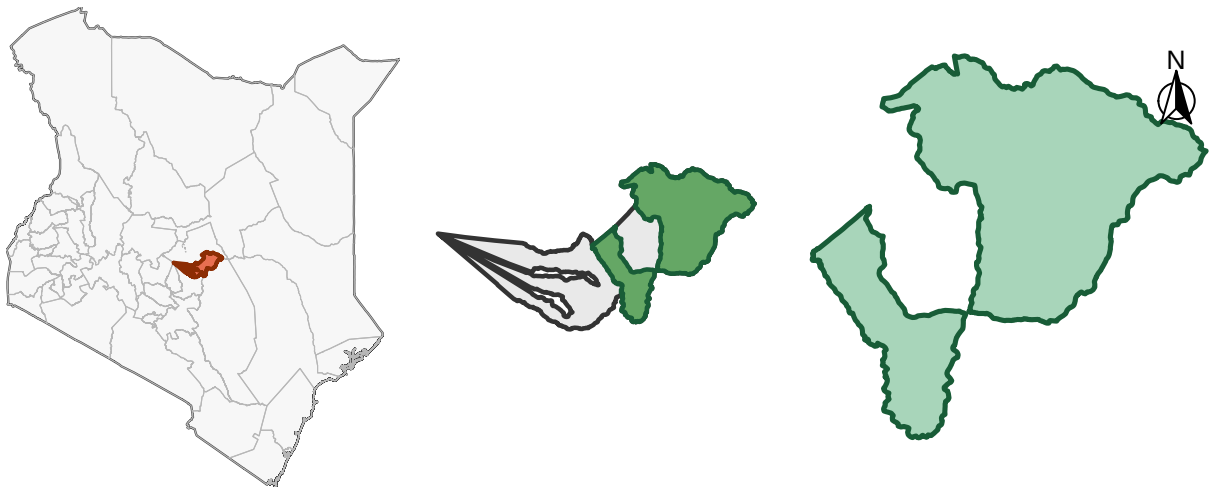
# ------------------------------------------------------------
# 5. Save high-resolution outputs
# ------------------------------------------------------------

locator_dir <- "Tharaka_Land_Cover_Results"
dir.create(locator_dir, showWarnings = FALSE, recursive = TRUE)

ggsave(
  file.path(locator_dir, "Tharaka_Study_Area_Location_Map.png"),
  plot = p_locator,
  width = 15,
  height = 7,
  units = "in",
  dpi = 300,
  bg = "white"
)

ggsave(
  file.path(locator_dir, "Tharaka_Study_Area_Location_Map.pdf"),
  plot = p_locator,
  width = 15,
  height = 7,
  units = "in",
  bg = "white"
)

print(p_locator)
```




```
cat("Locator map saved as PNG and PDF.\n")
```

```
## Locator map saved as PNG and PDF.
```

```
# ------------------------------------------------------------
# SAVE BOTH LOCATOR MAP VERSIONS IN THE SAME DIRECTORY
# ------------------------------------------------------------

plot_dir <- "Tharaka_Landsat_Enhanced_Maps"

dir.create(
  plot_dir,
  showWarnings = FALSE,
  recursive = TRUE
)

# ============================================================
# MAP 1: ORIGINAL LOCATOR MAP (p_locator)
# ============================================================

ggsave(
  filename = file.path(
    plot_dir,
    "Tharaka_Study_Area_Location_Map_Original.png"
  ),
  plot = p_locator,
  width = 15,
```

```
  height = 7,
  units = "in",
  dpi = 300,
  bg = "white"
)

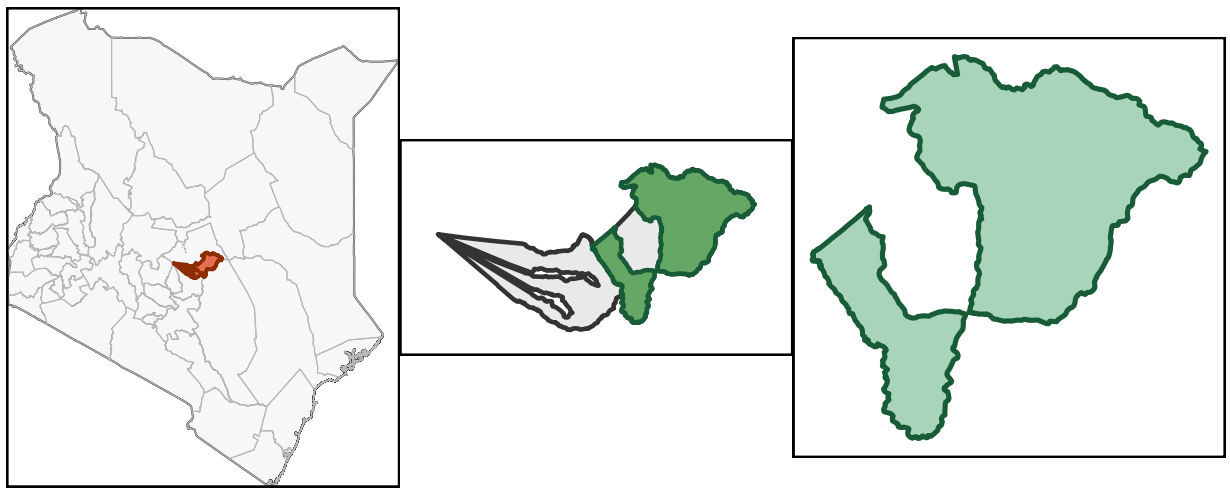
ggsave(
  filename = file.path(
    plot_dir,
    "Tharaka_Study_Area_Location_Map_Original.pdf"
  ),
  plot = p_locator,
  width = 15,
  height = 7,
  units = "in",
  bg = "white"
)

# ========================================================
# MAP 2: UPDATED LOCATOR MAP (p_locator_0)
# ========================================================

ggsave(
  filename = file.path(
    plot_dir,
    "Tharaka_Study_Area_Location_Map_Compass_Key.png"
  ),
  plot = p_locator_0,
  width = 15,
  height = 7,
  units = "in",
  dpi = 300,
  bg = "white"
)

ggsave(
  filename = file.path(
    plot_dir,
    "Tharaka_Study_Area_Location_Map_Compass_Key.pdf"
  ),
  plot = p_locator_0,
  width = 15,
  height = 7,
  units = "in",
  bg = "white"
)

# Display the updated map
print(p_locator_0)
```



Administrative boundaries: GADM. Study boundary: ROI used in the Earth Engine analysis.

```
cat(
  "\nBoth locator map versions saved successfully in:\n",
  normalizePath(plot_dir),
  "\n"
)
```

```
##
## Both locator map versions saved successfully in:
##  C:\Users\pmwaura\DSAS\Dr. Peris\Tharaka_Landsat_Enhanced_Maps
```

```
# ============================================================
# SECTION 34: PRELIMINARY ANNUAL LAND-COVER AREA ESTIMATES
# Source: MODIS MCD12Q1.061 / IGBP LC_Type1, 500 m
# Does not require the historical reference-label worksheet.
# ============================================================

library(dplyr)
library(tidyr)
library(purrr)
library(tibble)
library(ggplot2)
library(readr)
library(reticulate)

if (!exists("roi", inherits = TRUE)) {
  stop(
    "Earth Engine study-area geometry 'roi' is missing. ",
```

```
    "Run the preceding study-area and Earth Engine chunks first."
  )
}

if (!exists("ee", inherits = TRUE)) {
  ee <- reticulate::import("ee", convert = FALSE)
  ee$Initialize(project = "thinking-leaf-305215")
}

# Reuse the existing Earth Engine helpers; define them if absent.
if (!exists("ee_method", mode = "function", inherits = TRUE)) {
  ee_method <- function(object, method, ...) {
    fn <- reticulate::py_get_attr(object, method)
    reticulate::py_call(fn, ...)
  }
}

if (!exists("ee_get_info", mode = "function", inherits = TRUE)) {
  ee_get_info <- function(object) {
    reticulate::py_to_r(ee_method(object, "getInfo"))
  }
}

# MCD12Q1 version 6.1 currently supplies annual maps from 2001-2024.
# Use nearest available source years for requested periods 2000 and 2025.
landcover_year_map <- tibble::tribble(
  ~requested_year, ~dataset_year, ~year_note,
  2000L,           2001L,         "2001 source year; proxy for 2000",
  2010L,           2010L,         "Exact source year",
  2020L,           2020L,         "Exact source year",
  2025L,           2024L,         "2024 source year; proxy for 2025"
)

class_levels <- c(
  "Forest",
  "Bushland / shrub-savanna",
  "Grassland",
  "Agriculture",
  "Settlement",
  "Other / non-target"
)

modis_lc_collection <- ee$ImageCollection("MODIS/061/MCD12Q1")

# The grouped reduction sums pixel area (hectares) by the LC_Type1 code.
# Image band order is important: area first, class code second.
area_ha_band <- ee$Image$pixelArea()$
  divide(10000)$
  rename("area_ha")

group_reducer <- ee_method(
  ee$Reducer$sum(),
  "group",
```

```
  groupField = 1L,
  groupName = "land_cover_code"
)

area_results <- vector("list", nrow(landcover_year_map))

for (i in seq_len(nrow(landcover_year_map))) {

  requested_year <- landcover_year_map$requested_year[i]
  dataset_year <- landcover_year_map$dataset_year[i]

  start_date <- sprintf("%04d-01-01", dataset_year)
  end_date <- sprintf("%04d-01-01", dataset_year + 1L)

  message(
    "Calculating area for source year ", dataset_year,
    " (requested period ", requested_year, ")..."
  )

  annual_collection <- ee_method(
    modis_lc_collection,
    "filterDate",
    start_date,
    end_date
  )

  n_images <- as.integer(
    ee_get_info(ee_method(annual_collection, "size"))
  )

  if (is.na(n_images) || n_images < 1L) {
    stop(
      "No MCD12Q1 image found for dataset year ", dataset_year,
      ". Check dataset availability in Earth Engine."
    )
  }

  annual_image <- ee_method(annual_collection, "first")
  class_band <- ee_method(
    ee_method(annual_image, "select", "LC_Type1"),
    "rename",
    "land_cover_code"
  )

  grouped_image <- ee_method(area_ha_band, "addBands", class_band)

  reduction_object <- ee_method(
    grouped_image,
    "reduceRegion",
    reducer = group_reducer,
    geometry = roi,
    scale = 500,
    maxPixels = 1e10,
```

```
    tileScale = 4
  )

  reduction_info <- ee_get_info(reduction_object)
  groups <- reduction_info[["groups"]]

  if (is.null(groups) || length(groups) == 0L) {
    stop(
      "The grouped area reduction returned no classes for ",
      dataset_year, ". Check the ROI and Earth Engine response."
    )
  }

  group_table <- purrr::map_dfr(groups, function(g) {
    g <- as.list(g)
    tibble(
      modis_code = as.integer(unlist(g[["land_cover_code"]])),
      area_ha = as.numeric(unlist(g[["sum"]]))
    )
  }) %>%
    mutate(
      land_cover_class = case_when(
        modis_code %in% 1:5 ~ "Forest",
        modis_code %in% 6:9 ~ "Bushland / shrub-savanna",
        modis_code == 10 ~ "Grassland",
        modis_code %in% c(12, 14) ~ "Agriculture",
        modis_code == 13 ~ "Settlement",
        TRUE ~ "Other / non-target"
      )
    ) %>%
    group_by(land_cover_class) %>%
    summarise(area_ha = sum(area_ha, na.rm = TRUE), .groups = "drop")

  area_results[[i]] <- group_table %>%
    mutate(
      requested_year = requested_year,
      dataset_year = dataset_year,
      .before = 1
    )
}

observed_area_long <- bind_rows(area_results)

# Include classes absent in a year as zero so tables/graphs stay comparable.
landcover_area_trends <- tidyr::crossing(
  landcover_year_map,
  land_cover_class = class_levels
) %>%
  left_join(
    observed_area_long,
    by = c("requested_year", "dataset_year", "land_cover_class")
  ) %>%
  mutate(area_ha = tidyr::replace_na(area_ha, 0)) %>%
```

```
  group_by(requested_year, dataset_year, year_note) %>%
  mutate(
    total_mapped_area_ha = sum(area_ha, na.rm = TRUE),
    area_pct = if_else(
      total_mapped_area_ha > 0,
      100 * area_ha / total_mapped_area_ha,
      NA_real_
    )
  ) %>%
  ungroup() %>%
  arrange(dataset_year, match(land_cover_class, class_levels))

# Preserve the detailed long-format results, including bushland and other.
results_dir <- "Tharaka_Land_Cover_Results"
dir.create(results_dir, recursive = TRUE, showWarnings = FALSE)

readr::write_csv(
  landcover_area_trends,
  file.path(results_dir, "Tharaka_MODIS_Landcover_Area_Trends_Long.csv")
)

cat("\nAnnual area calculations completed.\n")
```

```
##
## Annual area calculations completed.
```

```
print(landcover_area_trends, n = Inf, width = Inf)
```

```
## # A tibble: 24 x 7
##    requested_year dataset_year year_note
##             <int>        <int> <chr>
##  1           2000         2001 2001 source year; proxy for 2000
##  2           2000         2001 2001 source year; proxy for 2000
##  3           2000         2001 2001 source year; proxy for 2000
##  4           2000         2001 2001 source year; proxy for 2000
##  5           2000         2001 2001 source year; proxy for 2000
##  6           2000         2001 2001 source year; proxy for 2000
##  7           2010         2010 Exact source year
##  8           2010         2010 Exact source year
##  9           2010         2010 Exact source year
## 10           2010         2010 Exact source year
## 11           2010         2010 Exact source year
## 12           2010         2010 Exact source year
## 13           2020         2020 Exact source year
## 14           2020         2020 Exact source year
## 15           2020         2020 Exact source year
## 16           2020         2020 Exact source year
## 17           2020         2020 Exact source year
## 18           2020         2020 Exact source year
## 19           2025         2024 2024 source year; proxy for 2025
## 20           2025         2024 2024 source year; proxy for 2025
## 21           2025         2024 2024 source year; proxy for 2025
## 22           2025         2024 2024 source year; proxy for 2025
## 23           2025         2024 2024 source year; proxy for 2025
```

```
## 24           2025         2024 2024 source year; proxy for 2025
##    land_cover_class           area_ha total_mapped_area_ha area_pct
##    <chr>                        <dbl>                <dbl>    <dbl>
##  1 Forest                       119.                121119.  0.0982
##  2 Bushland / shrub-savanna    2082.                121119.  1.72
##  3 Grassland                 102588.                121119. 84.7
##  4 Agriculture                16330.                121119. 13.5
##  5 Settlement                     0                 121119.  0
##  6 Other / non-target             0                 121119.  0
##  7 Forest                         0                 121119.  0
##  8 Bushland / shrub-savanna    3142.                121119.  2.59
##  9 Grassland                 105861.                121119. 87.4
## 10 Agriculture                12116.                121119. 10.0
## 11 Settlement                    0                 121119.  0
## 12 Other / non-target            0                 121119.  0
## 13 Forest                       38.2               121119.  0.0315
## 14 Bushland / shrub-savanna    2166.                121119.  1.79
## 15 Grassland                 105516.                121119. 87.1
## 16 Agriculture                13399.                121119. 11.1
## 17 Settlement                    0                 121119.  0
## 18 Other / non-target            0                 121119.  0
## 19 Forest                        4.67              121119.  0.00386
## 20 Bushland / shrub-savanna    2408.                121119.  1.99
## 21 Grassland                 104526.                121119. 86.3
## 22 Agriculture                14180.                121119. 11.7
## 23 Settlement                    0                 121119.  0
## 24 Other / non-target            0                 121119.  0
```

```
# ============================================================
# SECTION 35: TABLES FOR REQUESTED LAND-COVER TYPES
# ============================================================

requested_classes <- c(
  "Forest",
  "Grassland",
  "Agriculture",
  "Settlement"
)

landcover_trends_table_long <- landcover_area_trends %>%
  filter(land_cover_class %in% requested_classes) %>%
  select(
    requested_period = requested_year,
    dataset_year,
    year_note,
    land_cover_class,
    area_ha,
    area_pct
  ) %>%
  arrange(dataset_year, match(land_cover_class, requested_classes))

landcover_trends_table_wide_ha <- landcover_trends_table_long %>%
  select(requested_period, dataset_year, year_note,
         land_cover_class, area_ha) %>%
```

```
  pivot_wider(
    names_from = land_cover_class,
    values_from = area_ha,
    names_glue = "ha_{land_cover_class}"
  ) %>%
  arrange(dataset_year)

landcover_trends_table_wide_pct <- landcover_trends_table_long %>%
  select(requested_period, dataset_year, year_note,
         land_cover_class, area_pct) %>%
  pivot_wider(
    names_from = land_cover_class,
    values_from = area_pct,
    names_glue = "pct_{land_cover_class}"
  ) %>%
  arrange(dataset_year)

readr::write_csv(
  landcover_trends_table_long,
  file.path(results_dir, "Tharaka_Landcover_Trends_Table_Long.csv")
)

readr::write_csv(
  landcover_trends_table_wide_ha,
  file.path(results_dir, "Tharaka_Landcover_Trends_Table_Hectares.csv")
)

readr::write_csv(
  landcover_trends_table_wide_pct,
  file.path(results_dir, "Tharaka_Landcover_Trends_Table_Percent.csv")
)

cat("\nLand-cover area (hectares):\n")
```

```
##
## Land-cover area (hectares):
```

```
print(landcover_trends_table_wide_ha, width = Inf)
```

```
## # A tibble: 4 x 7
##   requested_period dataset_year year_note                        ha_Forest
##              <int>        <int> <chr>                                <dbl>
## 1             2000         2001 2001 source year; proxy for 2000    119.
## 2             2010         2010 Exact source year                     0
## 3             2020         2020 Exact source year                    38.2
## 4             2025         2024 2024 source year; proxy for 2025      4.67
##   ha_Grassland ha_Agriculture ha_Settlement
##          <dbl>          <dbl>         <dbl>
## 1      102588.         16330.             0
## 2      105861.         12116.             0
## 3      105516.         13399.             0
## 4      104526.         14180.             0
```

```
cat("\nLand-cover share of mapped area (%):\n")
```

```
##
## Land-cover share of mapped area (%):
```

```
print(landcover_trends_table_wide_pct, width = Inf)
```

```
## # A tibble: 4 x 7
##   requested_period dataset_year year_note                        pct_Forest
##              <int>        <int> <chr>                                 <dbl>
## 1             2000         2001 2001 source year; proxy for 2000    0.0982
## 2             2010         2010 Exact source year                   0
## 3             2020         2020 Exact source year                   0.0315
## 4             2025         2024 2024 source year; proxy for 2025    0.00386
##   pct_Grassland pct_Agriculture pct_Settlement
##           <dbl>           <dbl>          <dbl>
## 1          84.7            13.5              0
## 2          87.4            10.0              0
## 3          87.1            11.1              0
## 4          86.3            11.7              0
```

```
# ============================================================
# SECTION 36: LINE GRAPH OF LAND-COVER AREA TRENDS
# ============================================================

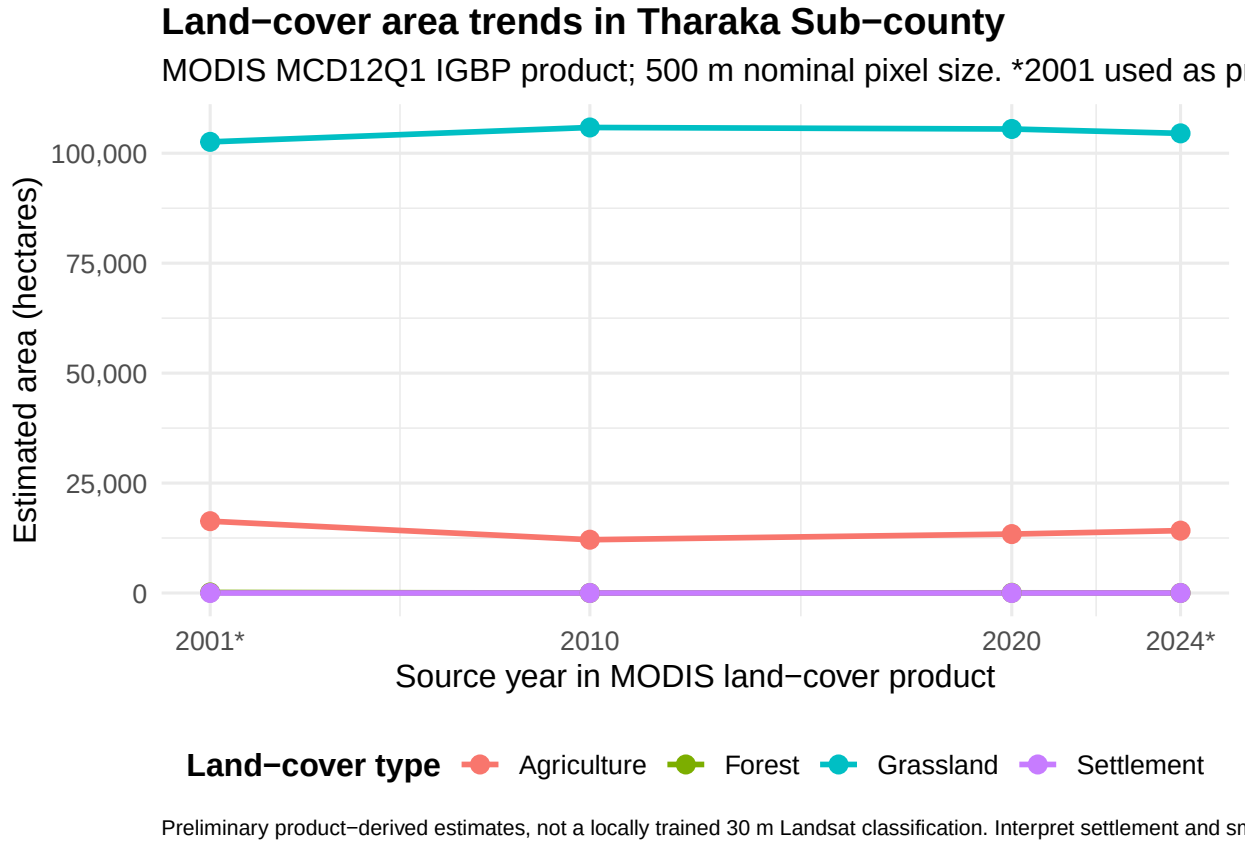
plot_data <- landcover_area_trends %>%
  filter(land_cover_class %in% requested_classes)

x_breaks <- landcover_year_map$dataset_year
x_labels <- ifelse(
  landcover_year_map$requested_year == landcover_year_map$dataset_year,
  as.character(landcover_year_map$dataset_year),
  paste0(
    landcover_year_map$dataset_year,
    "*"
  )
)

p_landcover_trends <- ggplot(
  plot_data,
  aes(
    x = dataset_year,
    y = area_ha,
    colour = land_cover_class,
    group = land_cover_class
  )
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.8) +
  scale_x_continuous(
    breaks = x_breaks,
    labels = x_labels
  ) +
  scale_y_continuous(
    labels = scales::label_comma()
  ) +
  labs(
```

```
    title = "Land-cover area trends in Tharaka Sub-county",
    subtitle = paste(
      "MODIS MCD12Q1 IGBP product; 500 m nominal pixel size.",
      "*2001 used as proxy for 2000; 2024 used as proxy for 2025."
    ),
    x = "Source year in MODIS land-cover product",
    y = "Estimated area (hectares)",
    colour = "Land-cover type",
    caption = paste(
      "Preliminary product-derived estimates, not a locally trained",
      "30 m Landsat classification. Interpret settlement and small",
      "land-cover patches cautiously at 500 m resolution."
    )
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold"),
    legend.position = "bottom",
    legend.title = element_text(face = "bold"),
    plot.caption = element_text(size = 8, hjust = 0)
  )

print(p_landcover_trends)
```




```
ggsave(
  filename = file.path(
```

```
    results_dir,
    "Tharaka_Landcover_Trends_Line_Graph.png"
  ),
  plot = p_landcover_trends,
  width = 10,
  height = 6,
  units = "in",
  dpi = 300,
  bg = "white"
)

ggsave(
  filename = file.path(
    results_dir,
    "Tharaka_Landcover_Trends_Line_Graph.pdf"
  ),
  plot = p_landcover_trends,
  width = 10,
  height = 6,
  units = "in",
  bg = "white"
)

cat(
  "\nTrend outputs saved in: ",
  normalizePath(results_dir, winslash = "/", mustWork = TRUE),
  "\n",
  sep = ""
)
```

```
##
## Trend outputs saved in: C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Land_Cover_Results
```

```
# ============================================================
# DIAGNOSTIC: VERIFY MODIS URBAN / BUILT-UP PIXELS
# ============================================================

settlement_pixel_check <- purrr::map_dfr(
  seq_len(nrow(landcover_year_map)),
  function(i) {

    requested_year <- landcover_year_map$requested_year[i]
    dataset_year <- landcover_year_map$dataset_year[i]

    annual_collection <- ee_method(
      modis_lc_collection,
      "filterDate",
      sprintf("%04d-01-01", dataset_year),
      sprintf("%04d-01-01", dataset_year + 1L)
    )

    image <- ee_method(annual_collection, "first")
    lc <- ee_method(image, "select", "LC_Type1")
```

```
    # IGBP class 13 = urban and built-up
    urban_mask <- ee_method(lc, "eq", 13L)
    urban_mask <- ee_method(urban_mask, "selfMask")
    urban_mask <- ee_method(urban_mask, "rename", "urban_pixel")

    count_result <- ee_method(
      urban_mask,
      "reduceRegion",
      reducer = ee$Reducer$count(),
      geometry = roi,
      scale = 500,
      maxPixels = 1e10,
      tileScale = 4
    )

    count_info <- ee_get_info(count_result)
    pixel_count <- count_info[["urban_pixel"]]

    tibble::tibble(
      requested_year = requested_year,
      dataset_year = dataset_year,
      urban_class_pixels = if (is.null(pixel_count)) {
        0
      } else {
        as.numeric(pixel_count)
      }
    )
  }
)

print(settlement_pixel_check, n = Inf)
```

```
## # A tibble: 4 x 3
##   requested_year dataset_year urban_class_pixels
##            <int>        <int>              <dbl>
## 1           2000         2001                  0
## 2           2010         2010                  0
## 3           2020         2020                  0
## 4           2025         2024                  0
```

```
readr::write_csv(
  settlement_pixel_check,
  file.path(
    results_dir,
    "Tharaka_MODIS_Urban_Pixel_Diagnostic.csv"
  )
)
```

11 34. Optional labelled Landsat classification (disabled by default)

This labelled Random Forest approach is retained for reference only and is disabled by default in this image-led version. Since historical labels are hard to assign confidently for every year, Section 38 instead discovers

spectral/seasonal clusters from the seasonal Landsat imagery, then asks for a transparent review of the cluster-to-cover interpretation before producing named land-cover trends.

11.0.1 Optional alternative: labelled Random Forest

The label-led Random Forest code in Section 34 is retained in this document for reference, but it is not run by default in this image-led version. Set its chunk option `eval=TRUE` only if you later decide to use independently reviewed year-specific labels.

11.0.2 Interpretation note for the supervised Landsat results

The supervised maps use the supplied labels exactly as provided. The current label file contains 50 unique locations repeated across the four years, and each of those locations has the same class label in all four years. If this reflects independent visual review and truly stable locations, the labels can be used as entered; if labels were carried forward for convenience, review them independently by year before interpreting change. Only four forest points and five farmland points are present in the supplied point sample, so per-class accuracy for those classes will be uncertain. Add more well-distributed independent reference locations before using these results for high-stakes land management decisions.

12 36. Export seasonal Landsat composites for image-led learning

If the 16 seasonal composites are not already saved as local multiband GeoTIFFs, run this optional chunk interactively after the earlier Earth Engine sections have created `selected_composites`. It submits one Google Drive export task for each year/window. Download the resulting `Tharaka_Seasonal_*.tif` files into `01_data/landsat/qgis_input`, then rerun Section 40 to clip them with QGIS and Section 38 to learn clusters and calculate trends. The export switch is deliberately off by default.

13 37. Local raster processing with QGIS from R

This section uses `qgisprocess` to repair the Tharaka boundary geometry and, when GeoTIFF files have been downloaded locally, clip those rasters to the corrected study-area boundary. Earth Engine images must first be exported and downloaded; `qgisprocess` cannot operate directly on server-side Earth Engine image objects.

```
## QGIS version: 3.44.15-Solothurn

## <Result of `qgis_run_algorithm("native:fixgeometries", ...)`>
## List of 1
##  $ OUTPUT: 'qgis_outputVector' chr "01_data/boundary/qgisprocess/Tharaka_SubCounty_QGIS_fixed.gpkg"

## QGIS boundary repair passed. Features: 1

## Corrected boundary: C:/Users/pmwaura/DSAS/Dr. Peris/01_data/boundary/qgisprocess/Tharaka_SubCounty_Q

## QGIS processing log: C:/Users/pmwaura/DSAS/Dr. Peris/02_outputs/tables/Tharaka_QGIS_Processing_Log.cs
```

14 38. Image-led unsupervised land-cover learning in R

This is the primary image-led analysis path in this version of the workflow. It does **not** train on the historical point labels. It uses the local QGIS-clipped, four-season Landsat composites to discover recurring spectral/seasonal pixel groups. One pooled clustering model is fitted across all four years so the cluster IDs remain comparable across years.

The first run produces cluster maps, cluster signatures, and annual area/share plots for the clusters. These are exploratory clusters, not automatically confirmed land-cover names. Review the maps and spectral signatures, complete the generated `Tharaka_Cluster_to_LandCover_Crosswalk.csv`, and rerun this section

to create the named land-cover trend plots. This avoids pretending that an unsupervised algorithm knows which cluster is farmland or bushland without interpretation.

Input requirement: one multiband GeoTIFF per year and seasonal window (16 files total), with the first six bands in this order: blue, green, red, NIR, SWIR1, SWIR2. Filenames must include a target year (`2000`, `2010`, `2020`, or `2025`) and a month/season token (`Feb`, `Sep`, `May`, or `Dec`). If these TIFFs do not exist locally yet, run Section 36 interactively, download the 16 completed exports from Google Drive into `01_data/landsat/qgis_input`, and rerun the document so Section 40 can clip them with QGIS. The code checks the input inventory before processing.

```
## GeoTIFF search folders:
##  01_data/landsat/qgis_clipped
## 01_data/landsat/qgis_input
##
## GeoTIFF files found:
##  (none)
```

```
## Warning: Section 38 has been skipped because the 16 seasonal GeoTIFFs are not
## all available. If needed, run Section 36 interactively, download the completed
## seasonal exports, place them in 01_data/landsat/qgis_input, then rerun the
## document.
```

14.1 Interpretation and validation note

This is an exploratory, image-derived classification. Cluster trends are quantitative patterns in spectral/seasonal groups; they become named land-cover trends only after the crosswalk has been reviewed against the imagery. A crosswalk reviewed by the analyst is not an independent accuracy assessment. For stronger reporting, validate mapped classes with a separate, year-appropriate sample and report class-specific accuracy and area uncertainty. Compare scene quality, cloud/no-data coverage and mapped-area totals before interpreting a change as real land-cover conversion.